

Informática Industrial

Curso 2019-2020

Escuela de Ingeniería Industrial y Aeroespacial de
Toledo

Autor

Juan Moreno García



Índice general

1. Introducción	2
1.1. Introducción a SCADA	2
1.2. Arquitectura de un sistema SCADA	3
1.3. Elementos hardware de un SCADA	5
1.3.1. Interfase Hombre-Máquina (HMI, MMI)	6
1.3.2. Unidad central (MTU, Master Terminal Unit)	6
1.3.3. Unidad Remota (RTU, Remote Terminal Unit)	7
1.3.4. Sistema de comunicación	8
1.4. Tecnologías software en un SCADA	9
1.4.1. Comunicación entre aplicaciones	10
1.4.2. Almacenamiento de datos	11
1.5. Módulos	12
1.6. Criterios de Selección y Diseño	13
1.6.1. Disponibilidad	13
1.6.2. Robustez	15
1.6.3. Seguridad	15
1.6.4. Prestaciones	16
1.6.5. Mantenibilidad	16
1.6.6. Escalabilidad	17
2. Arquitectura del Computador	20
2.1. Fundamentos de Computadores.	20
2.1.1. Estructura básica del computador.	20
2.1.2. Elementos Internos de un Procesador.	21

2.1.3. Temporalización en la ejecución de la instrucción.	22
2.1.4. Unidad Aritmético-Lógica.	22
2.1.5. Unidad de Control.	26
2.1.6. Memoria Principal.	26
2.1.7. Organización de Entrada/Salida.	27
2.2. Organización de E/S.	28
2.3. Periféricos Industriales.	29
2.3.1. Introducción.	29
2.3.2. Características generales de los periféricos.	30
2.3.3. Clasificación de los periféricos.	31
2.3.4. Conexiones de periféricos a un PC standard.	32
2.3.5. Protocolo RS-232.	34
2.3.6. RS-232 en el PC.	36
3. Sistemas de Tiempo Real	44
3.1. Introducción.	44
3.2. Ejemplos.	45
3.3. Características de los STR.	46
3.4. Diseño de STR.	49
3.4.1. Especificación de requisitos.	50
3.4.2. Diseño arquitectónico y Diseño Detallado.	50
3.4.3. Implementación.	52
3.4.4. Prueba.	55
3.5. Fiabilidad y tolerancia a fallos.	56
3.5.1. Fiabilidad.	56
3.5.2. Técnicas para la fiabilidad.	58
3.5.2.1. Prevención de fallos.	59
3.5.2.2. Tolerancia a fallos.	59
3.6. Planificación de STR.	66
3.6.1. Introducción.	66
3.6.1.1. Interfaz con el tiempo.	66
3.6.1.2. Representación de requisitos temporales.	68

3.6.2. Planificación.	70
3.6.2.1. Modelo de proceso simple.	70
3.6.2.2. Planificación basada en procesos.	72
3.6.2.3. Test de planificabilidad basados en la utilización.	73
4. Redes de Comunicaciones	76
4.1. Introducción.	76
4.2. Modelo de Referencia OSI de ISO.	77
4.3. Redes de Área Local.	82
4.3.1. Arquitectura de Red.	82
4.3.2. Tecnología Ethernet.	83
4.3.2.1. Introducción.	83
4.3.2.2. Estándares de redes de área local.	84
4.3.2.3. Métodos de Acceso al Medio.	85
4.3.2.4. Control de Enlace Lógico (LLC).	88
4.4. Protocolos de nivel de Red/Transporte. El protocolo TCP/IP.	88
4.4.1. Introducción.	88
4.4.2. Arquitectura de los Protocolos TCP/IP.	89
4.4.3. Direccionamiento y subdireccionamiento.	91
4.4.3.1. Asignación de direcciones.	93
4.4.3.2. Direccionamiento CIDR.	95
4.4.3.3. Direccionamiento por dominios.	96
4.4.3.4. Puertos y Sockets en TCP/IP.	96
4.4.3.5. Datagramas IP y tramas TCP y UDP	97
Bibliografía	101

1

Introducción

En este tema se definirá y se presentarán los sistemas SCADA. Se detallará la arquitectura de los mismos, así como los elementos hardware que los componen. Se expondrán las tecnologías software utilizadas para la comunicación entre aplicaciones y para el almacenamiento de datos relevantes para el sistema SCADA. Se describirá su diseño modular típico, y, finalmente, se especificarán los criterios a considerar en la selección y diseño del SCADA. Ha sido sacado en su mayor parte del libro con referencia [5].

1.1. Introducción a SCADA

SCADA proviene de las siglas Supervisory Control And Data Acquisition (Adquisición de datos y supervisión de control). Es una aplicación software de control de producción, que se comunica con los dispositivos de campo y controla el proceso de forma automática desde el ordenador. Proporciona:

- Información del proceso a diversos usuarios: operadores, supervisores de control de calidad, supervisión, mantenimiento, etc.
- Acceso a datos remotos de un proceso y control del mismo.
- Software de monitorización y/o supervisión: realiza la tarea de interfase entre los niveles de control y los de gestión a un nivel superior.

Por prestación de un sistema SCADA se entiende como el conjunto de funciones y utilidades encaminadas a establecer una comunicación entre el proceso y el operador. Estas prestaciones son:

Adquisición de datos de los procesos: Herramientas registradoras que almacenan la información obtenida para poder evaluarla con posterioridad.

Supervisión: Vigilar la evolución del proceso para que el operario pueda tomar las decisiones apropiadas.

Mando: Permitir el control por parte de los operadores, cambiando consignas u otros datos del proceso (marcha, paro, modificación de parámetros, etc). Se escriben valores sobre los elementos de control.

Monitorización: Representación de datos en tiempo real a los operadores de planta. Se leerán los datos de los sensores (temperaturas, velocidades, detectores...).

Seguridad de los datos: Proteger el envío y la recepción de datos de influencias no deseadas, intencionadas o no, por ejemplo: fallos en la programación, intrusos, situaciones inesperadas, etc.

Seguridad en los accesos: Garantizar los accesos y acciones llevadas a cabo por cualquier operador, restringiendo zonas de programa comprometidas a usuarios no autorizados.

Grabación de recetas: En procesos que lo necesitan se utilizan combinaciones de variables que son siempre las mismas. Un sistema de recetas permite configurar toda una planta de producción ejecutando un solo comando. Por ejemplo, la línea de vulcanizado en continuo (donde fabrican los perfiles de goma de las ventanas, por ejemplo) se compone de varias máquinas encadenadas con múltiples parámetros (velocidad y temperatura principalmente) que dependen del tipo de perfil a elaborar (la goma más ancha, más estrecha, con forma más o menos compleja, etc.). En este caso, con una sola pulsación se pueden poner en marcha todas las máquinas y programar las diferentes zonas de temperatura o velocidad de toda la línea del ejemplo.

Posibilidad de programación numérica: Permite realizar cálculos aritméticos sobre la CPU del ordenador mediante lenguajes de alto nivel, generalmente C o Visual Basic.

1.2. Arquitectura de un sistema SCADA

El sistema consta de tres bloques principales (Figura 1.1):

- Software de adquisición de datos y control (SCADA).
- Sistemas de adquisición y mando.
- Sistema de interconexión.

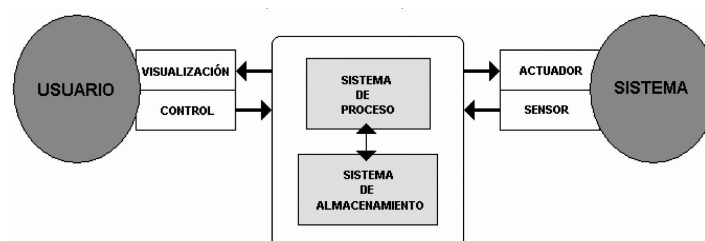


Figura 1.1: Estructura básica de un sistema de supervisión y mando.

Software de adquisición de datos y control (SCADA): Software que integra el mundo de las máquinas en la red empresarial, pasando a formar parte de los elementos que permitirán crear estrategias de empresa globales, esto se denomina “Fabricación Integral Informatizada” (Computer Integrated Manufacturing) (Figura 1.2). Un sistema SCADA es una aplicación de software especialmente diseñada para funcionar sobre ordenadores en el control de producción que proporciona comunicación entre los dispositivos de campo, llamados también RTU (Remote Terminal Units), donde se pueden encontrar elementos tales como controladores autónomos o autómatas programables, y un centro de control o Unidad Central (MTU, Master Terminal Unit), donde se controla el proceso de forma automática desde la pantalla de uno o varios ordenadores.

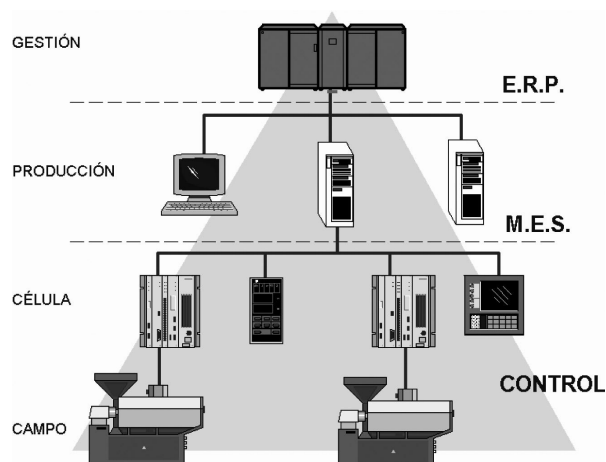


Figura 1.2: Fabricación Integral Informatizada.

Sistemas de adquisición y mando: Son los equipos informáticos (PCs), las redes de comunicaciones, los sensores y los actuadores. En un PC reside la aplicación de control y supervisión (sistema servidor). El usuario tiene acceso al Sistema de Control de Proceso mediante software de visualización y control. Las Redes de comunicaciones (Ethernet) se utilizan para la comunicación entre el sistema de proceso y las herramientas Interfase Hombre-Máquina (HMI). Los sensores captan el estado del sistema, y en base a sus valores el usuario ejecuta los comandos para que el sistema de Proceso inicie las acciones pertinentes para mantener el control del sistema a través de los elementos actuadores.

Sistema de interconexión (comunicaciones): La transmisión de los datos entre el Sistema de Proceso y los elementos de campo (sensores y actuadores) se lleva a cabo generalmente mediante los denominados buses de campo. La tendencia actual es englobar los sistemas de comunicación en una base común, como Ethernet Industrial. Toda la información generada durante la ejecución de las tareas de supervisión y control se almacena para disponer de los datos a posteriori.

El sistema de visualización y adquisición de datos se realiza generalmente mediante el modelo Maestro-Eslavo. La estación central (el maestro o master) se comunica con el resto de estaciones (esclavos o slaves) requiriendo de éstas una serie de acciones o datos.

1.3. Elementos hardware de un SCADA

Conceptualmente el hardware de un SCADA está dividido en:

1. Captadores de datos: Recopilan los datos de los elementos de control del sistema (autómatas, reguladores, registradores) y los procesan para su utilización. Son los servidores del sistema.
2. Utilizadores de datos: Utilizan la información recogida por los anteriores. Son los clientes.

Los datos residentes en los servidores pueden evaluarse por los clientes, permitiendo realizar las acciones oportunas para mantener las condiciones del sistema. Los Controladores de Proceso (Figura 1.3) envían la información a los Servidores de Datos a través de los buses de campo, los cuales, a su vez, intercambian la información con niveles superiores del sistema automatizado a través de redes de comunicaciones de Área Local.

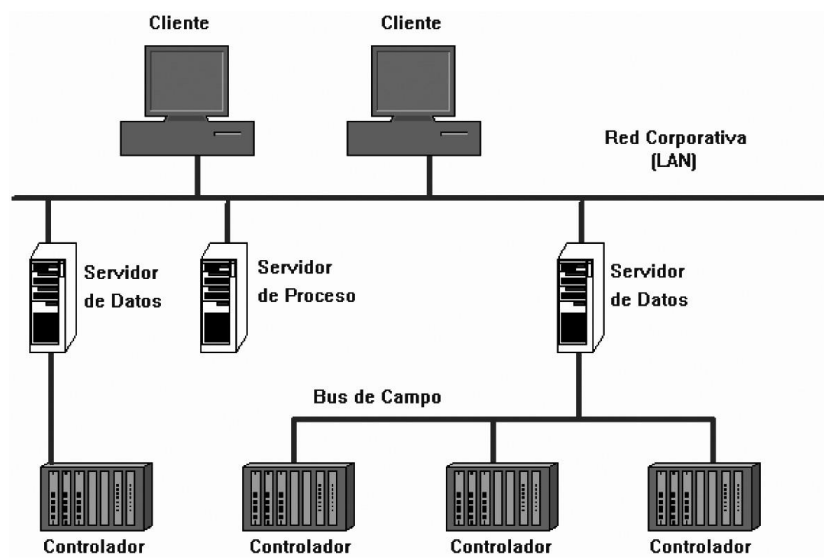


Figura 1.3: SCADA, arquitectura básica de hardware.

Existen múltiples posibilidades de implementación de sistemas SCADA, desde una máquina aislada hasta un gran conjunto de sistemas interconectados. Todas estas implementaciones están formadas por los siguientes elementos hardware:

- Interfase Hombre-Máquina.
- Unidad Central (MTU).
- Unidad Remota (RTU).
- Sistema de Comunicaciones.

Las siguientes subsecciones detallan estos componentes.

1.3.1. Interfase Hombre-Máquina (HMI, MMI)

Su función es la de representar de forma simplificada el sistema bajo control, son los sinópticos de control y los sistemas de presentación gráfica.

Inicialmente, los paneles sinópticos eran de tipo estático, colocados en grandes paneles plagados de indicadores y luces. Con el tiempo han ido evolucionando, junto al software, en forma de representaciones gráficas en pantallas de visualización. En los sistemas complejos aparecen los terminales múltiples, que permiten la visualización, de forma simultánea, de varios sectores del sistema. En ciertos casos interesa mantener la forma antigua del Panel Sinóptico, ya que la representación del sistema completo es más clara para el usuario.

1.3.2. Unidad central (MTU, Master Terminal Unit)

Es el terminal o terminales que centraliza el mando del sistema, y utilizan protocolos abiertos que permiten la interoperabilidad de multiplataformas y multisistemas necesaria en estos sistemas. Un sistema de este tipo debe estar basado en estándares asequibles a bajo precio para cualquier parte interesada. De esta manera es posible intercambiar información en tiempo real entre el MTU y subestaciones situadas en cualquier lugar. El MTU suele realizar la recopilación y archivado de datos, se encarga de:

- Mando.
- Gestionar las comunicaciones.
- Recopilar los datos de todas las estaciones remotas (RTU).
- Envío de información.
- Comunicación con los operadores.
- Seguridad.
- Análisis.
- Impresión.
- Visualización de datos.

Estas tareas las realizan equipos informáticos con funciones específicas y exclusivas:

- Administración: Permite la gestión y el mantenimiento del sistema SCADA, controlar los sistemas de seguridad, modificar la configuración de las tareas de backup, etc.
- Comunicaciones: Permite el intercambio de datos en tiempo real con los RTUs. Éste es un punto de entrada y salida de datos, por tanto, debe prestarse especial atención a la seguridad.
- Almacenar datos (Database Server): Se ocupa del archivado de datos para el proceso posterior de los mismos.
- Almacenar archivos (File Server): Almacena los resultados de los análisis de los datos recogidos, guarda los datos concernientes a los eventos del sistema, datos de configuraciones, alarmas, etc.

1.3.3. Unidad Remota (RTU, Remote Terminal Unit)

Es el conjunto de elementos dedicados a labores de control y/o supervisión de un sistema, están alejados del MTU y comunicados con éste mediante algún canal de comunicación. Se clasifican en (algunos elementos caen en varias categorías):

RTU (Remote Terminal Unit) - Especializados en comunicación. Se encargaban en un principio de recopilar los datos de los elementos de campo y transmitirlos hacia el MTU, junto con los comandos de control a éstos, son los denominados Procesadores de Comunicaciones. Con la introducción de sistemas inteligentes aparecen también las funciones de recogida y proceso de datos, así como de seguridad ante accesos sin autorización o situaciones anómalas que puedan perjudicar al funcionamiento de la estación y provocar daños en sus componentes. Suelen estar basados en ordenadores especiales que controlan directamente el proceso mediante tarjetas convertidoras adecuadas o que se comunican con los elementos de control (PLC, Reguladores) mediante los protocolos de comunicación adecuados. Su construcción es más robusta, son operativos dentro de un rango de temperaturas mayor que los ordenadores normales, y su robustez eléctrica también es mayor (transitorios de red, variaciones de alimentación, interferencias electromagnéticas). El software de estos elementos suele estar elaborado en lenguajes de alto nivel (C, VisualBasic, Delphi) que permiten interpretar los comandos provenientes de la estación Maestra (Master Terminal Unit).

PLC (Programmable Logic Controller) - Tareas generales de control.

Es una computadora utilizada en la automatización industrial para automatizar procesos electromecánicos. A diferencia de las computadoras de propósito general, el PLC está diseñado para múltiples señales de entrada y de salida, rangos de temperatura ampliados, inmunidad al ruido eléctrico y resistencia a la vibración y al impacto. Los programas para el control de funcionamiento de la máquina se suelen almacenar en baterías de copia de seguridad o en memorias no volátiles. Empezaron como sistemas de dedicación exclusiva al control de instalaciones, máquinas o procesos. Con el tiempo han ido evolucionando, incorporando cada vez más prestaciones en forma de módulos de ampliación, entre ellos los Procesadores de Comunicaciones, que han hecho desvanecerse la línea divisoria entre RTU y PLC, quedando incluidas todas las prestaciones en el PLC. A su vez, los PLC pueden tener elementos distribuidos con los cuáles se comunican a través de sistemas de comunicación llamados Buses de Campo.

IED (Intelligent Electronic Device) - Tareas específicas de control. Se utiliza en la industria para describir los controladores basados en microprocesador del equipo del sistema de alimentación, tales como interruptores de circuito, transformadores y bancos de condensadores. Los IEDs reciben datos de los sensores y equipos de energía, y pueden emitir comandos de control, como disparo interruptores de circuito si detectan anomalías de tensión, corriente, frecuencia, o subir niveles de tensión más bajos con el fin de mantener el nivel deseado. Como puede verse disponen de programas que se ocupan de tareas de control, regulación y comunicación. Dentro de esta clasificación se pueden encontrar elementos tales como PCL, Reguladores, Variadores de Frecuencia, Registradores, Procesadores de comunicaciones, Generadores de tiempo y frecuencia, Controladores de energía reactiva, Transductores, etc. Es todavía habitual encontrar elementos de este tipo que utilizan protocolos propietarios y dan origen a las denominadas islas de automatización.

Sistemas remotos - Sistemas complejos de control. Es un gran sistema complejo que forma parte de un sistema de control mucho más extenso, por ejemplo, el control de

la distribución eléctrica de un país, donde las estaciones remotas pueden tener a su cargo una ciudad. En este caso, la estación remota tiene implementadas funciones de control, interfase hombre-máquina, adquisición de datos, control de bases de datos, protocolos de seguridad y comunicaciones internas entre subsistemas. En la Figura 1.4 se muestra un ejemplo de un sistema remoto, donde un MTU se comunica a través de Internet con el sistema remoto.

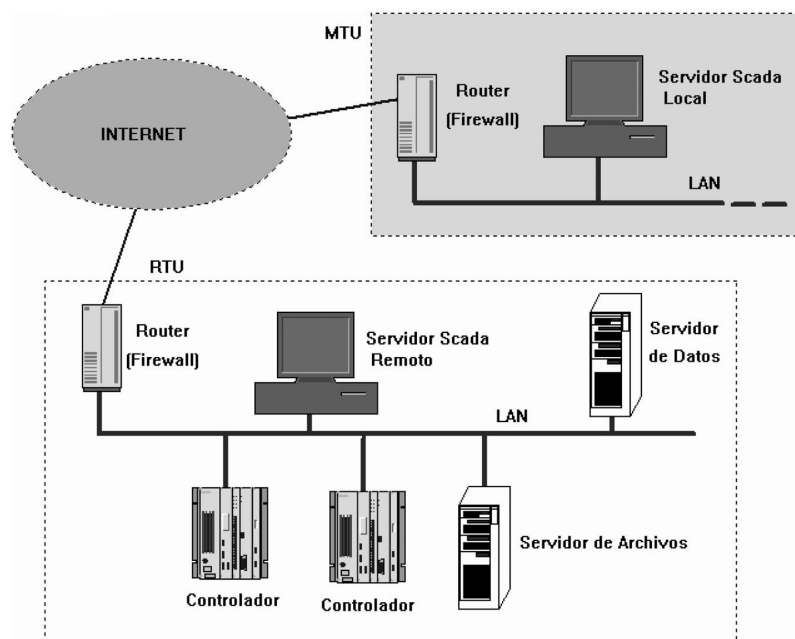


Figura 1.4: Ejemplo de Sistema Remoto.

El sistema remoto está protegido de dos maneras:

1. Por hardware: Son barreras físicas, desde la valla de protección de los recintos y los sistemas de vigilancia, hasta las llaves de las salas de control o de los armarios que contienen los elementos de mando.
2. Por software: Son barreras lógicas. Los accesos no autorizados desde dentro se evitan mediante sistemas de contraseñas en los equipos. Los accesos desde fuera, mediante dispositivos especiales que limitan el acceso (Cortafuegos o firewalls).

1.3.4. Sistema de comunicación

Los buses especiales de comunicación proporcionan al operador la posibilidad de comunicarse con cualquier punto, local o remoto, de la planta en tiempo real. Los controladores deben tener compatibilidad con los estándares de comunicación, permitiendo así la comunicación entre un servidor de datos y cualquier elemento de campo. De esta forma, un servidor de datos puede gestionar varios protocolos de forma simultánea, estando limitado por su capacidad física de soportar las interfases de hardware (tarjetas de comunicación). El protocolo de comunicaciones permite el intercambio de datos bidireccional entre el MTU y las RTUs, y un sistema de transporte de la información enlaza los diferentes elementos de

la red: Línea telefónica, cable coaxial, fibra óptica, telefonía (GPRS, UMTS), radio (enlaces de radio VHF, UHF, Microondas), etc.

El intercambio de información entre servidores y clientes se basa en la relación de productor-consumidor. Los servidores de datos interrogan de manera cíclica a los elementos de campo (*polling*) recopilando los datos generados por registradores, autómatas, reguladores de proceso, etc.

1.4. Tecnologías software en un SCADA

El programa SCADA se comunica mediante programas específicos con los dispositivos de control de planta (hacia abajo) y los elementos de gestión (hacia arriba). Al conjunto de estos programas se denominan controladores de comunicaciones. Una parte del mismo contiene los *controladores de comunicación entre la aplicación y el exterior*, ocupándose de gestionar los enlaces de comunicación, tratamiento de la información a transferir y protocolos de comunicación (Profibus, AS-i, Can, Ethernet...). Por lo general son programas de pago.

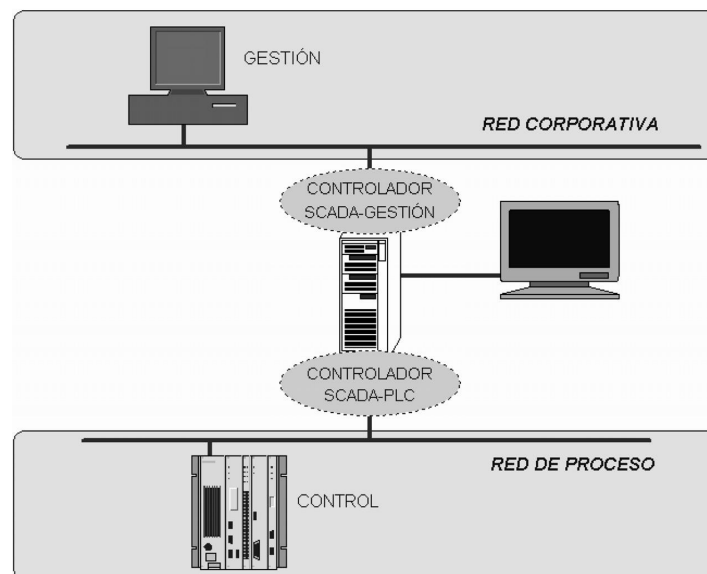


Figura 1.5: Concepto de controlador.

El controlador realiza la función de traducción entre el lenguaje del programa SCADA y los RTU (se suelen usar buses de campo, por ejemplo Profibus), o entre el SCADA y la red de gestión de la empresa (se usan tecnologías de red, por ejemplo Ethernet). Generalmente la configuración del controlador de comunicaciones se realiza durante la instalación del software principal o como programa de acceso externo al ejecutar la aplicación principal.

Respecto a la *gestión de datos*, que consiste en el análisis de la información generada en el proceso de producción (eventos, alarmas, emergencias, etc.), existen dos bloques software de gestión de datos:

- Programa de Desarrollo: engloba las utilidades relacionadas con la creación y edición

de las diferentes ventanas de la aplicación, así como sus características (textos, dibujos, colores, propiedades de los objetos, programas, etc.).

- Programa Run-time: permite ejecutar la aplicación creada con el programa de desarrollo (en Industria se entrega, como producto acabado, el Run-time y la aplicación).

1.4.1. Comunicación entre aplicaciones

Los métodos de intercambio de información entre aplicaciones informáticas más destacados son:

OPC (OLE for Process Control): Es el estándar de intercambio de datos por excelencia y es un estándar abierto que permite un método fiable para acceder a los datos desde aparatos de campo. El método de acceso siempre es el mismo con independencia del tipo y origen de los datos. Se basa en la tecnología COM (Component Object Model) de Microsoft que permite definir cualquier elemento de campo mediante sus propiedades, convirtiéndolo en una interfase. De esta manera es posible conectar fácilmente cualquier elemento de campo con un servidor de datos local (COM), o remoto (DCOM). Los componentes OPC se pueden clasificar en:

- Cliente OPC: Es una aplicación que sólo utiliza datos, tal como hace un paquete SCADA. Cualquier cliente OPC se puede comunicar con cualquier servidor OPC sin importar el tipo de sistemas de comunicaciones que recoge esos datos.
- Servidor OPC: Es una aplicación que realiza la recopilación de datos de los diversos elementos de campo de un sistema automatizado y permite el acceso libre a estos elementos desde los clientes OPC.

ODBC: Un estándar que permite a las aplicaciones el acceso a datos en Sistemas de Gestión de Bases de Datos (Data Base Management Systems) utilizando SQL como método estándar de acceso y es también de Microsoft. Permite que una aplicación pueda acceder a varias bases de datos mediante la inclusión del controlador correspondiente en la aplicación que debe acceder a los datos.

SQL: Es el estándar por excelencia para la comunicación con bases de datos, y permite una interfase común para el acceso a los datos por parte de cualquier programa que cumpla el estándar. El primer SQL aparece en 1986 bajo el nombre: ANSI X3. Las posibilidades de esta tecnología incluyen:

- Procedimientos: Son bibliotecas de comandos almacenados en la base de datos. Permiten reducir el tráfico de red y simplificar los procedimientos de acceso a los usuarios de las bases de datos.
- Eventos: Son comandos que se activan de forma automática bajo ciertas condiciones, facilitando el mantenimiento de la integridad de los datos.
- Replicación: Permite la duplicación y sincronización de bases de datos. Por ejemplo, para actualizar los datos de la base de datos central con los almacenados en una RTU, o para actualizar un servidor de datos que ha quedado temporalmente fuera de servicio y se vuelve a poner en funcionamiento.
- Accesibilidad: Permite el intercambio o envío de información basándose en eventos. Por ejemplo, el envío automático de mensajes cuando se cumplen ciertas condiciones dentro de un sistema.

API: Las herramientas API (Application Programming Interfaces) permiten que el usuario pueda adaptar el sistema a sus necesidades mediante rutinas de programa propias escritas en lenguajes estandarizados, tales como Visual Basic, C++, o Java, lo cual les confiere una potencia muy elevada y una gran versatilidad. Permiten el acceso a las bases de datos de los servidores (valores almacenados temporalmente o archivos históricos).

ASCII: Mediante el formato ASCII, común a prácticamente todas las aplicaciones informáticas, se tiene un estándar básico de intercambio de datos. Es sencillo exportar e importar datos de configuración, valores de variables, etc.

1.4.2. Almacenamiento de datos

Inicialmente los ordenadores estaban muy limitados en sus capacidades de almacenamiento de variables, luego los métodos de almacenamiento sufrieron una evolución que se detalla a continuación:

- **Ficheros:** Se basa en el almacenamiento de información en archivos accesibles por los programadores de las aplicaciones. Estos ficheros eran complicados de tratar debido a que tenían que estar perfectamente identificados y localizados en el disco, así como la situación y el formato de los datos dentro de éstos. La primera revolución aparece con la técnica del indexado, evitando así el acceso secuencial. Un archivo puede entonces estar ordenado por un criterio determinado, por ejemplo, la fecha o el nombre de variable. De esta manera es fácil acceder a unos datos si el nombre de la variable es conocido. La limitación de este método radica en que la base de datos tiene un solo punto de acceso.
- **Bases de datos Jerárquicas:** Permiten ordenar los elementos por jerarquías en las cuáles un tipo de datos consiste en un subconjunto de otro tipo de datos más genérico. Este modelo está limitado en prestaciones si se quiere acceder, por ejemplo, a variables pertenecientes a distintos grupos de datos situados en diferentes niveles del esquema de variables. Surgen entonces las bases de datos de red, capaces de interpretar las relaciones más complejas entre los diversos tipos de variables que aparecen. Los programas, de todas formas, siguen necesitando conocer las formas de acceder a los datos dentro de estas estructuras.
- **Bases de datos relacionales:** El paso definitivo se da con la aparición de las bases de datos relacionales. Este tipo de bases de datos permite reflejar estructuras de datos, independientemente del tipo de programas que accede a los datos o de la estructura de éstos. Una base de datos relacional es un conjunto de tablas de datos que contienen campos que sirven de nexo de unión (relación) y que permiten establecer múltiples combinaciones mediante la utilización de estos nexos. Las combinaciones posibles son prácticamente ilimitadas, sólo hay que configurar el método de búsqueda o el tipo de datos que se quiere consultar y aplicarlo a los datos. Simplifica la administración de los datos y los programas que trabajan con éstos. También disminuye las necesidades de espacio de almacenamiento y reduce los problemas asociados a las bases de datos redundantes (inconsistencias debidas a repeticiones de registros).
- **Bases de datos industriales:** Las bases de datos relacionales normales no son adecuadas para los sistemas actuales de producción. Las limitaciones principales son:
 - La cantidad de datos a almacenar en un periodo dado de tiempo.

- El espacio necesario es considerable debido a la cantidad de información a almacenar.
- SQL no está optimizado para trabajar con datos con indexación temporal, lo cual hace difícil la tarea de especificar resoluciones temporales.

Las nuevas técnicas desarrolladas permiten aumentar el rendimiento de las bases de datos y, por tanto, el acceso a la información.

1.5. Módulos

Los módulos software más habituales de un paquete SCADA desde la perspectiva del programador son:

Configuración: Permite definir el entorno de trabajo para adaptarlo a las necesidades de la aplicación/usuario. Puede gestionar opciones relativas a organización de pantallas, usuarios, realización de tareas, alarmas, recetas, etc.

Interfase gráfica: Permiten la elaboración de pantallas de usuario con múltiples combinaciones de imágenes y/o textos, definiendo así las funciones de control y supervisión de planta.

Tendencias: Son las utilidades que permiten representar de forma cómoda la evolución de variables del sistema. Las utilidades más generales son relativas a visualización de variables del sistema: representación en tiempo real de variables (Real-time trending), recuperación de variables almacenadas (Historical Trending), visualización de valores, desplazamiento a lo largo de todo el registro histórico (scroll), ampliación y reducción de zonas concretas de una gráfica, etc.

Alarmas y eventos: Las alarmas se basan en la vigilancia de los parámetros de las variables del sistema. Son los sucesos no deseables, porque su aparición puede dar lugar a problemas de funcionamiento. Este tipo de sucesos requiere la atención de un operario para su solución antes de que se llegue a una situación crítica que detenga el proceso. El resto de situaciones, consideradas normales, serán los denominados eventos del sistema o sucesos. Los eventos no requieren de la atención del operador del sistema, registran de forma automática todo lo que ocurre en el sistema.

Registro y archivado: Por registro (logging) se entiende el archivo temporal de valores, generalmente basándose en un patrón cíclico y limitado en tamaño. Por ejemplo, se puede definir un archivo histórico de alarmas de manera que almacene en disco duro hasta mil alarmas de forma consecutiva. También será posible definir que, una vez el registro esté lleno, se guarde una copia en un archivo, quedando a disposición del usuario que necesite recuperar esos datos. Estos datos suelen ir acompañados de más identificadores (Time Stamp, el usuario activo en ese momento, etc).

Generación de informes: Mediante las herramientas SQL es posible realizar extractos de los archivos, los registros o las bases de datos del sistema, realizar operaciones de clasificación o valoración sin afectar a los datos originales. También permiten presentar los archivos en forma de informes o transferirlos a otras aplicaciones mediante las herramientas de intercambio disponibles.

Control de proceso: Lenguajes de alto nivel, como Visual Basic, C o Java, incorporados en los paquetes SCADA, permiten programar tareas que respondan a eventos del sistema, tales como enviar un correo electrónico al activarse una alarma concreta, un mensaje a un teléfono móvil del servicio de mantenimiento, o poner en marcha o detener partes del sistema en función de los valores de las variables adquiridas.

Recetas: Son archivos que guardan los datos de configuración de los diferentes elementos del sistema (velocidad de proceso, presiones, temperaturas, niveles de alarma, cantidades de piezas, etc.). Así, el procedimiento de cambiar la configuración de trabajo de toda una planta de proceso quedará reducido al simple hecho de pulsar un botón después de confirmar unos datos de acceso (usuario, contraseña y número o nombre de receta, por ejemplo). El sistema SCADA se encargará de enviar los datos a los correspondientes controladores, quedando la planta lista para las nuevas condiciones de trabajo.

Comunicaciones: Soporta el intercambio de información entre los elementos de planta, la arquitectura de hardware implementada y los elementos de gestión. Permite implementar el sistema de controladores que realizarán el intercambio de información entre los elementos de campo (autómatas, reguladores, etc) y los ordenadores que realizarán la recopilación de datos de información. La conexión se realizará de dos maneras:

1. Controladores específicos: sólo permiten la comunicación entre un elemento determinado de campo y un sistema de captación de datos (ordenador). Para cada enlace se necesita un controlador determinado.
2. Controladores genéricos: Son controladores de tipo abierto. Están hechos en base a unas especificaciones concretas y de dominio público, cuya idea básica es definir una interfase estándar entre elementos de campo y aplicaciones, independiente del fabricante, simplificando así las tareas de integración. Un buen ejemplo es la tecnología OPC.

1.6. Criterios de Selección y Diseño

Para que un sistema SCADA pueda desarrollar sus funciones correctamente es necesario un correcto diseño del mismo. Los parámetros a considerar son:

- Disponibilidad.
- Robustez.
- Seguridad.
- Prestaciones.
- Mantenibilidad.
- Escalabilidad.

1.6.1. Disponibilidad

Es la medida en la que sus parámetros de funcionamiento se mantienen dentro de las especificaciones de diseño. Se basa en el hardware y el software.

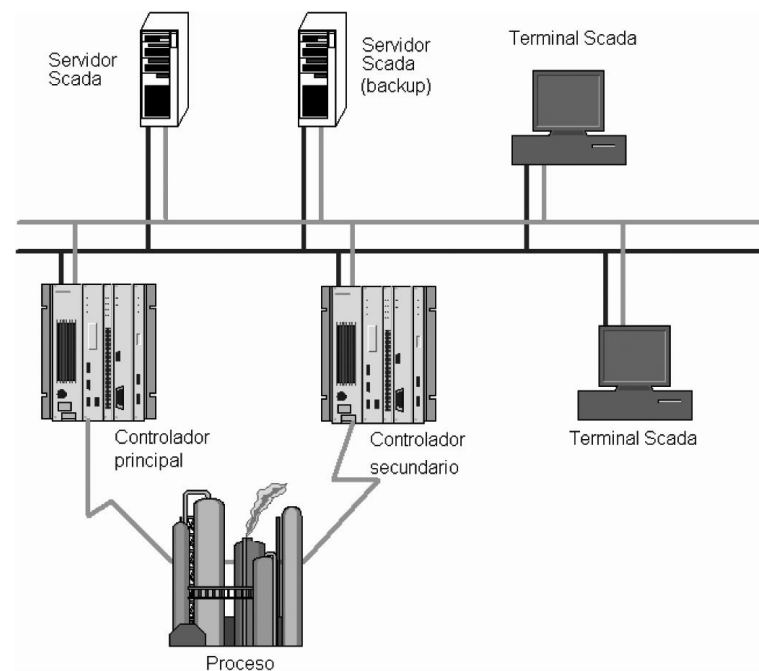


Figura 1.6: Ejemplo de redundancia.

El hardware es el elemento físico y su estrategia se fundamenta básicamente en el concepto de redundancia, entendida como la capacidad de un elemento de asumir las funciones de otro de forma transparente al sistema que sirve (Figura 1.6). El principio de redundancia se aplica a todos los niveles, desde componentes individuales hasta sistemas enteros: fuentes de alimentación, backup de datos, sistemas de comunicaciones, etc. De esta manera es posible continuar trabajando en caso de fallo de uno de los componentes. Esto aporta otra ventaja añadida, se puede realizar el mantenimiento y cambiar los componentes defectuosos sin necesidad de detener el sistema. Algunos ejemplos son:

- El SAI (Sistema de Alimentación Ininterrumpida), que ante un fallo de la tensión de red, conmuta a una alimentación auxiliar con la suficiente rapidez para que el sistema que alimenta no se vea afectado.
- En los equipos de bombeo se pueden colocar dos bombas trabajando en alternancia. Cada una trabaja durante un tiempo determinado mientras la otra está parada. También, si una falla o necesita mantenimiento, la otra puede seguir trabajando.
- En el ámbito de las comunicaciones entre equipos se puede utilizar la topología en anillo de fibra óptica (Figura 1.7). En esta configuración dos anillos concéntricos de fibra sirven de camino a la información que se intercambia entre estaciones. En funcionamiento normal el tráfico se puede repartir entre los dos anillos; mientras que en caso de rotura del anillo en cualquier punto, las estaciones más próximas a la rotura tienen la capacidad suficiente para redirigir el tráfico de un anillo a otro, evitando así la interrupción de las comunicaciones.

En general, en los equipos trabajando en paralelo uno de ellos hace de espejo. Si el

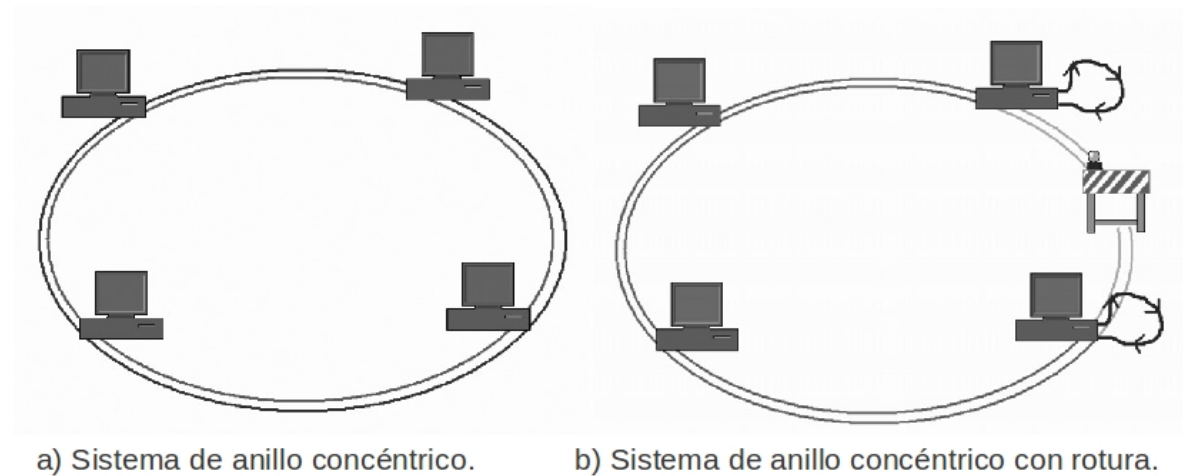


Figura 1.7: Rotura en dos anillos concéntricos con fibra.

equipo principal falla el de reserva asume sus funciones hasta que el problema se resuelve. En este momento se realiza una sincronización del equipo entrante con el suplente y queda el sistema completamente operativo. Otra posibilidad más tolerante a fallos es la que aplica la redundancia múltiple, modalidad en la cuál hay más de un equipo de reserva trabajando en segundo plano que se mantiene actualizado por si aparecen fallos en uno o más equipos.

La estrategia del software respecto a la redundancia se basa en un conjunto de especificaciones dentro de la Ingeniería del Software. Es un campo muy amplio dentro del estudio de la Informática que no es objeto de estudio en esta asignatura.

1.6.2. Robustez

Es la capacidad para mantener un nivel de operatividad suficiente como para proporcionar unos mínimos de servicio ante un fallo de diseño, accidente o intrusión. Hay que tener un plan de contingencia ante un fallo de diseño, un accidente o una intrusión.

Un sistema eficiente debe proporcionar un nivel de operatividad suficiente. Si una parte de un sistema queda aislada, accidentalmente o no, la parte aislada debe tener la suficiente capacidad de autogestión como para poder mantener un mínimo de control sobre su área de influencia. Por ejemplo, en el caso de ocurrir un fallo grave en el sistema central (MTU) puede establecerse un protocolo de desconexión de las estaciones remotas, pasando éstas al estado de autogestión (esclavos inteligentes) hasta que la Unidad Central esté de nuevo habilitada y pueda retomar el control.

1.6.3. Seguridad

Son las estrategias para prevenir, detectar y defenderse de acciones no deseadas (intencionadas o no) debido a los sistemas de comunicación utilizados por parte del sistema de control para enlazar los puntos de control de un proceso. Algunas son:

- Establecimiento de derechos y jerarquías de usuario que limitan el acceso a datos sensibles mediante contraseñas. Además, el acceso mediante usuarios, permite establecer un archivo de accesos para conocer en todo momento quién ha modificado algo en el sistema de control (log).
- Encriptación de los datos que se emiten desde las estaciones remotas (RTU) o desde el control central (MTU).
- Filtrado de la información recibida, comprobando si su origen es conocido o no, por ejemplo: (a) Mediante el uso de códigos preestablecidos que se envían con los datos y se comprueban en el centro de control antes de ser aceptados; (b) mediante las direcciones de los elementos emisores (por ejemplo, las direcciones IP).
- Fijación de caminos de acceso predeterminados a la información, provistos de las herramientas necesarias para asegurar la fiabilidad de la información que los atraviesa (puertos de acceso a un sistema).
- Detección de las incoherencias en los datos ya dentro del sistema, y reacción ante las mismas. Por ejemplo, el uso de datos predefinidos que evitan problemas en el funcionamiento normal del sistema, pudiendo provocar daños en alguno de sus componentes.
- Software de vigilancia del resto de software que ejecuta acciones predefinidas en caso de detectarse un problema (watchdog). Muchos autómatas tienen salidas que se pueden configurar para indicar una anomalía. En caso de fallo detectado en la CPU, dicha salida se activa y sirve para notificar este estado mediante un aviso.

1.6.4. Prestaciones

Es el tiempo de respuesta del sistema, tanto durante el desarrollo normal del proceso como en estado de alerta. Durante el desarrollo normal del proceso la carga de trabajo de los equipos y el personal se considera que es mínima y está dentro de los parámetros que determinan el tiempo real de un sistema. En caso de declararse un estado de alerta, la actividad que se desarrolla aumenta de forma considerable la carga de los equipos informáticos y del personal que los maneja. El equipo debe poder asimilar toda la información que se genera, incluso bajo condiciones de trabajo extremas, de manera que no se pierda información aunque su proceso y presentación no se realicen en tiempo real.

1.6.5. Mantenibilidad

La IEEE define mantenibilidad como: “La facilidad con la que un sistema o componente software puede ser modificado para corregir fallos, mejorar su funcionamiento u otros atributos o adaptarse a cambios en el entorno”.

Los tiempos de mantenimiento pueden reducirse al mínimo si el sistema está provisto de unas buenas herramientas de diagnóstico que permitan realizar tareas de mantenimiento preventivo, modificaciones y pruebas de forma simultánea al funcionamiento normal del sistema.

1.6.6. Escalabilidad

Es la capacidad de un sistema para ser ampliado con nuevas prestaciones en un plazo de tiempo requerido, esto se puede deber a:

1. Al espacio disponible.
2. A la capacidad del equipo informático (memoria, procesadores, alimentaciones).
3. A la capacidad del sistema de comunicaciones (limitaciones físicas, protocolos, tiempo de respuesta).

El software de control debe poder evolucionar (ampliarse y actualizarse), adaptándose al entorno que controla, de manera que funcione de forma eficiente sin importar el tipo de equipamiento o el volumen de datos. A continuación se detalla un ejemplo de escalabilidad.

EJEMPLO:

Un sistema puede empezar con un único servidor para todas tareas (Figura 1.8): SCADA, Archivo, Alarmas, Comunicaciones. El problema de esta configuración es que todo está centralizado y ante un fallo el sistema se parará. Una ampliación posterior será más costosa de realizar, ya que habrá que modificar el hardware y el software. Si se tienen nuevas exigencias se deberá cambiar el servidor por uno más rápido, además del software ya que esto tiene impacto sobre él. Un planteamiento más correcto en el diseño permitirá un mejor aprovechamiento de los recursos.

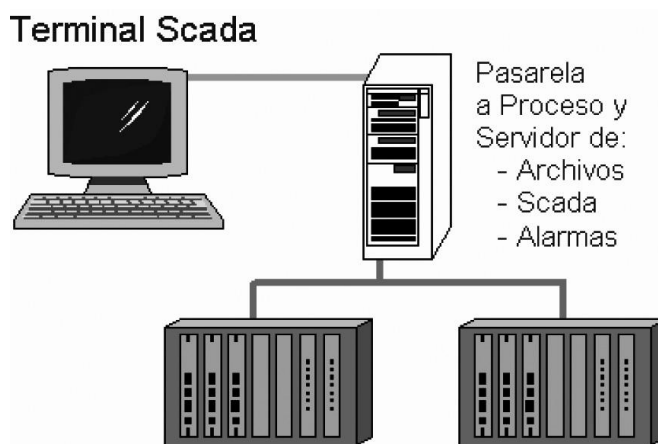


Figura 1.8: Sistema SCADA escalable, paso 1.

Un mejor planteamiento consiste en añadir más servidores que sirvan de apoyo al inicial, compartiendo las tareas del primero. Estos servidores serán de menor coste, pues las tareas serán más concretas. Por otro lado, si hay varios servidores compartiendo tareas el sistema será más tolerante a fallos. Además, una posible ampliación posterior es más sencilla en este caso. En la Figura 1.9 se puede observar que el servidor inicial se ha descargado del trabajo de comunicaciones con la Planta. En este caso se podría implementar un servidor de apoyo para archivos, alarmas y SCADA en el Servidor de comunicaciones por si fallase el Servidor principal.

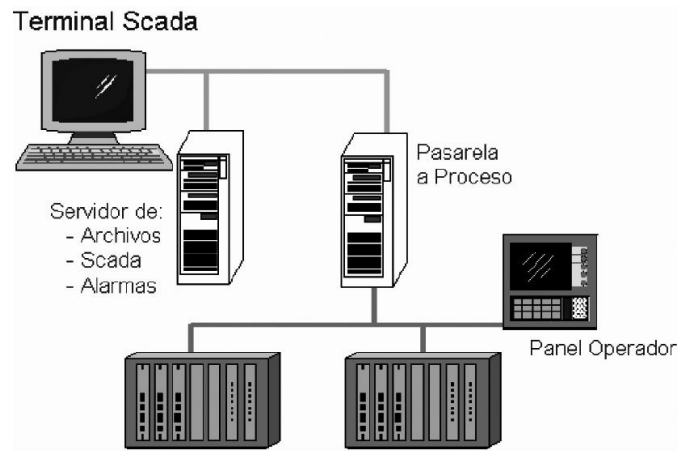


Figura 1.9: Sistema SCADA escalable, paso 2.

La siguiente evolución consiste en atomizar los grandes sistemas de supervisión y control en multitud de componentes, distribuyendo los sistemas de control y las aplicaciones en diferentes máquinas distribuidas en red y con capacidad de comunicarse entre ellas: servidores de datos y de alarmas, generadores de informes, de gráficas de tendencia, etc. La idea es un número de servidores mayor que en el caso anterior. La Figura 1.10 muestra esta situación. Ahora ya se tiene un servidor dedicado a cada tarea, permitiendo así más capacidad de proceso conjunto y varios accesos a los sistemas de supervisión.

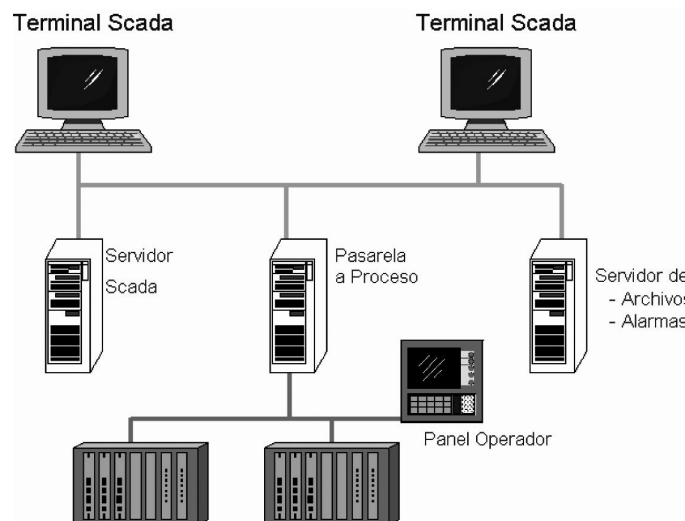


Figura 1.10: Sistema SCADA escalable, paso 3.

El paso último considera la seguridad y aplica el principio de redundancia como parte de la posibilidad de ampliación en un sistema. La Figura 1.11 muestra que la estructura inicial dispone de servidores redundantes que proporcionan un sistema seguro y

resistente a fallos:

1. Si cae la pasarela a Proceso, el control de Campo sigue operativo gracias al Panel de Operador.
2. El sistema de comunicaciones está duplicado. El Switch se ocupa de la gestión de la red corporativa.
3. Los terminales SCADA permiten el acceso al control de la instalación (incluyendo el Panel de Operador).
4. Los servidores redundantes toman el control en caso de problemas en los principales.

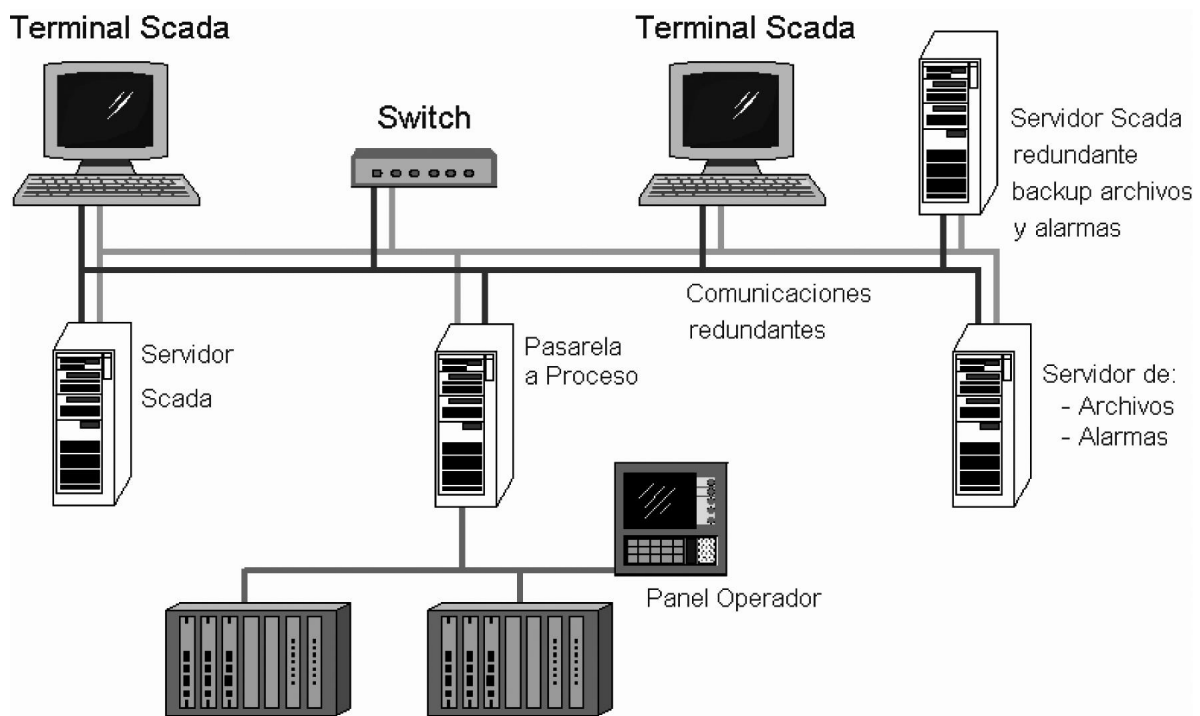


Figura 1.11: Sistema SCADA escalable, paso 4.

La visión del usuario consiste en un único sistema global de trabajo desde su ordenador, es decir, estas estructuras y sus cambios son transparentes a él. Para el ingeniero encargado del control, se trata de una herramienta muy potente, pues permite aislar las tareas de control y gestionarlas de forma mucho más eficiente. Por ejemplo, al ampliar un sistema de control en una factoría mediante la integración de nuevos servidores no representará mayor problema que la adición de éstos y las pruebas de funcionamiento pertinentes.

2

Arquitectura del Computador

Este tema mostrará la arquitectura del computador. Se desarrollará la arquitectura von Newman. También hay una sección para estudiar la organización de Entrada/Salida (E/S), una componente muy importante y que se va a mostrar en cierto detalle. Finalmente, se hablará de los periféricos industriales y del protocolo RS-232.

2.1. Fundamentos de Computadores.

El objetivo de esta sección es conocer los distintos componentes del computador, así como su funcionamiento [4]. En primer lugar se estudiará la estructura básica del computador. Se continúa presentando los elementos internos de la Unidad Central de Proceso, detallando a continuación las distintas etapas de la ejecución de una instrucción. Finalmente se detallan los componentes internos del computador para posteriormente estudiar su funcionamiento.

2.1.1. Estructura básica del computador.

El computador está compuesto por cuatro bloques básicos (Figura 2.1) que son:

Memoria principal: Almacena los programas y los datos necesarios para la ejecución de los mismos. Consta de una serie de celdas, del mismo tamaño (en bits), que almacenan un conjunto de bits. Cada celda se identifica mediante una dirección (Figura 2.1). Las dos operaciones permitidas sobre dichas celdas son las de lectura y las de escritura.

Unidad Central de Proceso (CPU): Está formada por la Unidad Aritmético-Lógica (ALU) y por la Unidad de Control (UC). La ALU se encarga de ejecutar las instrucciones (Sección 2.1.4), mientras que la UC coordina la ejecución de las instrucciones gracias

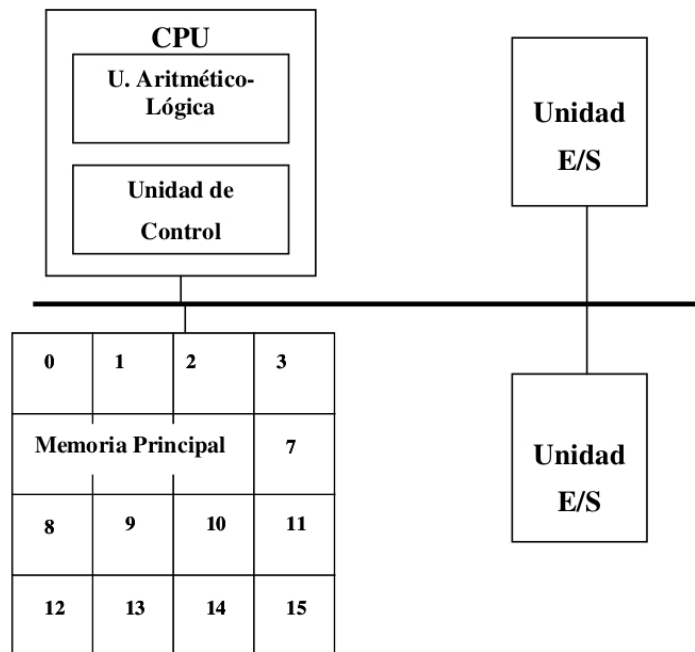


Figura 2.1: Estructura básica del computador.

a un conjunto de señales de control, gestiona la comunicación con los periféricos y resuelve los conflictos que surgen (Sección 2.1.5).

Bus: Líneas de comunicación internas que transfieren instrucciones y datos entre los distintos componentes del computador. Existen tres tipos de buses: el de Direcciones, el de Datos y el de Control.

Unidades de Entrada/Salida: Se encarga del intercambio de datos entre la CPU y los periféricos. Un periférico es cualquier dispositivo que está conectado al computador y no es parte del mismo, por ejemplo, una impresora, un teclado, un monitor, etc.

2.1.2. Elementos Internos de un Procesador.

La Unidad Central de Proceso (CPU), también llamada procesador, es el *cerebro* del computador y es la encargada de ejecutar las instrucciones. Para poder realizar esta función tiene dos componentes principales, la UC y la ALU.

La UC es la encargada de leer y decodificar las instrucciones máquina. Según la instrucción leída, genera las señales de control necesarias para que la ALU las ejecute. Las operaciones que realiza son elementales: sumas, restas, sumas lógicas, etc y los datos que intervienen en las operaciones se obtienen de la memoria principal.

2.1.3. Temporalización en la ejecución de la instrucción.

El computador tiene por función ejecutar de manera secuencial, una detrás de otra, una serie de órdenes llamadas *instrucciones máquina* que se agrupan en programas, es decir, un programa es un conjunto de instrucciones máquina ordenadas. Las fases necesarias para la ejecución de una instrucción son:

1. La UC solicita a la memoria principal que le envíe la instrucción almacenada en la dirección de memoria apuntada por un registro especial llamado *contador de programa* que se encuentra en la UC.
2. La UC recibe la instrucción, la analiza y si se necesitan operandos indica a la memoria o al banco de registros que los transfiera a la ALU para que pueda realizar la operación. Un operando es cada uno de los datos que participan en la instrucción, por ejemplo, en *SUMA A,B*, los operandos son *A* y *B*.
3. La ALU ejecuta la instrucción gracias a las señales de control enviadas por la UC, y si es necesario, se almacena el resultado en la memoria principal o en un registro.
4. Finalmente, se incrementa el contador de programa para que se ejecute la siguiente instrucción.

2.1.4. Unidad Aritmético-Lógica.

Se encarga de ejecutar las instrucciones aritméticas y lógicas. Éstas son operaciones elementales de poca complejidad.

El funcionamiento de la ALU se basa en el concepto de operador (Figura 2.2), que es un circuito electrónico que puede realizar diferentes operaciones aritméticas y lógicas, como son: sumas, restas, AND lógico, OR lógico, y en general todas aquellas para las que fue diseñada. Más concretamente, la ALU se basa en un operador sumador-restador.

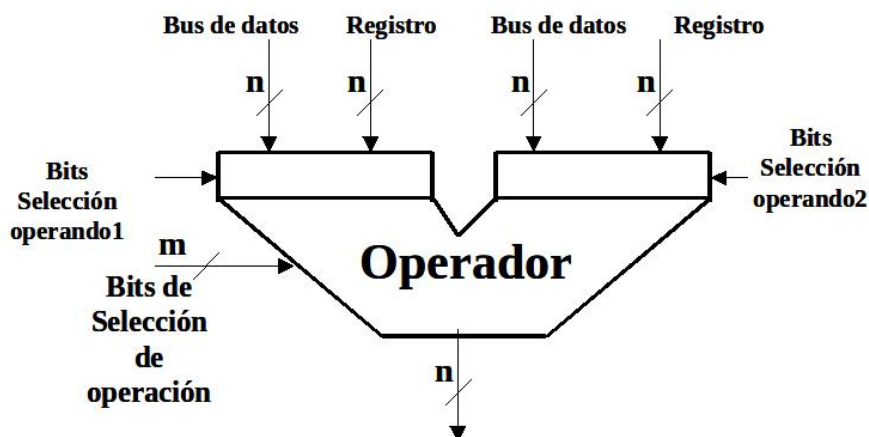


Figura 2.2: Operador.

Una ALU básica (Figura 2.3) está compuesta de un operador, un conjunto de registros y unos biestables de estado. La función de cada uno de éstos es:

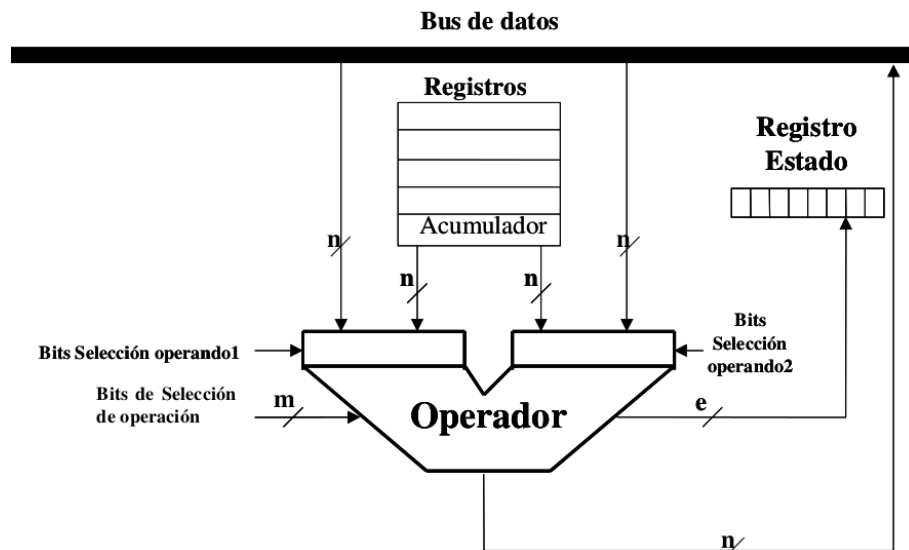


Figura 2.3: Unidad Aritmético-Lógica.

- **Banco de registros:** Los registros son elementos de almacenamiento que se utilizan para guardar temporalmente resultados intermedios y datos. El número de registros del banco es una potencia de dos, normalmente 16, 32, 64 ó 128. Existe un registro especial que es el acumulador que tiene la función de almacenar los resultados de la última operación realizada.
- **Biestables de estado:** Para la ejecución de algunas instrucciones es necesario disponer de alguna información relativa a la ejecución de la instrucción anterior. Ésta se almacena en los biestables de estado, que son:
 - Cero: Toma el valor 1 si el resultado es 0.
 - Negativo: Toma el valor 1 si el resultado es negativo.
 - Acarreo: Toma el valor 1 cuando la operación sobre los operandos sobrepasa el último bit necesitando uno más (bit de acarreo).
 - Desbordamiento: Toma el valor 1 si el resultado tiene desbordamiento, es decir, se obtiene un rango de representación que la CPU es capaz de manejar. Ocurre un desbordamiento porque el bit n y/o posteriores “no caben” en los n de la palabra.
- **Operador:** Realiza las operaciones aritméticas y lógicas que se seleccionan utilizando combinaciones de m bits (bits de selección de operación), luego tiene la capacidad de realizar como máximo 2^m operaciones diferentes. Para seleccionar si el operando proviene del bus o del banco de registros se dispone de un bit, el bit de selección de operando. En la Figura 2.3 se muestra como la entrada de cada operando se puede seleccionar del bus de datos o del banco de registros mediante un bit de selección de operando.

Normalmente una ALU realiza un conjunto de operaciones básicas que se clasifican en tres grupos: De desplazamiento, lógicas, de suma y resta, y de transferencia.

Operaciones de desplazamiento

Las operaciones de desplazamiento mueven los bits de una palabra hacia la izquierda o hacia la derecha un número de posiciones d . Este movimiento de bits se puede realizar dentro de una misma palabra o copiando los bits en otro lugar, como puede ser a un registro. Los desplazamientos se clasifican en:

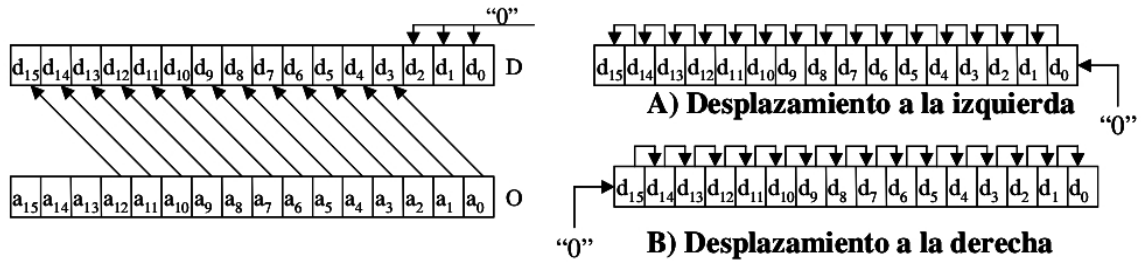


Figura 2.4: Desplazamiento Lógico.

1. **Desplazamiento lógico** (Figura 2.4): Si n es la longitud de palabra y k el número de posiciones a desplazar la ecuación será:

$$d_{i+k} = a_i \quad (2.1)$$

donde $i = 0, 1, \dots, n-1$

Dependiendo del signo de k el desplazamiento será hacia la derecha o hacia la izquierda provocando que haya posiciones que no reciben valores y que se completarán con ceros.

2. **Desplazamiento circular**: Muchas veces no se quieren perder bits, como ocurre en el desplazamiento lógico, por lo que se realiza un desplazamiento llamado circular en el que los bits que antes se perdían ahora son realimentados por un lado u otro dependiendo del sentido del desplazamiento. La Figura 2.5 muestra un ejemplo donde n es la longitud de palabra del número, k el número posiciones a desplazar y el módulo (MOD) es el resto de la división entera, la expresión será:

$$d_{(n+i+k)MODn} = a_i \quad (2.2)$$

donde $i = 0, 1, \dots, n-1$

3. **Desplazamientos aritméticos**: Es equivalente a una multiplicación o división por una potencia de dos. Por ejemplo, sea el número binario 01010, 10 en decimal, si se desplaza una posición hacia la derecha se obtiene el número 00101, que en decimal es el 5, es decir, un desplazamiento a la derecha equivale a dividir por 2^1 . Si se desplaza hacia la izquierda dos posiciones se obtiene que el número 01010 ($2^1 + 2^3$) pasa a ser el 101000 ($2^2 * (2^1 + 2^3) = 2^{1+2} + 2^{3+2} = 2^3 + 2^5$), que convertido a decimal es el 40, que equivale a multiplicar 10 por 2^2 . En general, desplazar a la derecha d posiciones equivale a dividir por 2^d , y desplazar a la izquierda d posiciones a multiplicar por 2^d . En estos desplazamientos hay que tener en cuenta la representación del número.

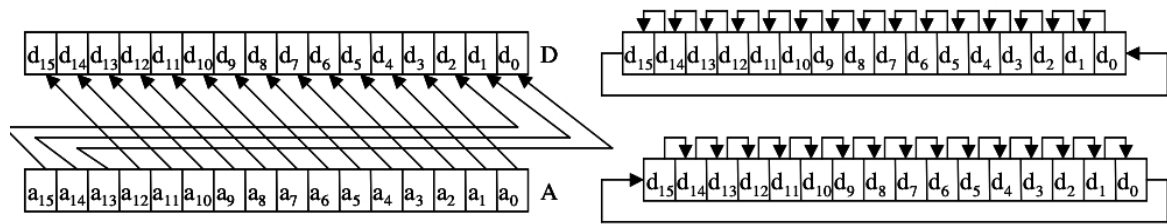


Figura 2.5: Desplazamiento Circular.

A	NOT(A)
0	1
1	0

Tabla 2.1: Función NOT.

4. **Desplazamientos concatenados:** Interviene más de un elemento (registros, biestables, etc). En la Figura 2.6 se muestra un ejemplo en el que intervienen dos registros y en el que se realiza un desplazamiento circular hacia la derecha.

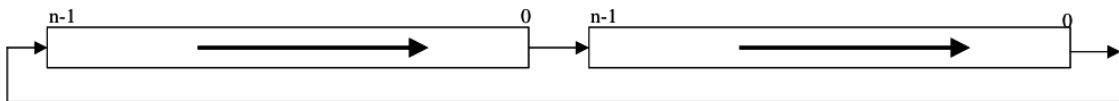


Figura 2.6: Desplazamiento Concatenado.

Operaciones lógicas.

Realizan funciones lógicas que quedan definidas por su tabla de verdad, en la que se asigna a cada uno de los posibles valores de entrada su correspondiente salida. Las funciones lógicas implementadas en la ALU son:

- Negación (*NOT*).
- Suma (*OR*).
- Producto (*AND*).
- Suma exclusiva (*XOR*). Cuya función equivale a $(\bar{A} \text{ AND } B) \text{ OR } (A \text{ AND } \bar{B})$, donde $\bar{A}$ y $\bar{B}$ son las negaciones de A y B respectivamente.

Dichas funciones se engloban en un solo operador, trabajan a nivel de bit y su realización es inmediata. Para aplicarlas sobre un registro hay que realizar el montaje del circuito que realiza la función. La Tabla 2.1 muestra la tabla de verdad de la función NOT y la Tabla 2.2 muestra la de las funciones lógicas OR, AND y XOR.

Operaciones suma y resta.

Las operaciones de suma y resta constituyen la base de la ALU, que está compuesta por operadores construidos con operadores de tipo lógico básico.

A	B	OR	AND	XOR
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	0

Tabla 2.2: Funciones OR, AND y XOR.

2.1.5. Unidad de Control.

La UC coordina la ejecución de las instrucciones, gracias a un conjunto de señales que le permiten indicar a cada uno de los componentes cuándo y cómo deben realizar su función. Éstas son:

1. Leer la instrucción a ejecutar. Para ello cuenta con un registro llamado *Contador de Programa* que se encarga de almacenar la dirección de la siguiente instrucción a ejecutar. La UC incrementa el contador de programa para obtener la siguiente instrucción. En caso de bifurcaciones carga la nueva dirección.
2. Cuando recibe la instrucción de la memoria principal y la decodifica, obteniendo el código de operación que determina la operación a realizar. Además debe localizar los operandos, transferirlos a la ALU y almacenar el resultado donde corresponda. Toda esta información está contenida en la instrucción.
3. Ejecutar la instrucción leída. Para ello envía las señales necesarias al resto de componentes indicando que es lo que tienen que hacer y cuando.
4. Resuelve situaciones de conflicto que puedan ocurrir, por ejemplo, una instrucción ilegal.
5. Se encarga de la comunicación con los periféricos.

La información necesaria para ejecutar estas funciones es la siguiente:

1. La instrucción que se está ejecutando, tanto su código de operación como sus operandos.
2. Los biestables de estado, que contienen información relativa a la última instrucción ejecutada.
3. Un contador de periodos, accionado mediante un reloj.
4. Las señales de control y estado externas a la CPU, utilizadas principalmente para el control de los periféricos.

2.1.6. Memoria Principal.

Como se vio al principio del tema, la memoria principal está compuesta por una serie de celdas que permiten dos operaciones, lectura y escritura. En este apartado, se estudiará su funcionamiento a partir de su representación esquemática, y las señales que recibe por parte de la UC y que permiten su manejo. Como puede verse en la Figura 2.7, la memoria consta de los siguientes elementos:

1. El registro de direcciones D , que es el encargado de contener la dirección en la que se quiere leer o escribir. La dirección debe de ser cargada antes de ordenar la operación de memoria.
2. El registro de datos RD , en el que se almacena el dato leído en las operaciones de lectura, y carga el dato a escribir en memoria en las operaciones de escritura.
3. Y las señales de memoria:
 - CM : Señal que sirve para iniciar un ciclo de memoria.
 - L : Señal que especifica ciclo de lectura, es decir, se quiere leer de memoria.
 - E : Señal que especifica ciclo de escritura, es decir, se quiere escribir en memoria.
 - CD : Señal de flanco, carga el contenido del bus en el registro D .
 - $CRDM$: Señal que carga en el registro RD la salida de la memoria. Se empleará al final del ciclo de lectura.
 - $CRDB$: Señal que carga en el registro RD la información existente en el bus de datos. Se utiliza cuando se va a proceder a una escritura.
4. Para la memoria utilizada se requieren ciclos de 3 periodos. La señal CM será de un periodo.
5. En el ciclo de lectura, la salida de la memoria se abre al bus en el último periodo, con lo que la información se puede almacenar en los registros.
6. En el ciclo de escritura, la memoria almacena el valor presente en el registro RD .

2.1.7. Organización de Entrada/Salida.

El objetivo de las unidades de E/S es el de realizar la conexión entre la CPU y los periféricos. Hay determinados factores que se deben tener en cuenta a la hora de estudiar el funcionamiento de estas unidades:

1. La velocidad de transmisión de los periféricos es inferior a la de la CPU y además ésta difiere en cada periférico.
2. Cada periférico tiene su propio tamaño de palabra, en muchas ocasiones diferente al de la CPU.

Además existen diferentes tipos de periféricos, según el tipo de operaciones que puedan realizar. Así se distinguen periféricos de lectura, de escritura y de lectura-escritura. Con los periféricos de lectura se pueden obtener datos externos, por ejemplo, el teclado. Se pueden enviar datos a los periféricos de escritura, por ejemplo, el monitor. En los periféricos de lectura-escritura se pueden intercambiar datos (obtener y enviar), por ejemplo, un pendrive. Para que la CPU y un periférico puedan intercambiar información deben de disponer de un conjunto de mecanismos que permitan controlar una serie de aspectos, como son:

1. Transmisión de la información.
2. Selección del periférico con el que trabajar.

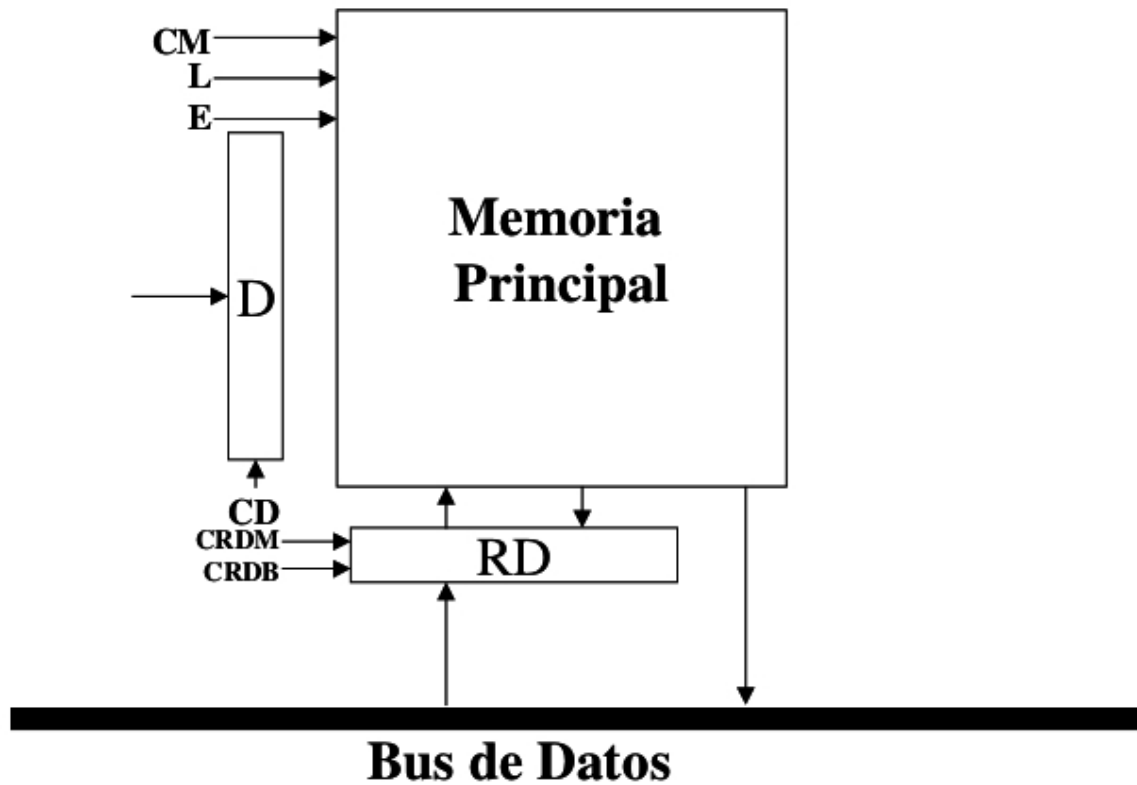


Figura 2.7: Esquema Memoria Principal.

3. Establecimiento del camino de los datos.
4. Conversión serie/paralelo, es decir, los datos pueden llegar bit a bit en vez de en palabras.
5. Mecanismos de control que indiquen cual es el origen y destino de la información, la cantidad de información a transmitir, códigos de protección, etc.

En el control de estos aspectos interviene la UC, el controlador del periférico y los programas de Entrada/Salida.

2.2. Organización de E/S.

En esta sección se va a seguir integramente el Tema 7 del libro Fundamentos y Estructura de Computadores de Jose María Angulo [1], por lo que debéis haceros con dicho tema para estudiarla.

2.3. Periféricos Industriales.

El objetivo de esta sección es detallar la función de los periféricos en un computador. Por ello, se estudiarán sus objetivos y sus características generales. También se presentará una clasificación y se mostrarán ejemplos de diferentes periféricos. Finalmente, se detallarán las conexiones de un PC, haciendo especial énfasis en la comunicación serie (RS-232) del PC.

2.3.1. Introducción.

Una primera definición que se puede dar de periférico es:

Se denomina periférico a todo aquel dispositivo que se conecta a una CPU a través de las unidades de E/S [4].

Esta primera definición hace alusión a que un periférico debe estar conectado a la CPU mediante las Unidades de E/S.

Otra definición sería:

Se denominan periféricos tanto a los dispositivos a través de los cuales el PC se comunica con el mundo exterior, como a los sistemas que almacenan o archivan la información, sirviendo de memoria auxiliar de la memoria principal [3].

Esta definición hace alusión a dos conceptos:

1. Sistemas de almacenamiento: Son necesarios para un computador dado que la información almacenada en la memoria principal se borra cuando no existe alimentación de energía eléctrica. Además, la memoria principal es baja en cuanto a capacidad se refiere.
2. Comunicación con el mundo exterior: La CPU necesita información procedente del exterior, así como enviar los resultados obtenidos.

Los periféricos transforman la información externa en señales codificadas de forma válida en el computador, permitiendo su transmisión, detección, interpretación, procesamiento y almacenamiento de forma automática. Según el sentido de la transferencia se distinguen:

- Dispositivos de Entrada: Transforman la información externa en información correctamente codificada para el PC.
- Dispositivo de Salida: La información codificada que llega del PC se transforma de acuerdo con el código de E/S en información inteligible por el usuario.
- Dispositivos Mixtos: Permiten la lectura y escritura de los mismos.

Por otro lado, no hay que confundir periférico con soporte de información. El dispositivo periférico se encarga de escribir la información al correspondiente soporte. Ejemplos:

- El CD es el soporte de la información, mientras que la unidad lectora de CDs es el periférico.
- El papel de la impresora es soporte de información, y la impresora es el periférico.

2.3.2. Características generales de los periféricos.

Los periféricos suelen estar formados por dos partes claramente diferenciadas en cuanto a su misión y funcionamiento:

1. La parte mecánica está formada básicamente por dispositivos electromecánicos (conmutadores manuales, motores, electroimanes, etc) controlados por los elementos electrónicos. La velocidad de funcionamiento de un periférico viene dada por sus componentes mecánicos.
2. La parte electrónica se encuentra casi en su totalidad integrada en los circuitos de las controladoras.

Las características más importantes a considerar de un periférico y de su soporte son:

Fiabilidad: Es la probabilidad de que se produzca un error de E/S y depende de la naturaleza del soporte (soporte más/menos fiable), de las condiciones ambientales en que se conserva el soporte, o de las características de la unidad.

Duración: Es la permanencia sin alteración de los datos a lo largo del tiempo. Algunos soportes van perdiendo la señal escrita a lo largo del tiempo y acaban perdiendo los datos por obsolescencia física del soporte.

Densidad: Se refiere a la cantidad de datos (bits o caracteres) contenidos por unidad de volumen, superficie o longitud ocupada.

Reutilización: Un soporte de información se dice reutilizable cuando permite guardar nueva información sobre datos obsoletos. Con este problema se han enfrentado los fabricantes de discos ópticos (CD-ROM), los cuáles hasta hace poco tiempo no han sido susceptibles de ser reutilizables.

Tipo de acceso: Característica vinculada al dispositivo lector/grabador. Se dice que un dispositivo es de acceso secuencial si para acceder a un dato determinado se debe acceder primero a todos los que le preceden físicamente (cintas magnéticas). Se dice, en cambio, que un dispositivo permite el acceso directo si se puede acceder a un dato sin necesidad de pasar por los datos que le preceden (CD).

Transportabilidad: Un soporte de información es transportable si puede ser trasladado de una unidad periférica a otra. Ejemplo: Un CD puede ser utilizado en distintas unidades de CD. Por el contrario hay soportes de información fijos, que no pueden extraerse de la unidad correspondiente.

Otro concepto importante es la “capacidad de memoria masiva de un computador”, que es la capacidad total que admiten las unidades periféricas de memoria masiva suponiendo que están montados todos los soportes de información que admite. Los parámetros que lo caracterizan son:

- **Velocidad de transferencia (dispositivos de almacenamiento secundario):** Es la cantidad de información que el dispositivo es capaz de leer/escribir por unidad de tiempo. Ejemplo: bits/s, caracteres/s.
- **Tiempo de acceso:** Es el tiempo promedio que necesita un dispositivo de almacenamiento secundario para leer/escribir un dato en su soporte de información.

Tipo	Dispositivo
Unidades de Entrada	Teclado Ratón Lápiz óptico Lector óptico Lector de caracteres imanables Lector de bandas magnéticas Lector de tarjetas inteligentes (Smart Card) Lector de marcas Lector de caracteres manuscritos Lector de códigos de barras Reconocedores de voz Joystick Digitalizador Pantalla sensible al tacto Scanner o rastreadores
Unidades de salida	Impresora Sintetizado de voz Visualizador Plotter Monitor Microfilm Instrumentación Industrial
Almacenamiento	Cinta magnética Disco magnético Tambor magnético Disco óptico Sistema de CD-ROM DVD- Disco Versátil Digital.

Tabla 2.3: Clasificación de los Periféricos.

Finalmente, se dice que un periférico es “ergonómico” si su diseño físico externo se adapta al usuario, consiguiendo una buena integración hombre-máquina y una adecuada eficiencia en su utilización. Por ejemplo, los equipos con homologación alemana GS son ergonómicos. Esta homologación también se aplica a otros productos.

2.3.3. Clasificación de los periféricos.

Se clasifican en dos categorías:

1. Almacenamiento: Dispositivos que se encargan de grabar la información temporalmente o definitivamente.
2. Comunicación: Dispositivos que se encargan de intercambiar información con el exterior, tanto para recibirla, como para enviarla al exterior. Por ejemplo, el teclado y la pantalla. Dentro de éstos se distinguirán las Unidades de Entrada y las Unidades de Salida.

La Tabla 2.3 muestra algunos ejemplos de dispositivos de cada uno de los tipos.

2.3.4. Conexiones de periféricos a un PC standard.

El PC tiene una gran importancia dentro de la informática industrial debido a que se encuentra en diferentes lugares dentro de un proceso productivo industrial, por ejemplo, como servidor, como equipo capturador de datos desde sensores, para manejar actuadores, PLC, reguladores, etc. Usualmente tiene conectados sensores, actuadores, periféricos industriales, etc. Todos estos dispositivos se conectan al PC mediante las Unidades de E/S pudiendo establecer así una comunicación periférico-PC. Las posibilidades para realizar esta comunicación son:

1. Existencia de una conexión libre en el bus para pinchar la tarjeta controladora del periférico.
2. Conectar a conexiones estándar del PC, como el ratón o el teclado.
3. Mediante los puertos de comunicaciones paralelo, donde se conectan periféricos como la impresora.
4. Uso de los puertos serie para infinidad de periféricos, muchos de ellos de uso industrial.
5. En la actualidad, se utilizan las conexiones USB que permiten conectar distintos periféricos de forma rápida y fácil.

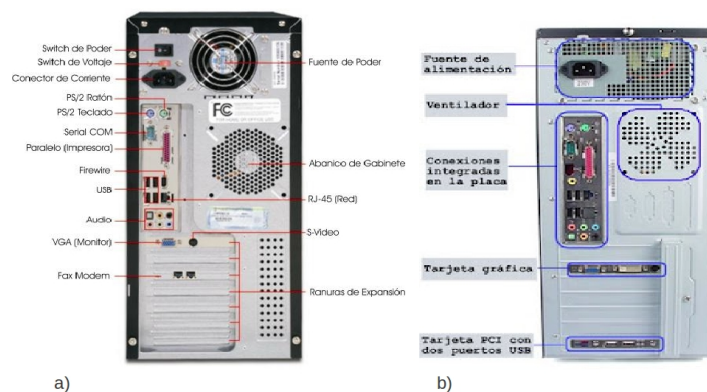


Figura 2.8: Puertos de un PC.

La Figura 2.8 muestra la parte trasera de dos computadores. La Figura 2.8.a muestra detalladamente cada una de los diferentes conectores de un PC. La parte superior de la Figura 2.8.b muestra las conexiones para la alimentación. Los conectores de los componentes integrados en placa están junto al ventilador. También tiene una tarjeta gráfica no integrada en placa, y, una tarjeta PCI con dos puertos USB adicionales. Los conectores integrados en la placa base son los siguientes:

1. Puertos del ratón y del teclado: Son de color verde y de color morado respectivamente. Ambos son de tipo minidin. El del teclado recibe el nombre de compatible PS/2, por ser este modelo de PC el primero que montó este tipo de conexión.

2. Puerto serie: Existen conectores de 9 y 25 patillas, denominados DB-9 y DB-25. En este caso se usan conectores tipo DB-9. Para nombrarlos se utiliza el término COM (derivado de comunicaciones) seguido de un número para designar un puerto serie, por ejemplo: COM1, COM2, COM3 y COM4. Puede enviar sólo un bit de información a la vez, una tasa muy baja pero aceptable para dispositivos de baja velocidad.
3. Puerto paralelo: Se utiliza principalmente para impresoras. Normalmente se usa el término LPT (impresora en línea) más un número para designar un puerto paralelo, por ejemplo LPT1. Este tipo de puerto puede enviar ocho bits de información a través de ocho cables en paralelo, mientras que una conexión serie envía un bit un puerto paralelo puede enviar un byte. El puerto paralelo puede funcionar de cuatro modos:
 - SPP (Standard Parallel Port): puerto paralelo estándar de hasta 500 Kbps.
 - EPP (Enhanced Parallel Port): puerto paralelo mejorado. En este modo puede haber comunicación bidireccional de hasta 2 MBps. El dispositivo periférico debe soportar este modo.
 - ECP (Extended Parallel Port): puerto con capacidades extendidas (alta velocidad y utiliza DMA). Transferencia bidireccional de hasta 2,4 MBps. El dispositivo periférico debe soportar este modo. Se requiere además un canal DMA de acuerdo con las especificaciones del dispositivo.
 - ECP+EPP: la unión de ambos.

Los modos ECP y EPP permiten las comunicaciones bidireccionales por el puerto paralelo a alta velocidad ya que incorporan un buffer entre las patillas de datos y la CPU, por el que aumentan la velocidad de transferencia de la información.

4. USB (Universal Serial Bus). Cuenta con la característica PnP (Plug and Play) y la facilidad de conexión en caliente, es decir, que se pueden conectar y desconectar los periféricos sin necesidad de reiniciar el PC. El bus USB permite la conexión de 127 dispositivos como máximo.

El primer dispositivo USB fue el USB 1.1 con una velocidad de transferencia de 12 MBps, después surgió una mejora considerable que fue el USB 2.0 con una velocidad de transferencia de 480 MBps. Finalmente existe el USB 3.0 que multiplica por 10 de la velocidad de transferencia respecto a USB 2.0 (4,8 GBps). Otra característica es su “regla de inteligencia”, los periféricos conectados si después de un rato no se utilizan pasan inmediatamente a un estado de bajo consumo.

5. HDMI (High-Definition Multimedia Interface) es una norma de audio y vídeo digital cifrado sin compresión que proporciona una interfaz entre cualquier fuente de audio y vídeo digital y monitor de audio/vídeo digital compatible, como un televisor digital (DTV). HDMI permite el uso de vídeo computarizado, mejorado o de alta definición, así como audio digital multicanal en un único cable. Es independiente de los estándares de DTV (Digital TeleVision). Un decodificador decodifica la señal, codificada en una de las normas de MPEG para DTV (ATSC, DVB), obteniendo los datos de vídeo sin comprimir. Estos datos se codifican en formato TMDS para ser transmitidos digitalmente por medio de HDMI.
6. RCA vídeo. Este conector lleva la señal de video compuesto. Suele ser de color amarillo para distinguirlo de los RCA de sonido. La calidad del vídeo no es la óptima, ya que la información se envía en una sola señal analógica.

7. Conectores Audio. El conector verde conecta los altavoces normales, el rojo el micrófono y el azul es la entrada de línea que se conecta a la salida de audio de cualquier fuente de sonido.

2.3.5. Protocolo RS-232.

Una gran cantidad de periféricos industriales se conectan al PC vía RS-232, por ello, se estudiará en detalle este protocolo. Fue definido inicialmente para conectar un PC a un modem. Este estándar fue diseñado en los 60 para comunicar un equipo terminal de datos o DTE (Data Terminal Equipment, el PC en nuestro caso) y un equipo de comunicación de datos o DCE (Data Communication Equipment). Es un estándar que rige los parámetros de uno de los modos de comunicación serie más utilizados, estandarizando las velocidades de transferencia de datos, la forma de control que utiliza dicha transferencia, los niveles de voltajes utilizados, el tipo de cable permitido, las distancias entre equipos, los conectores, etc. Además de las líneas de transmisión y recepción, las comunicaciones serie disponen de líneas de control de flujo, su uso es opcional dependiendo del dispositivo a conectar.

La codificación del valor 1 se realiza utilizando un voltaje bajo, mientras que el 0 se codifica con un voltaje alto. Más concretamente, los voltajes para un nivel lógico alto pueden estar entre -3V y -15V, y para un nivel bajo entre +3V y +15V. Los voltajes más usados son +12V/-12V y +9V/-9V.

La comunicación serie RS-232 trabaja con conectores DB-9 y DB-25. Con conectores DB-25 se puede trabajar de forma síncrona y asíncrona. DB-25 utiliza 22 de los 25 pines para el modo síncrono y 9 de los 25 pines para modo asíncrono (los mismos que con DB-9). Con conectores DB-9 sólo se puede trabajar en modo asíncrono.

Para el establecimiento de canales para desarrollar la comunicación existen tres técnicas:

1. Simplex: La comunicación serie se realiza en una única dirección (del DTE al DCE o viceversa) y una única línea de comunicación. Sólo habrá un transmisor y un receptor. Sólo es necesario un enlace a dos hilos. El principal problema es que el extremo receptor no tiene ninguna forma de avisar al extremo transmisor sobre su estado y sobre la calidad de la información que se recibe.
2. Semi-duplex (Half-Duplex): La comunicación serie se establece a través de una sola línea, pero en ambos sentidos. No se puede transmitir información en ambos sentidos de forma simultáneamente. Este modo permite la transmisión desde el receptor de la información, sobre el estado de dicho receptor y sobre la calidad de la información recibida, por lo que permite así la realización de procedimientos de detección y corrección de errores.
3. Full-duplex: Se utilizan dos líneas (una transmisora y otra receptora) y se transfiere información en ambos sentidos. La ventaja de este método es que se puede transmitir y recibir información de manera simultánea. La mayoría de los dispositivos pueden transmitir información en full-duplex y en semi-duplex (el modo simplex es un caso especial dentro de semi-duplex).

La información se representa en una serie de bits en forma de onda para realizar la transmisión. Los bits se transmiten por un periodo de tiempo fijo que se conoce como

“periodo baud” (T). Se transmite un bit tras otro desde el emisor al receptor durante cada periodo baud. La velocidad en baudios (tasa de baudios) de un puerto serie del PC es igual al número de bits por segundo que se transmiten o reciben (Ecuación 2.3).

$$\frac{1}{T} \quad (2.3)$$

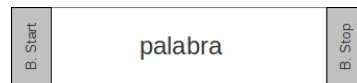


Figura 2.9: Trama RS-232.

El DTE y DCE tienen el tiempo que define el periodo baud para poder enviar información codificada de esta manera, además deben trabajar a la misma frecuencia y sincronizados. Los bits se transmiten como grupos separados, con una longitud típica de 7 u 8 bits, llamados *caracteres*. Se utiliza el nombre carácter porque cada grupo de bits representan un código ASCII, utilizado para representar los códigos alfanuméricos. Cada carácter se envía en una trama, frame en notación inglesa (Figura 2.9). Una trama tiene un bit 0 (Bit de Start, de inicio) seguido por el carácter mismo, seguido (opcionalmente) por un bit de paridad, y después uno o más bits a 1 (Bits de Stop, de paro). El Bit de Inicio le dice al receptor que empieza la trama, y el Bit de Stop indica el final de la trama. Los pasos a seguir para enviar una trama son los siguientes:

1. Transmitir Bit de Start.
2. Enviar bits de datos comenzando por el de menor peso.
3. Transmitir el Bit de Paridad si procede.
4. Enviar los Bits de Stop.

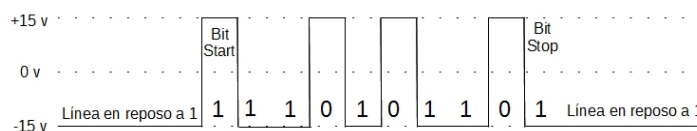


Figura 2.10: Ejemplo de transmisión de trama RS-232.

En esta Figura 2.10 se puede ver un ejemplo de la transmisión de la palabra 11010110. A continuación se detalla la secuencia de acciones:

1. Inicialmente, la línea está en reposo a nivel alto.
2. El emisor envía el Bit de Start para avisar al receptor del comienzo de la trama.
3. A continuación se envía la palabra comenzando por el bit de más peso. Se puede configurar para que sea a la inversa.
4. Después se transmitirá un bit de paridad (si se ha establecido en los parámetros iniciales).

5. Posteriormente se envía el número de Bits de Stop que se indicó en la configuración.
6. Finalmente, la línea vuelve a estar reposo.

Para terminar esta sección se quiere destacar con que el número de bits de datos (de 5 a 8), de bits de Stop y el criterio de paridad para la detección de errores son parámetros configurables, usualmente toman los siguientes valores: 8 bits de Datos, 1 bit de Stop y sin paridad. El bit de paridad P puede ser par o impar. Si es paridad par P tomará el valor 0 si el número de unos en el carácter es par y 1 en caso contrario, es decir, se fuerza a que la suma de los bits enviados sea par. Si la paridad es impar P tomará el valor 0 el número de unos es impar y 1 en caso contrario, es decir, se fuerza a que la suma de los bits enviados sea impar. Luego, a nivel de software, la configuración que se debe dar a una conexión a través del puerto serie es:

1. Selección de la velocidad en baudios (1200, 2400, 4800, etc).
2. Criterio de paridad (paridad par, paridad impar o sin paridad).
3. Bits de parada (1 ó 2).
4. Cantidad de bits por dato (5 a 8) que se utiliza para cada carácter enviado.



Figura 2.11: Localización física de las patillas: a) Macho, b) Hembra.

2.3.6. RS-232 en el PC.

El PC cuenta con conectores DB9 macho de 9 pines por los que se pueden conectar los dispositivos al puerto serie. Los conectores hembra que se enchufan tienen una colocación de pines diferente, con el fin de conectar cada pin con su igual, es decir, el pin 1 del macho con el pin 1 del hembra, y así sucesivamente. La Figura 2.11 muestra la localización física de las patillas en un DB9 Macho (Figura 2.11.a), y en un DB9 hembra (Figura 2.11.b).

La Tabla 2.4 detalla la señal asociada a cada patilla del conector DB-9. Es importante destacar, en lo referente a la descripción de las señales, que “poner a ON” se refiere a un nivel alto de voltaje RS-232 y “poner a OFF” se refiere a un nivel bajo de voltaje RS-232. Cada una de estas señales significa lo siguiente:

Ring Indicator (RI): Señal enviada desde el DCE al DTE para indicarle que se está recibiendo una señal por el canal de comunicaciones para establecer una comunicación. Se usa en los modem, y le indica al DTE que otro DTE quiere iniciar una comunicación con él.

Señal	Nº Pin
Carrier Detect (DCD)	1
Receive Data (RX)	2
Transmit Data (TX)	3
Data Terminal Ready (DTR)	4
Signal Ground (GND)	5
Data Set Ready (DSR)	6
Request To Send (RTS)	7
Clear To Send (CTS)	8
Ring Indicator (RI)	9

Tabla 2.4: Conector DB-9 de 9 pines.

Data Terminal Ready (DTR): El DTE la pone a ON para indicar que está preparado para transmitir/recibir (intercambiar) datos con el DCE. DTR debe estar a ON antes que el DCE pueda confirmar DSR. También se utiliza en los modem y, junto con la señal DSR, prepara al DTE para comunicar con un DTE remoto.

Data Set Ready (DSR): El DCE la pone a ON para indicar al DTE que todo está preparado para intercambiar datos. Se utiliza en los modem.

Request to Send (RTS): Cuando el DTE está preparado para transcribir datos al DCE, RTS se activa (poner ON). En sistemas simplex y duplex, esta condición mantiene el DCE en modo recibir. En sistemas semi-duplex, esta condición mantiene el DCE en modo recibir y desactiva el modo transmitir. La condición OFF mantiene el DCE en modo transmitir. Después de activar RTS, el DCE debe activar la señal CTS antes de que la comunicación pueda comenzar.

Clear to Send (CTS): El DCE confirma RTS poniendo CTS a ON cuando está preparado para comenzar la comunicación.

Data Carrier Detect (DCD): Señal desde el DCE al DTE. Se pone a ON cuando el DCE está recibiendo una señal desde un DCE remoto, lo que significa que se van a recibir datos del DTE remoto. Esta señal se mantiene a ON siempre que la línea sea detectada adecuadamente. Se usa en los Modem.

Transmitted Data (TD): Es la línea de transmisión de datos. Esta señal la genera el DTE y es recibida por el DCE.

Received Data (RD): Es la línea de recepción de datos. Esta señal la genera el DCE y la recibe el DTE.

La Figura 2.12 muestra las señales de un puerto serie indicando mediante una flecha si son de entrada o salida. Para que se produzca la comunicación entre el DTE y el DCE se necesita un cable con un conector en cada extremo. El conector del PC, DTE en nuestro caso, debe ser macho y el del DCE será hembra. El cable es un conjunto de hilos que unen las patillas de los conectores. Un cable serie muy básico es uno de tres hilos (Figura 2.13). Como puede verse, se une la salida TX del DTE con la entrada RX del DCE, esto se hace en los dos extremos, con lo que ambos pueden enviar información a la vez. Otra combinación utilizada consiste en la utilización de cinco hilos que se unen como muestra la Figura 2.14. Como puede verse, se incorporan las señales RTS y CTS que se conectan entre sí en ambos extremos. Otra configuración también utilizada es la de la Figura 2.15, que muestra las

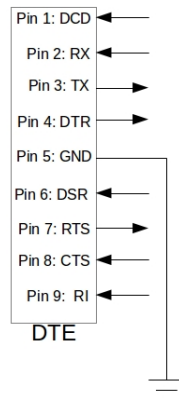


Figura 2.12: Señales RS-232.

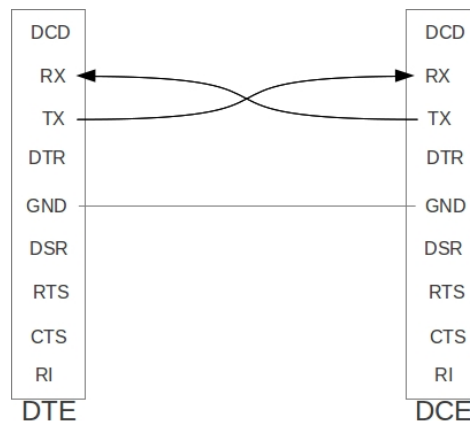


Figura 2.13: Conexiones cable NULL MÓDEM de 3 hilos.

conexiones de un cable RS-232 cruzado de 7 hilos. Se conectan las salida TX con la entrada RX, salida DSR con entrada DCD y salida CTS con entrada RTS. Como puede verse, cada salida se conecta con su entrada receptora para poder establecer comunicación en ambos sentidos.

Respecto al control de flujo de datos con la RS232 comentar que existen dos posibilidades:

Hardware (RTS/CTS): La línea CTS indica al PC si puede transmitir o no. En aplicaciones como la conexión de un PC a una impresora serie (dispositivo lento) la línea CTS está gobernada por la impresora para impedir que el PC desborde su buffer de entrada.

Software XON/XOFF: Otra posibilidad es usar el protocolo software XON/XOFF que consiste en lo siguiente:

1. Cuando la impresora está dispuesta para recibir datos (buffer de entrada vacío o casi vacío) transmite al PC la marca XON (XON y XOFF son códigos ASCII predefinidos).

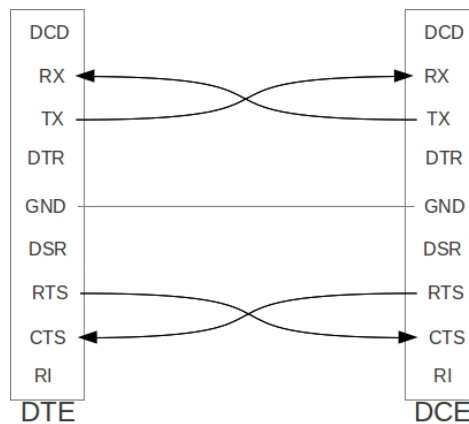


Figura 2.14: Conexiones cable NULL MÓDEM de 5 hilos.

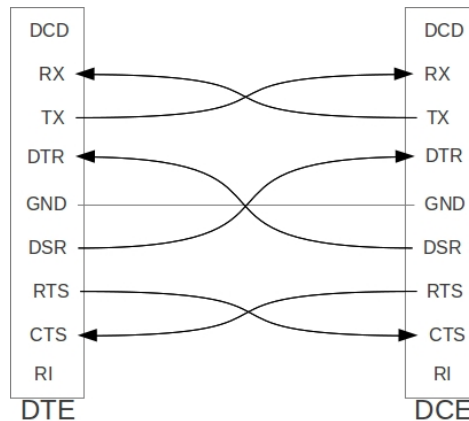


Figura 2.15: Conexiones cable NULL MÓDEM de 7 hilos.

2. Si el PC transmite demasiado rápido para la impresora y el buffer está próximo a llenarse, entonces se manda la marca XOFF.
3. El PC transmite sólo si la última marca recibida fue XON.

Dependiendo de las características de los equipos a conectar se puede hacer un control de flujo RTS/CTS, XON/XOFF, ambos o ninguno.

Para terminar el tema se verán dos ejemplos de cómo funcionan estas señales. En la explicación poner a ON significa lo mismo que activar, y poner a OFF es igual que desactivar. El primero de ellos consiste en las señales necesarias para comunicar un DTE con un DCE de forma asíncrona y en modo semiduplex. La secuencia de acciones que ocurre entre DTE y DCE es la siguiente:

1. El DTE envía tres tramas de información al DCE.
2. El DCE se satura y debe indicar al DTE que pare de enviar información.

3. El DTE envía una nueva trama de información al DCE.
4. El DCE envía dos tramas de información al DTE.

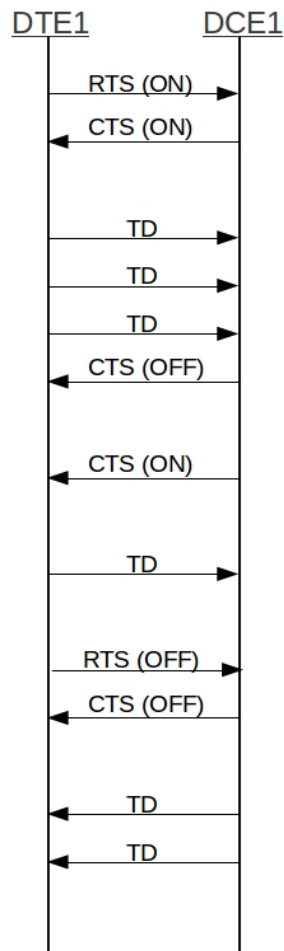


Figura 2.16: Ejemplo de tráfico de señales entre un DTE y DCE.

La Figura 2.16 muestra la señales utilizadas. Inicialmente el sistema está con las señales RTS y CTS en OFF, ya que si estuviera en ON impediría la comunicación del DCE. En el ejemplo ocurre la siguiente secuencia de acciones:

1. Primero el DTE desea transmitir, por ello, se debe de hacer cargo de la línea de comunicación. Pide la línea activando la señal RTS (ponerla a ON), y esperando a que el DCE ponga CTS a ON. Cuando estas dos acciones ocurren, el DTE puede transmitir pues tiene el control de la línea de comunicación.
2. Comienza la transmisión de datos mediante el envío de las tres tramas a enviar. El envío de tres señales de datos TD.
3. El DCE se satura (por ejemplo, tiene la memoria llena) y le indica al DCE que pare. Esto lo realiza poniendo CTS a OFF.

4. Cuando el DCE se encuentra preparado para recibir activa CTS de nuevo (CTS a ON). Destacar que durante todo este tiempo el DTE ha mantenido RTS a ON puesto que quería seguir transmitiendo.
5. El DTE realiza la transmisión de una nueva trama.
6. Ésta ha sido su última trama por lo que libera la línea para que pueda transmitir el DCE, si lo desea, poniendo RTS a OFF, y, el DCE responde poniendo CTS a OFF. Ahora el DCE está en modo transmitir.
7. El DCE desea transmitir dos tramas y puede, puesto que dispone de la línea de comunicación. Realiza la transmisión de sus dos últimas tramas mediante la señal TD.

El segundo ejemplo muestra las señales necesarias para comunicar un DTE (DTE1) con otro DTE (DTE2) de forma asíncrona y en modo semiduplex. DTE1 está conectado a la red (LAN, Internet, etc) mediante un modem (DTE1), la misma situación se repite para el DTE2. La secuencia de acciones entre DTE1 y DTE2 será el siguiente:

1. DTE1 desea enviar una trama de información al DTE2.
2. DCE2 responde mediante una trama de información al DTE1.

La Figura 2.17 muestra la señales utilizadas. En este ejemplo ocurre la siguiente secuencia de acciones:

1. Primeramente se debe establecer la comunicación entre los dos DTEs. DTE1 inicia la comunicación. DCE1 y DCE2 se comunican mediante el protocolo o protocolos que estén utilizando. DCE2 activa RI a ON indicando así al DTE2 que está recibiendo una señal por el canal de comunicaciones para establecer una comunicación.
2. DTE1 pone DRT a ON para indicar que está preparado para intercambiar datos su DCE. Lo mismo hace DTE2 con DCE2. Ambos DCEs (DCE1 y DCE2) ponen DSR a ON para indicar que están preparados para intercambiar datos con su correspondiente DTE.
3. DTE1 desea transmitir, luego solicita hacerse con el control de la línea de datos que lo une con DCE1. Esto se hace activando la señal RTS. DCE1 se da cuenta que DTE1 va a enviar datos y envía por la red a DCE2 el aviso, lo que provoca que DCE2 ponga DCD a ON, lo que indica a DTE2 que va a recibir datos de DTE1 remoto. DCE2 avisa que todo está correcto a DCE1 y éste pone CTS a ON, con lo que DTE1 pasa a modo transmitir y DCE1 pasa a modo recibir.
4. DTE1 envía una trama a DCE1, éste la envía por la red a DCE2, que se la envía DTE2 (señal RD).
5. DTE1 ha terminado de transmitir, y libera la línea de datos que tiene con DCE1 poniendo RTS a OFF, y DCE2 responde desactivando CTS y comunicando mediante la red a DCE2 que DTE1 deja de transmitir. Esto provoca la llegada de la puesta de DCD a OFF.
6. DTE2 desea enviar la trama de respuesta a DTE1, luego debe enviar a su DCE dicha trama. Para ello, activa la señal RTS, lo que provoca que DCE2 avise a DCE1 a través de la red. DCE1 pone a ON DCD, lo que avisa a DTE1 que le llegarán datos de DTE2. Finalmente DCE2 cede la línea de datos a DTE2 para que le pueda enviar la trama.

7. DTE2 envía la trama a DCE2 (señal TD), que se envía por la red y llega a DCE1. DCE1 entrega la trama a DTE1 enviando los datos (señal RD).
8. DTE2 pone RTS a OFF, ya no desea transmitir. DCE2 envía por la red el aviso a DCE1 que DTE2 ya no quiere transmitir, llegando al DTE1 mediante la señal DCD a OFF. Y DCE2 pone CTS a OFF.
9. Con esto se ha terminado la comunicación con lo que ambos DTEs-DCEs cierran la comunicación: DTE1 pone DRT a OFF, DCE1 pone DSR a OFF. Los mismos pasos se realizan entre DTE2 y DCE2.

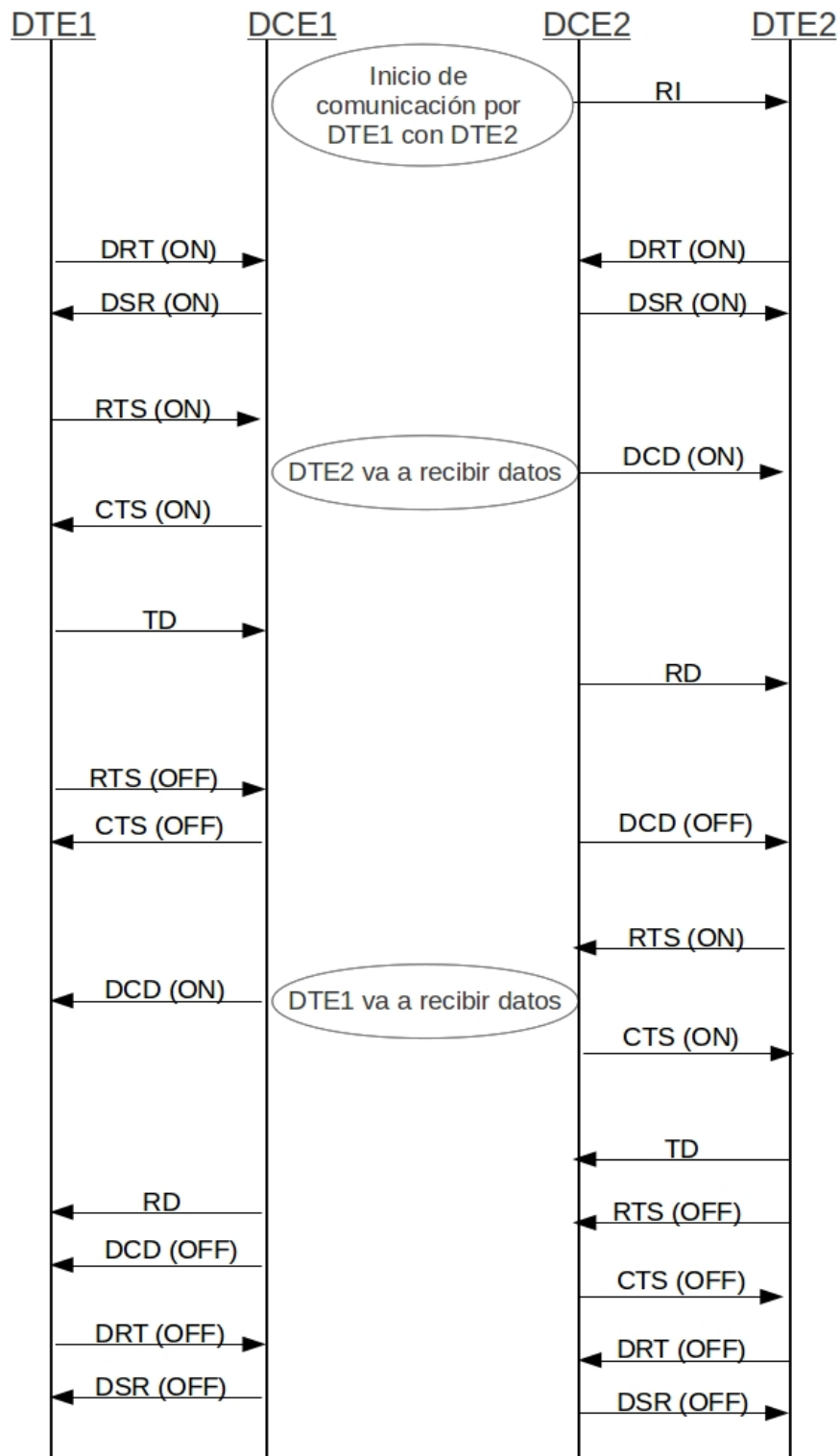


Figura 2.17: Ejemplo de tráfico de señales entre un DTE y otro DTE.

3

Sistemas de Tiempo Real

Esta sección mostrará los principios de los Sistemas de Tiempo Real. Estos sistemas engloban una gran cantidad de conceptos, siendo su programación muy difícil. Para conseguir una visión general de los mismos, objetivo fundamental del tema, se ha estructurado el tema en las siguientes secciones. En las dos primeras secciones se realiza una breve introducción mostrando varias definiciones de Sistemas de Tiempo Real, para continuar con unos ejemplos. A continuación se muestran las características de estos sistemas. En la sección 3.4 se exponen los conceptos de diseño de Sistemas de Tiempo Real. A continuación se detallan los conceptos de fiabilidad y tolerancia a fallos. Para terminar con el tema de planificación de Sistemas de Tiempo Real. Este tema está documentado de [2].

3.1. Introducción.

Muchos sistemas llevan incorporado un computador (o sólo el microprocesador) para realizar su función principal, por ejemplo: máquinas expendedoras, automóviles, lavadoras, etc. Este tipo de sistemas se conocen como Sistemas de Tiempo Real o embebidos. Estos sistemas plantean requisitos particulares para su programación debido a sus características diferenciadoras. Existe una amplia variedad de definiciones de Sistema de Tiempo Real (STR), por ejemplo, el Oxford Dictionary of Computing lo define como:

Cualquier sistema en el que el tiempo de salida es significativo. Esto generalmente es porque la entrada corresponde a algún movimiento en el mundo físico, y la salida está relacionada con dicho movimiento. El intervalo entre el tiempo de entrada y salida debe ser lo suficientemente pequeño para una temporalidad aceptable.

Otra definición es la de Young:

Cualquier actividad o sistema de proceso de información que tiene que responder

a un estímulo de entrada generado externamente en un periodo finito y especificado.

El proyecto PDCS (Predictably Dependable Computer Systems) proporciona la siguiente definición:

Un STR es aquél al que se le solicita que reaccione a estímulos del entorno (incluyendo el paso de tiempo físico) en intervalos del tiempo dictados por el entorno.

Hay sistemas en los que si la respuesta no se produce en ese tiempo ocurre un desastre, mientras que hay otros en los que un fallo al responder puede ser considerado como una respuesta incorrecta. Por tanto, la corrección de un STR no sólo depende del resultado lógico de la computación, sino también del tiempo en el que se producen los resultados.

Dentro del campo del diseño de STR se distinguen entre STR estrictos (hard) y no estrictos (soft). Los STR estrictos son aquéllos en los que es absolutamente imperativo que las respuestas se produzcan dentro del tiempo límite esperado, por ejemplo, un sistema de control de vuelo de un avión de combate. Los STR no estrictos son aquéllos en los que los tiempos de respuesta son importantes, pero el sistema seguirá funcionando correctamente aunque los tiempos límite no se cumplan ocasionalmente, por ejemplo, un sistema de adquisición de datos. Dentro de un sistema puede haber subsistemas estrictos y subsistemas no estrictos.

El uso del término soft no implica un tipo único de requisito, sino que incorpora un número de propiedades diferentes. Por ejemplo:

- El tiempo límite puede no alcanzarse ocasionalmente, normalmente con un límite superior de fallo en un intervalo definido.
- El servicio se puede proporcionar ocasionalmente tarde, con un límite superior en el retraso.

En un STR el computador normalmente interfiere directamente con algún equipamiento físico, y se dedica a monitorizar o controlar la operación de dicho equipamiento. Una característica fundamental de todas estas aplicaciones es el papel del computador como componente de proceso de información en un sistema de ingeniería más grande. Ésta es la razón por la que tales aplicaciones se conocen como sistemas embebidos.

3.2. Ejemplos.

El primer uso de un computador como componente embebido de ingeniería se produjo en la industria de control de procesos en los 60, en este caso, el computador controla el flujo de líquido en una tubería fuera estable controlando una válvula. Si ocurre un incremento en el flujo, el computador modifica el ángulo de la válvula en un periodo finito. Esta respuesta puede implicar una computación compleja para el cálculo del nuevo ángulo. En este primer ejemplo se presenta un computador embebido en un entorno completo de control de procesos (Figura 3.1). El computador interacciona con el equipamiento utilizando sensores y actuadores. La válvula y el agitador son los actuadores, mientras que un transductor de presión o temperatura es el sensor. El computador controla la operación de los sensores

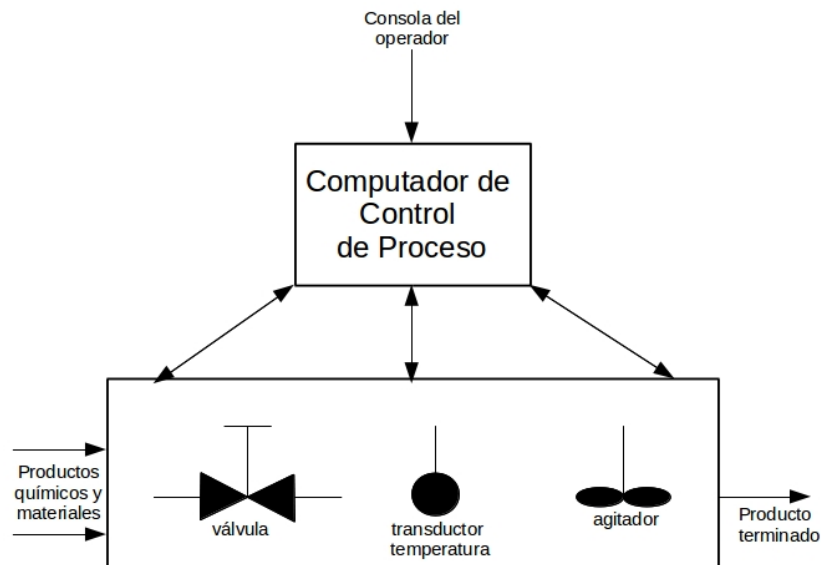


Figura 3.1: Un sistema de control de procesos.

y actuadores para garantizar que las operaciones de la planta se realizan en el tiempos debidos.

En fabricación el uso del computador permite tener bajos los costes de producción e incrementar la productividad. La Figura 3.2 muestra el papel del computador en el control de la producción en el proceso de fabricación. El sistema está formado por una variedad de dispositivos mecánicos controlados y coordinados con el computador. Estos dispositivos pueden ser, por ejemplo, máquinas herramientas, manipuladores o cintas transportadoras.

Existen sistemas que constan de un conjunto complejo de políticas, dispositivos de recogida de información y procedimientos administrativos que permiten apoyar decisiones y proporcionar los medios mediante los cuáles puedan ser implementados. A menudo, los dispositivos de recogida de información y los instrumentos requeridos para implementar decisiones están distribuidos sobre un área geográfica amplia. Ejemplos de estos sistemas son la reserva de plazas de una compañía aérea, las funcionalidades médicas para cuidado automático del paciente, el control del tráfico aéreo o la contabilidad bancaria remota. La Figura 3.3 representa un diagrama de dicho sistema.

La Figura 3.4 muestra un STR típico. El software que controla las operaciones del sistema está estructurado en módulos que reflejan la naturaleza física del entorno. Normalmente habrá un módulo que contenga los algoritmos necesarios para controlar físicamente los dispositivos, un módulo responsable del registro de los cambios del estado del sistema, un módulo para recuperar y presentar dichos cambios, y un módulo para interactuar con el operador.

3.3. Características de los STR.

Un STR tiene unas características que se detallan a continuación:

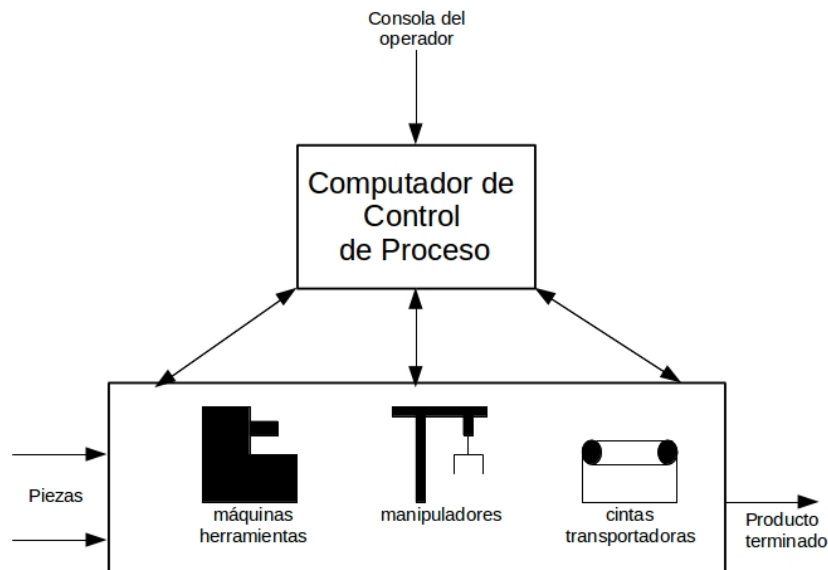


Figura 3.2: Un sistema de control de producción.

Grande y complejo: Problemas en el desarrollo de software debido al tamaño y la complejidad del sistema. Los STR deben responder a eventos del mundo real, suelen ser grandes y en cambio continuo. Los STR precisan un mantenimiento constante y mejoras durante su ciclo de vida. Deben ser extensibles.

Extremadamente fiable y seguro: El hardware y software deben ser fiables y seguros. Los fallos deben ocurrir de una forma controlada. En la interacción con un operador hay que tener cuidado en el diseño de la interfaz para minimizar la posibilidad de error humano. El tamaño y la complejidad de los STR agravan el problema de la fiabilidad, que considera las dificultades esperadas inherentes a la aplicación y las introducidas por un diseño de software defectuoso.

Control concurrente de los distintos de componentes separados del sistema: Los STR suelen constar de varios computadores y elementos externos que funcionan en paralelo. Luego en algunos STR es necesario considerar STR distribuidos o multiprocesadores. Uno de los problemas principales asociados con la producción de software de sistemas que presentan concurrencia es cómo expresar la concurrencia en la estructura del programa.

Funcionalidades de tiempo real: El tiempo de respuesta es crucial en cualquier STR, siendo difícil diseñar e implementar sistemas que garanticen la salida apropiada en los tiempos adecuados y bajo todas las condiciones posibles. Los STR se construyen habitualmente utilizando procesadores con considerable capacidad adicional, garantizando de este modo que el comportamiento en el peor caso no produzca ningún retraso durante los periodos críticos de operación del sistema. Además, el lenguaje de programación debe permitir al programador:

- Especificar los tiempos en los que deben ser inicializadas las acciones.
- Especificar los tiempos en los que las acciones deben ser completadas.

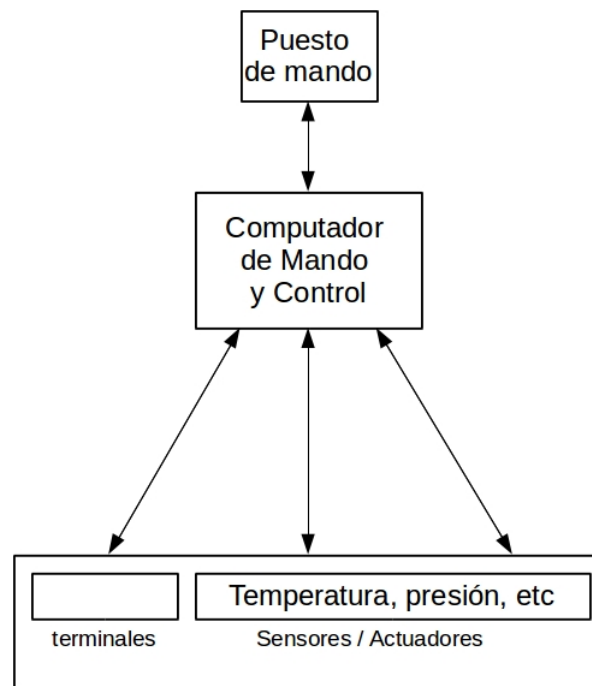


Figura 3.3: Un sistema de mando y control.

- Responder a situaciones en las que no todos los requisitos temporales se pueden satisfacer.
- Responder a situaciones en las que los requisitos temporales cambian dinámicamente.

Interacción con interfaces hardware: Los STR realizan la monitorización de sensores y control de actuadores. Estos dispositivos interactúan con el computador mediante un conjunto de registros, y sus requisitos operativos dependen del dispositivo y computador. Los dispositivos utilizan interrupciones para indicar al procesador que se han realizado ciertas operaciones o que se han alcanzado ciertas condiciones de error. La interacción con los dispositivos se realiza mediante su control directo, y no a través de una capa de funciones del sistema operativo debido al requisito del tiempo real. Además se evita el uso de técnicas de programación de bajo nivel debido a que los requisitos de calidad lo desaconsejan.

Implementación eficiente y entorno de ejecución: La implementación debe ser eficiente para cumplir con los requisitos temporales. El programador debe centrarse en los detalles de implementación, para obtener programas más rápidos, por ejemplo, estudia el coste temporal de la utilización de una determinada característica del lenguaje.

Manipulación de números reales: Relacionado con la teoría de control, sistemas que se modelan mediante un conjunto de ecuaciones diferenciales de primer orden que relacionan la salida del sistema con el estado interno de la planta y sus variables de entrada. Con un computador se resuelven las ecuaciones diferenciales mediante técnicas numéricas. Estos algoritmos son matemáticamente complejos y necesitan un grado de precisión elevado.

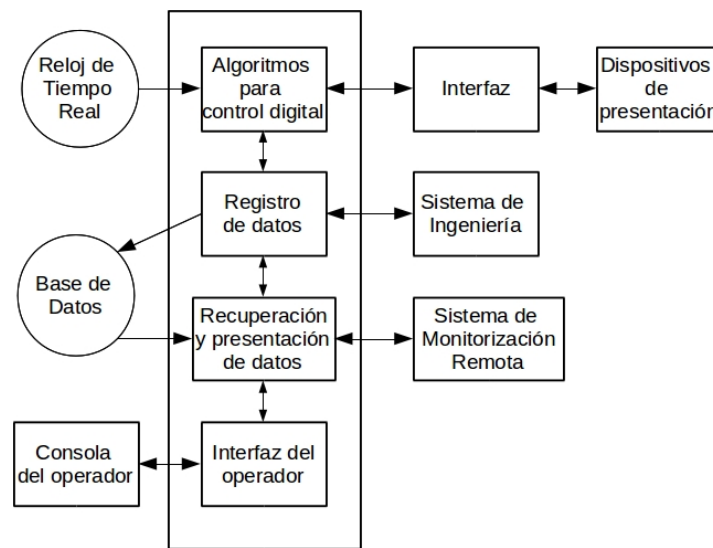


Figura 3.4: Un sistema embebido típico.

3.4. Diseño de STR.

La etapa de diseño consiste en la generación de un diseño que satisfaga una especificación de requisitos. El diseño de STR conlleva problemas de diseño fundamentales. La disciplina de la ingeniería del software se encuentra ampliamente aceptada como el núcleo del desarrollo de métodos, herramientas y técnicas con el fin de asegurar la correcta gestión del proceso de producción de software y la consecución de programas correctos y fiables. La mayoría de las aproximaciones al diseño son descendentes, se fundamentan en la comprensión de qué es realizable en los niveles inferiores, es decir, todos los métodos de diseño incluyen una secuencia de transformaciones desde el estado de los requisitos iniciales hasta el código ejecutable. La mayoría de los métodos tradicionales de desarrollo de software incorporan un modelo de ciclo de vida con las siguientes fases:

1. Especificación de requisitos: Se produce una especificación acreditada del comportamiento funcional y no funcional del sistema.
2. Diseño arquitectónico: Se desarrolla una descripción de alto nivel del sistema propuesto.
3. Diseño detallado: Se especifica el diseño completo del sistema.
4. Implementación o Codificación: Se implementa el sistema.
5. Prueba: Se comprueba la eficacia del sistema.

Antes de pasar a detallar las fases de diseño se hablará sobre la notación necesaria para la documentación de cada etapa. Así, las transformaciones de una etapa hacia otra no son más que traducciones de una notación a otra. Las técnicas de notación (o representación) se pueden clasificar en:

- Informal: Hacen uso del lenguaje natural y de diversos tipos de diagramas imprecisos. Son legibles por todo tipo de usuarios, pero presentan el problema de que pueden interpretarse de diversas formas.
- Estructurada: Suelen usar notación gráfica precisa, a diferencia de los diagramas informales. Los diagramas se crean utilizando un número de componentes menores predefinidas que se conectan de una forma controlada. La forma gráfica puede tener también una representación sintáctica en algún lenguaje bien definido.
- Formal: Deben tener una base matemática que permita analizar y manipular los diagramas. Éstos tienen la ventaja de que se pueden realizar descripciones precisas, y se puede probar la existencia de ciertas propiedades necesarias, por ejemplo, que el diseño de alto nivel satisface la especificación de requisitos. Son difíciles de comprender para quién no han recibido la formación necesaria.

Las empresas especializadas en tiempo real comienzan a emplear técnicas rigurosas de verificación. Debido a que pocos ingenieros de software poseen las habilidades matemáticas necesarias para explotar completamente el potencial de la verificación, y, que las técnicas formales no suelen disponer de herramientas de estructuración adecuadas para grandes especificaciones, la tendencia es emplear una combinación de métodos formales y estructurados. Las siguientes subsecciones detallan las fases de diseño.

3.4.1. Especificación de requisitos.

Consiste en un análisis profundo de los requisitos que definen la funcionalidad del sistema. Para un STR se debe detallar el comportamiento temporal del sistema, los requisitos de fiabilidad y el comportamiento en caso de fallos. También toma mucha importancia la construcción de un modelo del entorno de la aplicación que especifique las interacciones con su entorno, cuestiones como: tasa máxima de interrupciones, el número máximo de objetos externos dinámicos y los modos de fallos. En el análisis de requisitos se emplean técnicas y notaciones estructuradas como:

- PSL y CORE, que tiene ventajas a la hora de obtener un conjunto de requisitos no ambiguos.
- Métodos orientados a objetos, que se han hecho muy populares.
- UML, un estándar industrial muy destacado.
- Métodos formales, como el proyecto FOREST y el proyecto Esprit 11 ICARUS.

Uno de los principales problemas en esta fase es que el cliente no suele mencionar todos los requisitos. Esta fase debe proporcionar una buena especificación de requisitos, de ella saldrá el posterior diseño, y no hay una fase más crítica en todo el ciclo de vida del software.

3.4.2. Diseño arquitectónico y Diseño Detallado.

Las actividades de diseño de un STR grande se estructuran debido al tamaño del sistema. Se suelen emplear dos aproximaciones complementarias que forman conjuntamente la base de la mayoría de los métodos de ingeniería del software:

Descomposición: El sistema complejo se parte en componentes más pequeñas hasta que los componentes pueden ser comprendidos y diseñados por individuos o grupos pequeños. En cada nivel de descomposición, deberá haber un nivel de descripción apropiado y un método para documentar esta descripción.

Abstracción: Permite posponer cualquier consideración referente a los detalles, particularmente las concernientes a la implementación. Así se dispone de una visión simplificada del sistema que contendrá las propiedades y características esenciales.

La descomposición y abstracción provoca las siguientes cualidades en el desarrollo del software:

Encapsulamiento: El desarrollo jerárquico del software conduce a la especificación y desarrollo de los subcomponentes del programa. Por la naturaleza de la abstracción, estos subcomponentes deberían tener roles bien definidos, e interfaces y conexiones claras e inequívocas. Cuando la especificación completa del sistema software puede verificarse teniendo en cuenta únicamente la especificación de los subcomponentes inmediatos, se dice que la descomposición es composicional. Ésta es una propiedad importante cuando se trata de analizar formalmente programas. Los programas secuenciales son particularmente idóneos para los métodos composicionales, y existen técnicas para encapsular y representar subcomponentes como la modularidad y la programación orientada a objetos.

Cohesión y acoplamiento: El encapsulamiento conduce al empleo de módulos con interfaces bien definidas. La cohesión y el acoplamiento son dos medidas que describen la vinculación entre módulos:

- La cohesión expresa el grado de unión conceptual de un módulo, a más relación entre términos representados más cohesión.
- El acoplamiento es una medida de la interdependencia entre dos módulos de un programa. Ocurre un acoplamiento alto, rígido, o fuerte si dos módulos intercambian información de control entre ellos, si sólo intercambian datos el acoplamiento se llama débil. Otra forma de contemplar el acoplamiento es considerar la facilidad con que puede eliminarse un módulo de un sistema completo y reemplazarlo con un módulo alternativo.

Una buena descomposición modular es aquélla que posee una cohesión fuerte y un acoplamiento débil.

Aproximaciones formales: Uno de los modelos más destacados para el modelado de sistemas concurrentes son las redes Petri. Tienen las siguientes ventajas:

- Se definen matemáticamente y se puede realizar un análisis formal de ellas.
- Son simples, abstractas y gráficas, y dotan de las herramientas para analizar sistemas distribuidos y concurrentes, aunque suelen ser representaciones muy grandes y complicadas.

Otras alternativas formales son:

- **Autómatas temporizados y comprobación de modelos (Model Checking):** El sistema concurrente se representa mediante un conjunto de máquinas de estado finito, y se emplean técnicas de exploración de estados para investigar los posibles comportamientos del sistema.

- Utilización de occam: Un lenguaje de programación que fue creado con una descripción formal, lo que facilita la implementación en base al diseño.
- Communicating Sequential Processes (CSP): Permite la especificación y el análisis de sistemas concurrentes. Se usan eventos externos para describir cada proceso, de acuerdo con la comunicación que mantiene con otros procesos. La traza, secuencia finita de eventos, muestra la evolución de un proceso.
- Lógica temporal: Es una extensión del cálculo de proposiciones y de predicados que añade nuevos operadores para expresar propiedades relativas al tiempo real. Los operadores son del tipo: siempre, en-algún-momento, hasta, desde y conduce-a.

Las cualidades que un método de diseño deben tener son:

1. Estructurar el STR en tareas concurrentes.
2. Dar soporte al desarrollo de componentes reusables mediante la ocultación de información.
3. Definir los aspectos del comportamiento mediante máquinas de estado finito.
4. Analizar las prestaciones de un diseño para determinar sus propiedades de tiempo real.

Para los STR estrictos se presenta la desventaja importante de que los problemas de temporización sólo se podrán detectar durante la prueba o tras la instalación de la aplicación.

Para terminar la sección se enumeran brevemente los métodos de diseño más destacados. Dos técnicas de diseño usadas ampliamente en STR son JSD y Mascot3. Posteriormente, se presentó un método específico para Ada denominado HRT-HOOD. Es un método especialmente indicado para STR estrictos. No se detallará ninguno de estos métodos.

3.4.3. Implementación.

En esta fase, el primer paso es la elección del lenguaje de programación para la implementación del STR. Debe ser un lenguaje apropiado, que permita implementar apropiadamente todas las funciones del STR. El lenguaje debe permitir la descomposición y la abstracción. El concepto que permite estas características es el módulo, que es un conjunto de objetos y operaciones sobre ellos relacionadas lógicamente. El encapsulamiento es la técnica de aislar una función del sistema en un módulo y proporcionar una especificación precisa para la interfaz del módulo. La estructura modular consigue:

Ocultación de información: La estructura de módulo soporta una visibilidad reducida al permitir que la información esté oculta dentro del cuerpo del módulo. Todas las estructuras de módulo permiten que el programador controle el acceso a las variables del módulo. En C, el programador debe utilizar archivos separados: un archivo cabecera con extensión “.h” y un archivo cuerpo con extensión “.c”. En el lenguaje C no existe relación formal entre un archivo “.h” y uno “.c”, aunque sí conceptual, y está establecida como una norma entre los programadores C. En C, no se detectará la inexistencia del cuerpo de una función hasta el momento del montaje. La construcción modular en la programación de STR es importantísima.

Compilación por separado: Los programadores se centran en el módulo actual y pueden probar cada módulo avanzando la implementación del programa poco a poco. Una vez probada la nueva unidad puede incluirse en forma precompilada en la biblioteca. El ahorro de recursos y el apoyo a la gestión del proyecto mejoran claramente si no hay que recompilar todo el programa por cada pequeño cambio.

Una característica importante es que la especificación y cuerpo de un módulo son contemplados en la biblioteca como entidades distintas. Puede que una biblioteca en desarrollo sólo contenga especificaciones que podrán utilizarse para comprobar la consistencia lógica del programa antes de su implementación en detalle. En el contexto de la gestión de un proyecto, las especificaciones serán elaboradas por el personal con mayor experiencia; y esto es así porque las especificaciones representan interfaces entre módulos software. Un error en este código es más importante que uno en el cuerpo del módulo, ya que un cambio en la especificación conlleva tener que cambiar y recompilar cada módulo que utiliza éste, mientras que un cambio en el cuerpo sólo requiere la recompilación del cuerpo.

La compilación por separado permite la programación de abajo a arriba. Las unidades de biblioteca se construyen a partir de otras unidades; así hasta que se pueda codificar el programa final. La programación “de abajo a arriba”, en el contexto del diseño descendente, es ascendente en cuanto a las especificaciones (definiciones), no a las implementaciones (cuerpos), por lo que resulta bastante conveniente.

Tipos abstractos de datos: Los programadores no tienen que preocuparse de la representación física de los datos en el computador (tipos de datos). Los tipos abstractos de datos (TAD) son una ampliación de este concepto. Para definir un TAD, un módulo dará nombre a un nuevo tipo, y después proporcionará todas las operaciones que pueden ser aplicadas a ese tipo. La estructura del TAD se oculta dentro del módulo. Al estar permitida más de una instancia de cada tipo, es preciso incluir en la interfaz del módulo una rutina de creación.

Los TAD se complican por el hecho de requerir la compilación por separado de la especificación de un módulo de la de su cuerpo. Dado que la estructura de un TAD debe estar oculta, el lugar lógico de definición es el cuerpo. Pero entonces el compilador no conocerá el tamaño del tipo cuando compila código que utiliza la especificación.

Se pueden identificar tres clases de lenguajes de programación para el desarrollo de STR:

Lenguajes ensamblador: Inicialmente, la mayoría de los sistemas de tiempo real se programaron en el lenguaje ensamblador del computador embebido, debido a que la mayoría de los lenguajes de alto nivel no disponían del soporte suficiente para la mayoría de los microcomputadores, y el ensamblador era la única forma de obtener una implementación eficiente. El ensamblador está orientado a máquina y hace que los costes de desarrollo se eleven considerablemente. Los programas son prácticamente ilegibles, salvo por su autor, lo que hace difícil modificarlos. También dificulta la tarea de llevar los programas a otra máquina.

Lenguajes de implementación de sistemas secuenciales: Con el tiempo se dotó a los lenguajes de programación de la generalidad, es decir, los programas se hicieron independientes de la máquina, también se consiguió hacer programas eficientes con lenguajes de programación estructurados. Los lenguajes secuenciales suelen presentar carencias en lo que respecta a sus funciones de control y fiabilidad para tiempo real. Un ejemplo de estos lenguajes es el Lenguaje C.

Lenguajes de programación concurrente de alto nivel: Incluyen aspectos orientados a ayudar al desarrollo de STR, algunos destacados son: PEARL, Mesa, CHILL, Java y Ada. El lenguaje occam, por contra, es un lenguaje mucho más reducido. No da un soporte auténtico para la programación de sistemas grandes y complejos, aunque proporciona las estructuras de control habituales, así como procedimientos no recursivos. Occam tuvo un importante papel en el campo emergente de las aplicaciones embebidas distribuidas débilmente acopladas, en los últimos años su popularidad ha ido desvaneciéndose.

Según Young, los criterios generales de diseño de lenguajes son seis, una lista similar aparece en los requisitos originales de Ada:

1. Seguridad: Es la medida del grado en el que el compilador, o el sistema de soporte en tiempo de ejecución, pueden detectar automáticamente los errores de programación. Existe un límite en la cantidad y en los tipos de errores que pueden ser detectados por el sistema del lenguaje, por ejemplo, no es posible detectar de modo automático los errores de lógica del programador. Un alto grado de seguridad aporta una detección temprana de errores en el desarrollo del programa, y la ejecución del proceso no se sobrecarga al realizar las comprobaciones en tiempo de compilación. El inconveniente es que puede llegar a complicar el lenguaje e incrementar tanto el tiempo de compilación así como la complejidad del compilador.
2. Legibilidad: Depende de un conjunto de factores, como una elección adecuada de palabras clave, la facilidad para definir tipos, y los mecanismos de modularización de programas. Una buena legibilidad aporta menor coste de documentación, mayor seguridad y mayor facilidad de mantenimiento. El principal inconveniente es la mayor longitud de los programas.
3. Flexibilidad: Consiste en la permisibilidad al programador para expresar todas las operaciones oportunas de una forma directa y coherente. De otro modo, el programador tendrá que recurrir a funciones del sistema operativo o a fragmentos en código máquina para obtener el resultado deseado.
4. Simplicidad: Aporta las ventajas de: (1) Minimizar el esfuerzo de producción de compiladores; (2) reducir el coste de formación de programadores; (3) y disminuir la posibilidad de cometer errores de programación como consecuencia de una mala interpretación de las características del lenguaje.
5. Portabilidad: Un programa deberá ser independiente, en la medida de lo posible, del hardware sobre el que se ejecuta. Para un STR, esto es difícil de lograr, dado que una parte importante de cualquier programa estará involucrada en la manipulación de recursos hardware. A pesar de ello, un lenguaje deberá ser capaz de aislar la parte del programa que depende de la máquina de la parte independiente de la máquina.
6. Eficiencia: Los tiempos de respuesta deben estar garantizados, el lenguaje deberá permitir producir programas eficientes y predecibles, eliminando sobrecargas en tiempo de ejecución.

Para terminar comentar que el ciclo de desarrollo del software conlleva el problema de que un fallo en los requisitos iniciales sólo se detecta en la entrega del producto. Esto es económicamente muy costoso. El prototipado intenta descubrir estos fallos lo antes posible, presentando al cliente una primera versión del sistema. Esto permite ver si la especificación

de requisitos es correcta (en términos de lo que desea el cliente) y completa (ha incluido el cliente todo lo que desea).

Por otro lado, la interfaz gráfica (interfaz HMI) es crítica, por ello, las actividades de HMI deben encontrarse aisladas del resto del sistema (modularidad) y con especificaciones de interfaces bien definidas. El principal objetivo en el diseño de la componente de interfaz es la captura de todos los errores derivados del usuario en el programa de control de la interfaz. Los principios de diseño HMI más importantes son:

1. Predecibilidad: Proporcionar datos suficientes al usuario para que conozca los efectos de las operaciones realizadas sobre el STR.
2. Conmutatividad: El orden en que el usuario cumplimenta los parámetros de una operación no debe afectar al sentido de la operación.
3. Sensibilidad: Si el sistema puede presentar diferentes modos de operación, deberá mostrarse siempre el modo actual.

El diseño HMI debe ayudar a la satisfacción en el trabajo de los operadores y a las prestaciones mediante:

1. El nivel de control de que dispone el operador.
2. El grado en que el operador comprende la operación del sistema completo.
3. El número de tareas constructivas que se le permite realizar al operador.
4. La multitud de tareas interdependientes que hay que llevar a cabo.

El operador comprende todo mejor si se presenta una imagen de qué está pasando. Es posible construir sistemas de control de procesos que ilustren el comportamiento del sistema en forma gráfica, pictográfica, o como una hoja de cálculo. El operador puede experimentar con los datos para modelar formas de mejorar las prestaciones del sistema. Es muy razonable incorporar tales sistemas de ayuda a la decisión en el bucle de control, de modo que el operador pueda ver el efecto de cambios menores sobre los parámetros de la operación. De esta forma, se puede incrementar la productividad y mejorar el entorno de trabajo de los operadores. Además, también se reduce el número de equivocaciones que se cometen.

3.4.4. Prueba.

Las pruebas deben ser extremadamente exigentes. El problema de los programas concurrentes de tiempo real es que los errores de sistema más difíciles de tratar surgen habitualmente de sutiles interacciones entre procesos, y dependen del instante de tiempo, y sólo se manifestarán en situaciones poco comunes. Destacar que los métodos apropiados de diseño formal no evitan la necesidad de hacer pruebas: son estrategias complementarias.

Como ayuda para cualquier tarea de prueba compleja están los simuladores. Un simulador es un programa que imita las acciones del sistema en el que se encuentra inmerso el software de tiempo real. Simula la generación de interrupciones y efectúa otras acciones de E/S en tiempo real. Empleando un simulador, podrán crearse situaciones de comportamiento “normales” y “anormales”. Incluso cuando el sistema final haya sido completado,

determinados estados de error solo podrán ser experimentados de un modo seguro mediante un simulador. Los simuladores pueden reproducir con exactitud la secuencia de eventos que se esperan del sistema real. Además, es posible repetir experimentos en modos que normalmente son imposibles de obtener en el funcionamiento en vivo. Los simuladores son sistemas caros de desarrollar, y suponen un esfuerzo considerable. Además, pueden requerir hardware especial.

3.5. Fiabilidad y tolerancia a fallos.

Esta sección desarrolla los conceptos de fiabilidad y tolerancia a fallos. En la siguiente subsección se estudiará la Fiabilidad, y posteriormente las técnicas para la fiabilidad.

3.5.1. Fiabilidad.

Los requisitos de fiabilidad y seguridad de un STR son mucho más estrictos que los habituales debido a que si falla puede que el fallo no sea recuperable. Un fallo en un STR puede ser calificado de la siguiente manera:

1. Especificación inadecuada.
2. Defectos provocados por errores de diseño en los componentes de software.
3. Defectos provocados por fallos en uno o más componentes del procesador del STR.
4. Defectos provocados por una interferencia transitoria o permanente en el subsistema de comunicaciones subyacente.

Los errores introducidos por defectos de diseño suelen ser no previstos (en cuanto a sus consecuencias), mientras que los debidos a fallos en el procesador o en la red son “predecibles”. Los lenguajes de programación de tiempo real deben facilitar la construcción de sistemas fiables.

Randell define la fiabilidad de un sistema como:

Una medida del éxito con el que el sistema se ajusta a alguna especificación definitiva de su comportamiento.

Esta especificación debería ser completa, consistente, comprensible, no ambigua, y, debe tener en cuenta que los tiempos de respuesta. La anterior definición de fiabilidad también se puede utilizar para definir un fallo del sistema:

Un fallo ocurre cuando el comportamiento de un sistema se desvía del especificado para él.

Un fallo es consecuencia de un error, por ejemplo, un error en el hardware. Se pueden distinguir tres tipos de fallos:

1. Fallos transitorios: Comienza en un instante de tiempo concreto, se mantiene en el sistema durante algún periodo, y luego desaparece. Ejemplos de este tipo de fallos se pueden deber a componentes hardware en los que se produce una reacción adversa a una interferencia externa, después la perturbación desaparece y el fallo también.
2. Fallos permanentes: Comienzan en un instante determinado y permanecen en el sistema hasta que son reparados.
3. Fallos intermitentes: Son fallos transitorios que ocurren de vez en cuando, por ejemplo: un componente hardware sensible al calor, que funciona durante un rato, deja de funcionar, se enfría, y entonces comienza a funcionar de nuevo.

Un sistema puede fallar de varias maneras diferentes. Un diseñador que utiliza un sistema S_1 para implementar un sistema S_2 , usualmente realiza alguna suposición sobre los modos de fallo esperados de S_1 . Si S_1 falla de forma diferente a lo esperado, entonces el sistema S_2 puede fallar como consecuencia de ello. Se pueden clasificar los modos de fallo de un sistema de acuerdo con el impacto que tendrán en los servicios que proporciona:

1. Fallos de valor: El valor asociado con el servicio es erróneo.
2. Fallos de tiempo: El servicio se completa a destiempo.
3. Fallo arbitrario: Combinación de fallo de valor y de tiempo.

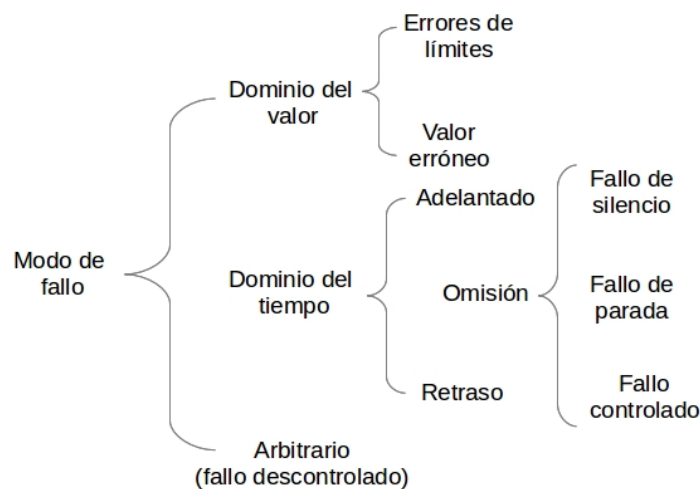


Figura 3.5: Modos de fallo de un STR.

La Figura 3.5 muestra la clasificación de los modos de fallo [2]:

- Dominio del valor: Fallos en el valor obtenido, se reconocen fácilmente:
 - Error de límites: Valor fuera del rango esperado para ese servicio, equivalente a un error de escritura en los lenguajes de programación.
 - Valor erróneo: Valor obtenido no correcto, pero dentro del rango de valores.

- Dominio del tiempo: Fallo en el tiempo de entrega:
 - Fallo por Adelantado: El servicio se entrega antes de lo requerido.
 - Fallo por Omisión: El servicio nunca se entrega.
 - Fallo de silencio: Un sistema que produce servicios correctos tanto en el dominio del valor como en el del tiempo, hasta que falla. El único fallo posible es un fallo de omisión, y cuando ocurre todos los servicios siguientes también sufrirán fallos de omisión.
 - Fallo de parada: Un sistema que tiene todas las propiedades de un fallo silencioso, pero que permite que otros sistemas puedan detectar que ha entrado en el estado de fallo de silencio.
 - Fallo controlado: Un sistema que falla de una forma especificada y controlada.
 - Fallo de retraso: Un sistema que produce servicios correctos en el dominio del valor, pero el servicio se entrega después de lo requerido (error de prestaciones).
- Fallo descontrolado: Un sistema que produce errores arbitrarios, tanto en el dominio del valor como en el del tiempo incluyendo errores de improvisación.
- Sin fallos: Un sistema que siempre produce los servicios correctos, tanto en el dominio del valor como en el del tiempo.

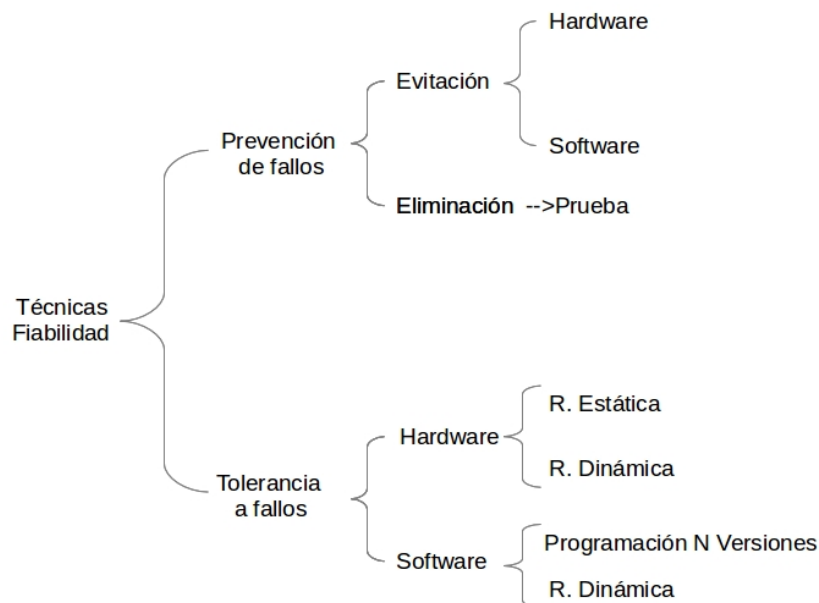


Figura 3.6: Clasificación de las técnicas de fiabilidad.

3.5.2. Técnicas para la fiabilidad.

Para mejorar la fiabilidad de los STR se usan dos técnicas: (1) la prevención de fallos; (2) y la tolerancia a fallos. La prevención de fallos intenta impedir cualquier fallo en el sistema antes de que esté operativo. La tolerancia a fallos permite que el sistema continúe

funcionando incluso ante la presencia de fallos. Ambas aproximaciones intentan producir sistemas con modos de fallo bien definidos. La Figura 3.6 muestra una clasificación de las técnicas de fiabilidad. Las siguientes subsecciones detallan cada una de ellas y sus tipos.

3.5.2.1. Prevención de fallos.

Existen dos fases en la prevención de fallos: evitación y eliminación. Con la evitación se intenta limitar la introducción de componentes potencialmente defectuosos durante la construcción del sistema. En lo que respecta al hardware, esto puede implicar:

- La utilización de los componentes más fiables.
- La utilización de técnicas refinadas para la interconexión de componentes y el ensamblado de subsistemas.
- El aislamiento del hardware para protegerlo de formas de interferencia esperadas.

Los componentes software de los STR actualmente son mucho más complejos que sus correspondientes elementos hardware. La calidad del software se puede mejorar mediante:

- Especificaciones de requisitos rigurosas o formales.
- Utilización de probadas metodologías de diseño.
- Utilización de lenguajes que faciliten la abstracción de datos y la modularidad.
- Uso de herramientas de ingeniería del software para ayudar en la manipulación de los componentes software y en la gestión de la complejidad.

La segunda fase es la eliminación de fallos, dado que los fallos ocurren aunque se intenten evitar. La eliminación de fallos especifica los procedimientos para encontrar y eliminar las causas de los errores. El método más usual es la prueba del sistema, que nunca puede ser exhaustiva y eliminar todos los potenciales fallos. Existen los siguientes problemas:

- Una prueba sólo demuestra la presencia de fallos, no su ausencia.
- Resulta imposible realizar pruebas bajo condiciones reales, por ello se usan simuladores.
- Los errores que han sido introducidos en la especificación de requisitos pueden que no se manifiesten hasta que el sistema está operativo.

3.5.2.2. Tolerancia a fallos.

Debido a las limitaciones de la prevención de fallos, los diseñadores de STR deben considerar recurrir a la tolerancia a fallos. Un STR puede proporcionar diferentes niveles de tolerancia a fallos:

- Tolerancia total frente a fallos: El sistema continúa en funcionamiento en presencia de fallos, aunque por un periodo limitado, sin una pérdida significativa de prestaciones.

- Degradación controlada o caída suave: El sistema continúa en operación en presencia de errores, aceptándose una degradación parcial de las prestaciones durante la recuperación o la reparación.
- Fallo seguro: El sistema cuida de su integridad durante el fallo aceptando una parada temporal de su funcionamiento.

El nivel de tolerancia a fallos requerido dependerá de la aplicación. Algunos sistemas pueden proporcionar varios grados de degradación controlada. También se hace necesaria la degradación controlada en aquellas aplicaciones altamente complejas que tengan que funcionar de forma continua, ya que la tolerancia total frente a fallos no es alcanzable para periodos indefinidos. En otras situaciones, simplemente resulta necesario apagar el sistema en un estado seguro, intentando limitar el alcance del daño causado por el fallo.

Las primeros acercamientos al diseño de sistemas tolerantes a fallos realizaban tres suposiciones:

1. Los algoritmos del sistema han sido diseñados correctamente.
2. Se conocen todos los posibles modos de fallos de los componentes.
3. Se han tenido en cuenta todas las posibles interacciones entre el sistema y el entorno.

Sin embargo, la creciente complejidad del software y la introducción de componentes hardware VLSI han hecho que no se puedan seguir haciendo esas suposiciones. Consecuentemente, se debe considerar tanto los fallos previstos como los imprevistos, incluyendo los fallos de diseño software y hardware.

Todas las técnicas utilizadas para conseguir tolerancia a fallos se basan en la redundancia, es decir, añadir elementos al sistema para que detecte y se recupere de los fallos. En el funcionamiento normal no son necesarios. La intención última de la tolerancia a fallos es minimizar la redundancia y maximizar la fiabilidad proporcionada. Se debe tener cuidado al estructurar los sistemas tolerantes a fallos, ya que los distintos componentes incrementan inevitablemente la complejidad de todo el sistema. Además, puede conducir a sistemas menos fiables. Para ayudar a reducir los problemas asociados con la interacción entre los componentes redundantes, resulta aconsejable separar los componentes tolerantes a fallos del resto del sistema.

Existen varias clasificaciones diferentes de redundancia, dependiendo de los componentes considerados y de la terminología utilizada. No se hablará mucho de las técnicas de **redundancia hardware** debido a que esto se hizo en el Capítulo 1. Sólo comentar que en el caso del hardware se distingue entre redundancia estática (o enmascarada) y dinámica. En la redundancia estática, los componentes redundantes son utilizados dentro de un sistema (o subsistema) para ocultar los efectos de los fallos. Un ejemplo de redundancia estática es la redundancia triple modular (TMR) que consiste en tres subcomponentes idénticos y en circuitos de votación por mayoría. Los circuitos comparan la salida de todos los componentes, y si alguna difiere de las otras dos es bloqueada. Aquí se supone que el fallo no se debe a un aspecto común de los subcomponentes (como un error de diseño), sino a un error transitorio o al deterioro de algún componente. Resulta evidente que se necesita más redundancia para enmascarar fallos de más de un componente. Para denominar a este enfoque se utiliza el término general de redundancia N modular (NMR).

La redundancia dinámica es la redundancia aportada dentro de un componente que hace que el mismo indique, explícita o implícitamente, que la salida es errónea. De este modo, se

proporciona la capacidad de detección de errores, más que la de enmascaramiento del error; la recuperación debe ser proporcionada por otro componente. Las sumas de comprobación (checksums) en las transmisiones de comunicaciones y los bits de paridad en las memorias son ejemplos de redundancia dinámica.

Respecto a la **redundancia software**, se pueden identificar dos enfoques. El primero es análogo a la redundancia enmascarada del hardware, y se llama programación de N-versiones. El segundo se basa en la detección y la recuperación de errores, y es análogo a la redundancia dinámica, en el sentido de que se activan los procedimientos de recuperación después de haberse detectado un error.

Programación de N-versiones.

La programación de N-versiones consiste en la implementación de N programas funcionalmente equivalentes (con N mayor o igual a 2) a partir de la misma especificación inicial. El proceso consiste en que N individuos o grupos producen N versiones necesarias del software sin interacción entre ellos. Posteriormente, los programas se ejecutan de forma concurrente con las mismas entradas, y un programa director compara sus resultados. El resultado final ofertado es decidido por consenso.

La programación de N-versiones se basa en la suposición de que se puede especificar completamente un programa de forma consistente y sin ambigüedad, y que los programas que han sido desarrollados independientemente fallarán de forma independiente. Esto es, que no existe relación entre los fallos de una versión y los fallos de otra. Por ello se debe usar diferentes lenguajes para cada versión para evitar los errores asociados con la implementación del lenguaje. Si se utiliza el mismo lenguaje, los compiladores y los entornos utilizados deben ser de distintos fabricantes. Además las N-versiones deben ser distribuidas entre diferentes máquinas con líneas de comunicación tolerantes a fallos. Como puede verse, cada versión debe ser totalmente independiente de la otra.

Existe comunicación entre el programa director y las N-versiones, para evitar problemas como la no finalización de los mismos. Esta interacción se especifica en los requisitos de las versiones, y se compone de tres elementos:

1. Vectores de comparación: Estructuras de datos que representan las salidas (llamadas votos) producidos por las versiones, además de cualquier atributo asociado con su cálculo. Serán comparados por el director.
2. Indicadores de estatus de la comparación: Indican las acciones que cada versión debe ejecutar como resultado de la comparación realizada por el director.
3. Puntos de comparación: Puntos dentro de las versiones donde deben comunicar sus votos al director.

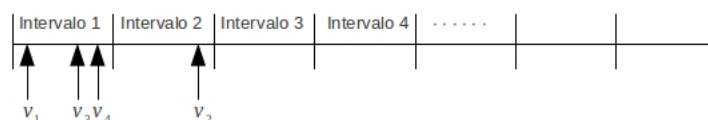


Figura 3.7: Ejemplo de comparación por intervalos.

En la programación de N-versiones resulta crucial la eficiencia y la facilidad con la que el programa director compara los votos. En aplicaciones en las que se manipula texto o

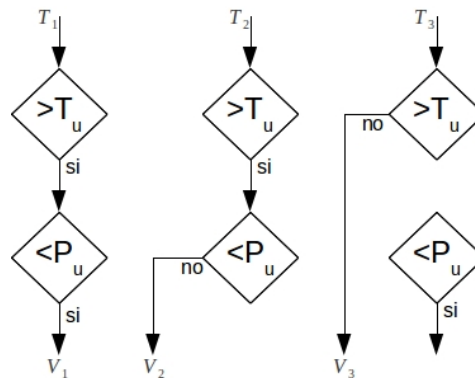


Figura 3.8: Ejemplo del problema de comparación consistente.

operaciones aritméticas sobre enteros habrá un único resultado correcto y la comparación de resultados es sencilla. En el caso de cálculos con números reales es improbable que diferentes versiones produzcan “exactamente” el mismo resultado (precisión), por ejemplo, debida a la representación hardware de los números reales, o a la sensibilidad de los datos respecto a un algoritmo concreto. Las técnicas utilizadas en la comparación de estos tipos de resultados se denominan “votación inexacta”. Una técnica simple consiste en utilizar intervalos (Figura 3.7), es decir, el dominio se parte en intervalos y se cuentan los votos que caen en cada intervalo del dominio. Otra dificultad asociada a números reales es el llamado problema de la comparación consistente que ocurre cuando se tiene que realizar una comparación basada en un valor finito dado en la especificación; el resultado de la comparación determina entonces el curso de la acción (Figura 3.8).

El éxito de la programación de N-versiones depende de varios aspectos:

- **Especificación inicial:** Un error de especificación se manifestará en todas las versiones de la implementación.
- **Independencia en el diseño:** Se han realizado algunos experimentos que obtienen resultados conflictivos. Sin embargo, existe la certeza de que allí donde la especificación resulte compleja, inevitablemente se producirá una distorsión de la comprensión de los requisitos entre los equipos independientes. Si estos requisitos también se refieren a datos de entrada poco frecuentes: entonces los errores de diseño comunes pueden no ser capturados en la fase de pruebas. Se ha encontrado que un sistema de tres versiones es entre 5 y 9 veces más fiable que un sistema de alta calidad de versión única.
- **Presupuesto adecuado:** El principal coste se debe al software. Un sistema de N versiones multiplicará por N el presupuesto, y causará más problemas de mantenimiento. Esto hace que la técnica de la programación de N-versiones sólo se suele usar cuando es obligatoria. Además, no está claro que no se pueda producir un sistema más fiable si los recursos potencialmente disponibles para construir las N versiones fueran utilizados para construir una única versión.

La técnica de las N versiones debe ser utilizada con cuidado junto con otras técnicas debido a los problemas expuestos anteriormente.

Redundancia dinámica.

Con la redundancia dinámica los componentes redundantes sólo se activan cuando se ha detectado un error. Esta técnica utiliza cuatro fases:

1. Detección de errores: No se utiliza ningún esquema de tolerancia a fallos hasta que se haya detectado un error. Se pueden identificar dos clases de técnicas de detección de errores:
 - a) Detección en el entorno: Se detectan en el entorno en el cual se ejecuta el programa, incluyen errores detectados por hardware (instrucción ilegal, desbordamiento aritmético, violación de protección, etc) y software (error por límite en vector, referencia de puntero nulo, valor fuera de rango, etc). Estos errores de programación permiten ser detectados por algunos lenguajes de programación.
 - b) Detección en la aplicación: Los errores se detectan por la misma aplicación. Se distinguen las siguientes técnicas:
 - Comprobación de réplicas: Programación de N-versiones como técnica para la detección de errores.
 - Comprobaciones temporales: Existen dos tipos de comprobaciones temporales. El primer tipo implica un proceso temporizador guardián, que si no es puesto a cero por un cierto componente dentro de un cierto periodo de tiempo se supone que dicho componente está en un estado de error. El componente software debe poner a cero el temporizador de forma continua para indicar que está funcionando correctamente. En los sistemas embebidos, donde los tiempos de respuesta son importantes, se necesita un segundo tipo de comprobación. De esta manera se detectan fallos asociados con el incumplimiento de tiempos límite (deadlines). Cuando el sistema de soporte de ejecución planifique tiempos límite, la detección de los fallos asociados con el incumplimiento se puede realizar dentro del entorno. Por supuesto que las comprobaciones temporales no aseguran que un componente esté funcionando correctamente, sino sólo que está funcionando a tiempo.
 - Comprobaciones inversas: Éstas son posibles en componentes donde exista una relación uno a uno (isomórfica) entre la entrada y la salida. Una comprobación de este tipo toma la salida, calcula la entrada correspondiente, y la compara con el valor de la entrada real, por ejemplo, la raíz cuadrada de un número.
 - Códigos de comprobación: Los códigos de comprobación se utilizan para comprobar la corrupción de los datos. Se basan en la redundancia de la información contenida en los datos. Por ejemplo, se puede calcular un valor en función de los datos de entrada (checksum) y enviarlo con éstos a través de una red de comunicación, cuando se reciben los datos, el valor se recalcula y se compara con el checksum recibido.
 - Comprobaciones de racionalidad: Se basan en el conocimiento del diseño y construcción del sistema. Comprueban que el estado de los datos o el valor de un objeto es "razonable". Por ejemplo, los valores enteros se pueden restringir entre ciertos valores razonables en la aplicación y se representan como subtipos de enteros con rangos explícitos. La violación de rango se puede detectar por el sistema de soporte en tiempo de ejecución.
 - Comprobaciones estructurales: Se utilizan para comprobar la integridad de los objetos de datos tales como listas o colas. Podrían consistir en contar el

- número de elementos en el objeto, en punteros redundantes, o en información extra sobre su estatus.
- Comprobaciones de racionalidad dinámica. En la salida producida por algunos controladores digitales, existe una relación entre dos salidas consecutivas. Por lo tanto, se podrá detectar un error si el valor de una salida nueva difiere considerablemente del valor de la salida anterior.
2. Confinamiento y valoración de daños: Cuando se detecte un error, debe estimarse la extensión del sistema que ha sido corrompida; el intervalo de tiempo entre el fallo y la manifestación del error asociado es debido a que la información errónea tiene que difundirse a través del sistema. El tipo de error detectado dará alguna idea a la rutina de tratamiento del error sobre el daño. La valoración de los daños estará estrechamente relacionada con las precauciones que hayan sido tomadas por los diseñadores del sistema para el confinamiento del daño causado. El confinamiento del daño se refiere a la estructuración del sistema de modo que se minimicen los daños causados por un componente defectuoso.
 3. Recuperación del error: Es la fase más importante de cualquier técnica de tolerancia a fallos. Se debe transformar un estado erróneo del sistema en otro desde el cual el sistema pueda continuar con su funcionamiento normal, aunque quizás con una cierta degradación en el servicio. Respecto a la recuperación de errores, se han propuesto dos estrategias: recuperación hacia adelante (forward) y hacia atrás (backward). La *recuperación de errores hacia adelante* intenta continuar desde el estado erróneo realizando correcciones selectivas en el estado del sistema. En los STR, esto puede implicar proteger cualquier aspecto del entorno controlado que pudiera ser puesto en riesgo o dañado por el fallo. Aunque la recuperación de errores hacia adelante puede resultar eficiente, es específica de cada sistema, y depende de la precisión de las predicciones sobre la localización y la causa del error (esto es, de la valoración de los daños). La *recuperación hacia atrás* se basa en restaurar el sistema a un estado seguro previo a aquél en el que se produjo el error, para luego ejecutar una sección alternativa del programa con un algoritmo diferente. Se espera que esta alternativa no provoque el mismo defecto de forma recurrente. Al punto al que el proceso es restaurado se le denomina punto de recuperación (checkpoint), y al proceso de su establecimiento normalmente se le denomina checkpointing. Para establecer un punto de recuperación, es necesario salvar en tiempo de ejecución la información apropiada sobre el estado del sistema. La restauración de estado tiene la ventaja de que el estado erróneo ha sido eliminado y no se tiene que buscar la localización o la causa del defecto. La recuperación hacia atrás de errores se puede usar para recuperar defectos no considerados, incluyendo los errores de diseño. Sin embargo, su desventaja es que no puede deshacer ninguno de los efectos que el defecto pudiera haber producido en el entorno del STR (lanzamiento de un misil). Además, la recuperación de errores hacia atrás consume tiempo de ejecución, lo cual puede excluir su uso en algunas aplicaciones de tiempo real.
 4. Tratamiento del fallo y continuación del servicio: Un error es un síntoma de un defecto; aunque el daño pudiera haber sido reparado, el error continua existiendo, y por lo tanto el error puede volver a darse a menos que se realice algún tipo de mantenimiento. El tratamiento automático de fallos resulta difícil de implementar, y tiende a ser específico de cada sistema. Algunos sistemas no disponen de tratamiento de fallos, asumiendo que son transitorios. Otros suponen que las técnicas de tratamiento de errores son lo suficientemente potentes para tratar los fallos recurrentes.

Esta técnica se pueden aplicar a la programación de N-versiones: (1) la detección de erro-

res es realizada por el director que realiza la comprobación de los votos; (2) no se requiere la valoración de daños, ya que las versiones son independientes; (3) la recuperación de errores implica descartar el resultado erróneo, (4) y el tratamiento del fallo se reduce a ignorar la versión que ha producido el valor erróneo. Si todas las versiones producen votos diferentes, entonces se tiene que llevar a cabo la detección de errores, aunque no existen posibilidades de recuperación.

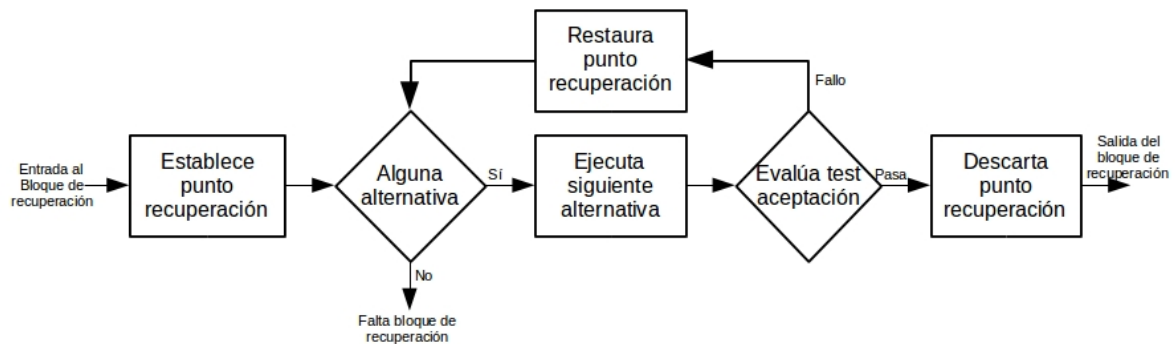


Figura 3.9: Funcionamiento del bloque de recuperación.

Bloques de recuperación en la tolerancia a fallos software.

Los bloques de recuperación son bloques en el sentido normal de los lenguajes de programación, excepto que en la entrada del bloque se encuentra un punto de recuperación automático, y en la salida un test de aceptación. El test de aceptación se utiliza para comprobar que el sistema se encuentra en un estado aceptable después de la ejecución del bloque. La Figura 3.9 muestra su funcionamiento. El fallo en el test de aceptación provoca que el programa sea restaurado al punto de recuperación del principio del bloque, y que se ejecute un módulo alternativo. Si el módulo alternativo también falla en el test de aceptación, entonces el programa se restaura otra vez al punto de recuperación, y entonces se ejecuta otro módulo, y así sucesivamente. Si todos los módulos fallan entonces el bloque falla, y la recuperación debe llevarse a cabo a un nivel superior.

En relación con las cuatro fases de la tolerancia a fallos software se tiene que: la detección de errores es asumida por el test de aceptación; la evaluación de daños no es necesaria, ya que se supone que la recuperación de errores hacia atrás limpia los estados erróneos; y el tratamiento del fallo es realizado utilizando un recambio que estaba a la espera.

El test de aceptación proporciona el mecanismo de detección de errores que hace posible explotar la redundancia en el sistema. El diseño del test de aceptación es crucial para la eficiencia del esquema de bloques de recuperación. Como ocurre dentro de todos los mecanismos de detección de errores, existe una relación entre proporcionar un test de aceptación exhaustivo y mantener al mínimo la sobrecarga que esto implica, de modo que la ejecución normal libre de errores se vea afectada lo menos posible. Hay que destacar que se utiliza el término aceptación, y no el término corrección, lo que permite que un componente pueda proporcionar un servicio degradado. Todas las técnicas de detección de errores discutidas se pueden utilizar para el test de aceptación. Sin embargo, hay que tener mucho cuidado al diseñarlo, ya que un test de aceptación defectuoso puede permitir que errores residuales permanezcan sin ser detectados.

3.6. Planificación de STR.

3.6.1. Introducción.

Un STR necesita coordinar su ejecución con el tiempo de su entorno, esto se realiza mediante el acceso a un reloj. Este acceso se realiza de dos formas distintas:

1. Accediendo directamente al marco temporal del entorno.
2. Mediante un reloj hardware externo que dé una aproximación adecuada del paso del tiempo en el entorno.

Desde la perspectiva del programador la primera opción no es adecuada, es más eficiente el acceso al reloj mediante una primitiva del lenguaje o a través del controlador de un reloj interno o externo.

El tiempo en los lenguajes de programación se describe en relación con tres elementos independientes:

1. La interfaz con el tiempo, por ejemplo, acceso a relojes para medir el tiempo, retardo de procesos a un tiempo futuro, o programación de tiempos límite de espera (timeouts).
2. La representación de los requisitos temporales, por ejemplo, especificación de tasas de ejecución o deadlines (tiempos límite).
3. La satisfacción de los requisitos temporales.

3.6.1.1. Interfaz con el tiempo.

El control del reloj en C se hace mediante una biblioteca estandar que contiene los prototipos de las funciones, macros, y tipos para manipular la hora y la fecha del sistema. Ésta define un tipo de tiempo básico *time_t* y varias rutinas para la manipulación de variables de este tipo. Contiene la estructura *tm* que guarda los componentes de la hora y la fecha, en formato separado. La estructura contiene los siguientes miembros, la semántica de los miembros y sus intervalos normales están expresados en los comentarios.

```
struct tm {  
    int tm_sec; /* los segundos después del minuto – [0,61] */  
    int tm_min; /* los minutos después de la hora – [0,59] */  
    int tm_hour; /* las horas desde la medianoche – [0,23] */  
    int tm_mday; /* el día del mes – [1,31] */  
    int tm_mon; /* los meses desde Enero – [0,11] */  
    int tm_year; /* los años desde 1900 */  
    int tm_wday; /* los días desde el Domingo – [0,6] */  
    int tm_yday; /* los días desde Enero – [0,365] */  
    int tm_isdst; /* el flag del Horario de Ahorro de Energía */  
};
```

Algunas funciones destacadas son las siguientes:

- `double difftime(time_t tiempo1, time_t tiempo0); /* resta dos valores de tiempo */`
- `time_t mktime(struct tm *tiempoPtr); /* compone un valor de tiempo */`
- `time_t time(time_t *tiempoPtr); /* devuelve el tiempo actual y si el reloj no es nulo también devuelve el tiempo en esa zona de memoria */`

El valor del reloj se define mediante la estructura *timespec*, donde *tv_sec* que se encuentra definida en *dce/utc.h*. Esta estructura representa el número de segundos transcurridos desde el 1 de enero de 1970, y *tv_nsec* la cantidad adicional de nanosegundos (aunque se muestra como un entero largo, el valor que toma *tv_nsec* debe ser siempre menor que 1.000.000.000 y no negativo).

```
struct timespec {
    time_t tv_sec; /* segundos desde el 1 de enero de 1970 */
    long tv_nsec; /* segundos adicionales desde tv_sec */
} timespec_t;
```

También cuenta con un conjunto de funciones para el manejo de la estructura de las que destacan la siguientes:

```
int clock_gettime(clockid_t reloj_id, struct timespec *tmpo);
int clock_settime(clockid_t reloj_id, const struct timespec *tmpo);
int clock_getres(clockid_t reloj_id, struct timespec *res);
int clock_getcpuclockid(pid_t pid, clockid_t *reloj_id);
int clock_getcpuclockid(pthread_t thread_id, clockid_t *reloj_id);
int nanosleep(const struct timespec *tmpodormir, struct timespec *tmporest);

/* nanosleep devuelve -1 si es interrumpido por una señal. */
/* tmporest contendrá el tiempo de sueño restante */
```

Los procesos también deben ser capaces de retrasar su ejecución por un periodo fijo de tiempo (retardo relativo) o hasta cierto instante absoluto de tiempo futuro (retardo absoluto).

Un retardo relativo posibilita que un proceso sea encolado hasta cierto evento futuro en lugar de efectuar una espera ocupada basada en llamadas al reloj. En C se pueden obtener retardos invocando la llamada del sistema *sleep* o *nanosleep* (si se desea más detalle).

Un retardo absoluto retrasa la ejecución de un proceso hasta un instante de tiempo absoluto. El programador necesita primitivas concretas o calcular el periodo a retrasar.

La restricción temporal más simple que puede tener un STR es reconocer y actuar sobre la no ocurrencia de cierto suceso externo. En general, el tiempo límite de espera (timeout) es una restricción sobre el tiempo que un proceso puede esperar para que un trozo de código se ejecute en un cierto tiempo o por una comunicación. C soporta un tiempo límite de espera explícito en las esperas en mecanismos de concurrencia como semáforos y variables de condición.

3.6.1.2. Representación de requisitos temporales.

Las aplicaciones STR deben satisfacer restricciones temporales provenientes del sistema físico. Estas restricciones pueden ir más allá de los simples tiempos límite de espera. Frecuentemente, el sistema lógicamente correcto se especifica, diseña y construye, finalmente se prueba para ver si cumple los requisitos temporales. Si no lo hace, se aplicarán varios ajustes finos y reescrituras. El resultado es un sistema que puede ser difícil de entender y caro de mantener y de actualizar. Se necesita por lo tanto un tratamiento del tiempo más sistemático.

Los métodos teóricos propuestos utilizan dos vías: (1) El uso de lenguajes de semántica formalmente definida y requisitos temporales, junto con notaciones y lógicas que posibiliten que se puedan representar y analizar las propiedades temporales; (2) Estudio de la factibilidad de planificación de la carga de trabajo exigida en función los requisitos disponibles (procesadores y otros). Dado que las técnicas formales no están todavía lo suficientemente maduras como para basar en ellas sistemas de tiempo real grandes y complejos, se mostrará en más detalle la planificación de la carga.

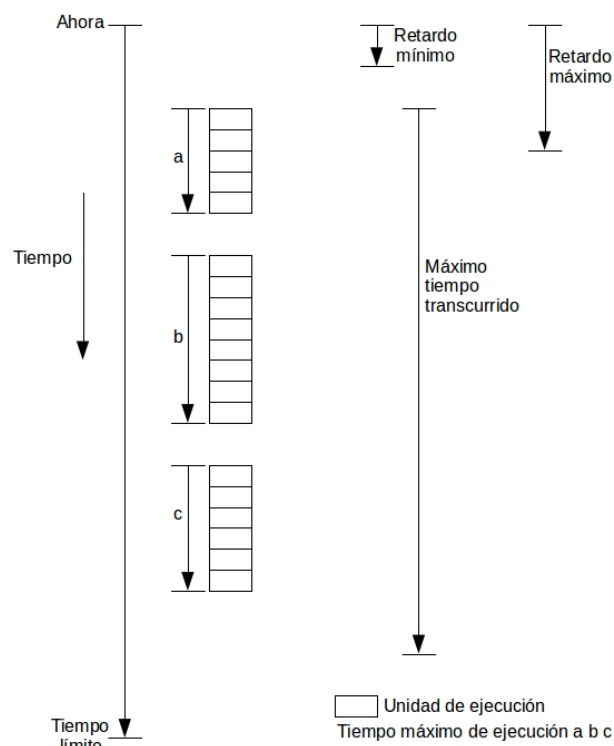


Figura 3.10: Ámbitos temporales.

Para facilitar la especificación de las diversas restricciones temporales presentes en las aplicaciones de tiempo real, resulta útil introducir la noción de ámbitos temporales. Dichos ámbitos identifican a un conjunto de sentencias asociadas con una restricción temporal. Los posibles atributos de un ámbito temporal (AT) incluyen entre otros (Figura 3.10):

- Límite temporal: el instante de tiempo en el cual la ejecución de un AT debe haber

finalizado.

- Retardo mínimo: la mínima cantidad de tiempo que debe pasar antes del comienzo de ejecución de un AT.
- Retardo máximo: la máxima cantidad de tiempo que puede pasar antes del comienzo de la ejecución de un AT.
- Tiempo máximo de ejecución de un AT.
- Combinaciones de los atributos anteriores.

Los ámbitos temporales pueden ser periódicos o esporádicos. Habitualmente, los ámbitos temporales periódicos toman datos o ejecutan un bucle de control, y tienen tiempos límite de espera que deben cumplir. Los ámbitos temporales esporádicos normalmente surgen como consecuencia de eventos asíncronos externos al computador embebido. Estos ámbitos llevan asociados tiempos de respuesta. La activación de los ámbitos temporales se suele considerar arbitraria, siguiendo, por ejemplo, una distribución de Poisson. Esta distribución permite la llegada de ráfagas de eventos externos, pero no excluye una posible concentración de actividad esporádica. No resulta posible, por lo tanto, realizar un análisis basado en el peor caso (existe una probabilidad no nula de que ocurra cualquier número de eventos no periódicos en un tiempo dado). Para permitir los cálculos del peor caso, se suele definir un periodo mínimo entre dos eventos cualesquiera no periódicos (de un mismo origen). Si éste fuera el caso, el proceso involucrado se dice que es esporádico.

En muchos lenguajes de tiempo real, los ámbitos temporales están asociados con los procesos que los implementan. Los procesos se pueden describir como periódicos o esporádicos, dependiendo de las propiedades de sus ámbitos temporales internos. La mayoría de los atributos temporales enumerados en la anterior lista pueden ser satisfechos mediante:

- Ejecución de procesos periódicos a una tasa correcta.
- Ejecución de todos los procesos dentro de sus límites temporales.

El problema de satisfacer las restricciones temporales se convierte, por tanto, en el problema de la planificación de los procesos para cumplir los tiempos límite, o planificación con tiempo límite. Hay que darse cuenta de que el requisito del retardo máximo se puede conseguir repartiendo el ámbito entre dos procesos con una relación de precedencia. El primero, que representa la fase inicial del AT puede tener un tiempo límite que asegure el cumplimiento del máximo retardo en el comienzo. Las aplicaciones que necesitan este tipo de estructura suelen ser aquellas en las que primero se lee de un sensor y luego se produce una salida. Para conseguir un mejor control sobre el momento en el que se lee el sensor, se necesita un límite temporal estricto en esta acción inicial. La salida se puede producir, entonces, con un límite temporal posterior. La variación del momento en el que se lee el sensor o en la salida de un actuador sobre una válvula se denomina fluctuación (jitter) de entrada o salida.

El lenguaje C no soporta la especificación explícita de procesos periódicos o esporádicos con límites temporales; en su lugar se debe utilizar una primitiva de retardo, un temporizador o algo similar dentro de los procesos cíclicos. C requiere el uso explícito de un temporizador y el manejo de señales.

3.6.2. Planificación.

Los procesos independientes pueden ejecutarse de forma no apropiativa de muchas formas distintas en un único procesador, en un sistema multiprocesador existen muchas más formas de entrelazarlos. Aunque las salidas del programa sean idénticas en todos estos posibles entrelazamientos, el comportamiento temporal puede variar considerablemente. Por ejemplo, si uno de los procesos tiene un tiempo límite ajustado, entonces quizás sólo se consigan los requisitos temporales en aquellos entrelazamientos en los que se ejecute el primero. Un STR necesita restringir el no determinismo que se da en los sistemas concurrentes.

Este proceso se conoce como planificación o scheduling. Un esquema de planificación proporciona dos elementos:

- Un algoritmo para ordenar el uso de los recursos del sistema (CPUs).
- Un modo de predecir el comportamiento en el peor caso del sistema cuando se aplica el algoritmo de planificación.

Las predicciones son útiles para coordinar que se pueden satisfacer los requisitos temporales del sistema. Un esquema de planificación puede ser estático (si las predicciones se realizan antes de la ejecución) o dinámico (si se toman decisiones en tiempo de ejecución).

3.6.2.1. Modelo de proceso simple.

Dado un programa complejo, no es fácil analizarlo para poder predecir el comportamiento en el peor caso. Por tanto, resulta imprescindible imponer algunas restricciones en la estructura de los programas concurrentes de tiempo real. Se utilizará un modelo simple, que permitirá describir algunos esquemas de planificación estándar, y tiene las siguientes características:

- Se supone que la aplicación está compuesta por un conjunto fijo de procesos.
- Todos los procesos son periódicos, con periodos conocidos.
- Los procesos son completamente independientes unos de otros.
- Se ignoran todas las sobrecargas del sistema, tiempos de cambio de contexto y demás, se supone coste cero.
- Todos los procesos tienen tiempos límite iguales a sus periodos, es decir, el proceso acaba antes de ser ejecutado nuevamente.
- Todos los procesos tienen tiempos de ejecución constantes en el peor caso.

Una consecuencia de la independencia entre procesos es que se puede suponer que en algún instante de tiempo todos los procesos son ejecutados a la vez. Esto representará la carga máxima para el procesador, y se conoce como un *instante crítico*.

Con un conjunto fijo de procesos periódicos resulta posible presentar un esquema de planificación completo en el que la ejecución repetida de la planificación consiga que todos los procesos se ejecuten a la tasa apropiada. El *ejecutivo cíclico* es una tabla de llamadas

Proceso	Periodo(T)	Tiempo de Ejecución (C)
a	25	10
b	25	8
c	50	5
d	50	4
e	100	2

Tabla 3.1: Ejecutivo cíclico de un conjunto de procesos.

a procedimientos, donde cada procedimiento representa parte del código de un proceso. La tabla completa se le conoce como *ciclo principal*, y habitualmente consta de cierto número de ciclos secundarios, cada uno de ellos de duración fija. De esta forma, por ejemplo, cuatro ciclos secundarios de 25 ns de duración conforman un ciclo principal de 100 ms. Durante la ejecución, una interrupción de reloj cada 25 ms permite que el planificador realice rondas entre los cuatro ciclos secundarios. La Tabla 3.1 proporciona un conjunto de procesos que deben ser implementados mediante un ciclo principal simple de cuatro ranuras. El código de tal sistema podría ser:

```

loop
    espera-interrupción;
    procedimiento-para-a;
    procedimiento-para-b;
    procedimiento-para-c;
    espera-interrupción;
    procedimiento-para-a ;
    procedimiento-para-b;
    procedimiento-para-d;
    procedimiento-para-e;
    espera-interrupción;
    procedimiento-para-u;
    procedimiento-para-b;
    procedimiento-para-c;
    espera-interrupción;
    procedimiento-para-a;
    procedimiento-para-b;
    procedimiento-para-d;
end loop;

```

Los inconvenientes son los siguientes:

- Todos los periodos de los procesos deben ser múltiplos del tiempo de ciclo.
- Difícil incorporar procesos esporádicos.
- Dificultad para incorporar procesos con periodos grandes; el tiempo del ciclo principal es el máximo periodo acomodable sin planificación secundaria.

- La dificultad a la hora de construir el ejecutivo cíclico.
- Cualquier proceso con un tiempo de cálculo notable podrá tener que ser dividido en número fijo de procedimientos de tamaño fijo, lo que podría romper la estructura del código desde una perspectiva de la ingeniería del software, y por lo tanto ser propenso a error.

Si puede construirse un ejecutivo cíclico, no será necesario ningún test de planificabilidad. Sin embargo, en sistemas de alta utilización la construcción del ejecutivo es problemática.

3.6.2.2. Planificación basada en procesos.

Con el enfoque del ejecutivo cíclico, la ejecución consiste en una secuencia de invocaciones a procedimientos. La noción de proceso (hilo) no se preserva durante la ejecución. Un enfoque alternativo es soportar la ejecución de procesos de forma directa, y determinar cuál es el proceso que deberá ejecutarse en cada instante de tiempo, mediante uno o más atributos de planificación. Con este enfoque, un proceso está limitado a estar en uno de los posibles siguientes estados (comunicación entre procesos):

- Ejecutable.
- Suspendido en espera de un evento temporizado (apropiado para procesos periódicos).
- Suspendido en espera de un evento no temporizado (apropiado para proceso esporádicos).

Existe un gran número de aproximaciones distintas a la planificación. Se presentarán tres:

1. Planificación de prioridad estática (FPS; Fixed-Priority Scheduling). Ésta es la modalidad más ampliamente utilizada. Cada proceso tiene cierta prioridad fija, estática, que se calcula antes de su ejecución. Los procesos ejecutables se ejecutan en el orden determinado por su prioridad. En los STR la prioridad de un proceso se deriva de los requisitos de temporización, no de su importancia para el correcto funcionamiento de la integridad del sistema.
2. Planificación primero el tiempo límite más temprano (EDF; Earliest Deadline First). En este caso, los procesos ejecutables se ejecutan en el orden determinado por los tiempos límite absolutos: el siguiente proceso a ejecutar es aquél con el tiempo límite más próximo (el más cercano). Aunque se suelen conocer los tiempos límite relativos de cada proceso, los tiempos límite absolutos se calculan en tiempo de ejecución, y por ello se dice que el esquema es dinámico.
3. Planificación basada en el valor (VBS; Value-Based Scheduling). Si el sistema puede llegar a sobrecargarse, no será suficiente el simple tipo de prioridades estáticas o de tiempos límite: se precisará un esquema más adaptable. Esto suele realizarse asignando cierto valor a cada proceso y empleando un algoritmo de planificación en línea basado en el valor para decidir cuál será el siguiente proceso a ejecutar.

Proceso	Periodo(T)	Prioridad (P)
a	25	5
b	60	3
c	42	4
d	105	1
e	75	2

Tabla 3.2: Ejemplo de asignación de propiedades.

En un esquema de planificación basado en prioridad, puede que un proceso de alta prioridad tenga que ser ejecutado mientras se está ejecutando otro de menor prioridad. En un esquema apropiativo se producirá inmediatamente un cambio de procesos a favor del proceso de alta prioridad. Alternativamente, en un caso no apropiativo el proceso de menor prioridad podrá acabar su ejecución antes de que se ejecute el otro proceso. En general, los esquemas apropiativos mejoran la reacción de los procesos de alta prioridad. Entre la apropiación y la no apropiación existen estrategias alternativas que permiten que un proceso de menor prioridad continúe su ejecución durante un tiempo limitado. Estos esquemas se conocen como de apropiación diferida i distribución cooperativa. Vamos a suponer que el despacho es apropiativo. Los esquemas como EDF y VBS pueden tomar apropiativas o no apropiativas.

A partir del modelo básico presentado anteriormente aparece un esquema óptimo de asignación de prioridades denominado tasa monotónica (rate monotonic). A cada proceso se asigna una única prioridad basada en su periodo: mayor prioridad para periodos menores (para dos procesos i y j: $T_i < T_j \Rightarrow P_i > P_j$). Esta asignación es óptima en el sentido de que cualquier conjunto de procesos puede ser planificado con un esquema de prioridades estático utilizando una planificación apropiativa basada en prioridades, entonces el conjunto dado también se puede planificar con un esquema de planificación de tasa monotónica. La Tabla 3.2 muestra 5 procesos y sus prioridades relativas para un comportamiento óptimo, a mayor valor más grandes prioridad.

3.6.2.3. Test de planificabilidad basados en la utilización.

Se describe un test de planificabilidad muy simple para FPS. Liu y Layland (1973) demostraron que considerando sólo la utilización del conjunto de procesos, puede obtenerse un test de planificabilidad si se emplea una ordenación de tasa monotónica. Si se cumple la siguiente condición, todos los N procesos cumplirán sus tiempos límite (el sumatorio calcula la utilización total del conjunto de procesos):

$$\sum_{i=1}^N \left(\frac{C_i}{T_i} \right) \leq N * (2^{\frac{1}{N}} - 1) \quad (3.1)$$

La Tabla 3.3 muestra los límites de utilización para valores pequeños de N. Para valores grandes el límite se aproxima asintóticamente al 69,3 %, y siempre será planificable, utilizando un esquema de planificación apropiativo basado en prioridades, con las prioridades calculadas mediante el algoritmo de tasa monotónica.

Se tomarán tres ejemplos sencillos para ilustrar el modo de utilización de este test. En estos ejemplos, no se definen las unidades de tiempo (en magnitudes absolutas), ya que

N	Límite
1	100 %
2	82,8 %
3	78 %
4	75,7 %
5	74,3 %
10	71,8 %

Tabla 3.3: Límites de utilización.

Proceso	Periodo(T)	Tiempo de ejecución (C)	Prioridad (P)	Utilización (U)
a	50	12	1	0,24
b	40	10	2	0,25
c	30	10	3	0,33

Tabla 3.4: El conjunto de procesos A.

todos los valores (T_s , C_s , etc.) se expresan en las mismas unidades, la unidad de tiempo considerada será un tick (pulso) en cada base temporal concreta.

La Tabla 3.4 contiene tres procesos a los que se les han asignado prioridades mediante el algoritmo de tasa monotónica (el proceso c tiene la prioridad más alta y el a la menor). Su utilización combinada es 0,82, el 82 %. Esto está por encima del umbral para tres procesos (0,78), por lo que este conjunto de procesos no pasa el test de utilización.

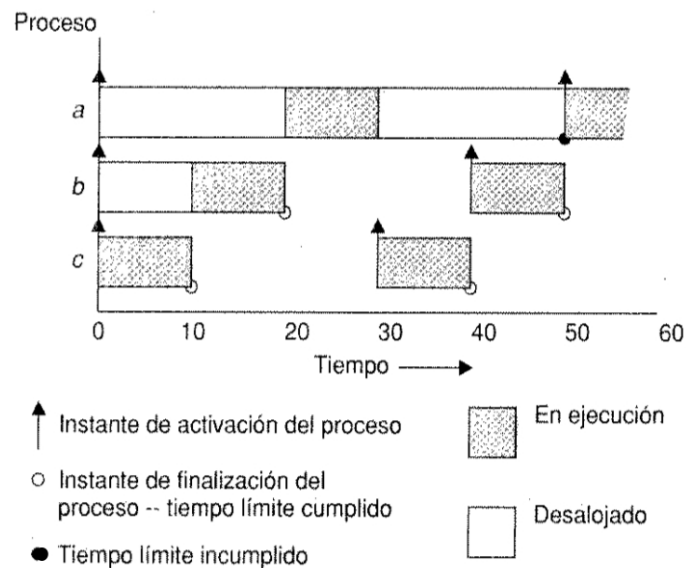


Figura 3.11: Línea de tiempo para el conjunto de procesos A.

El comportamiento real de este conjunto de procesos se muestra dibujando su línea temporal. La 3.11 muestra cómo los tres procesos se podrían ejecutar si todos ellos comenzaran sus ejecuciones en el instante 0. En el instante 50, el proceso 1 ha consumido sólo 10 pulsos de ejecución (cuando necesita 12), y por lo tanto ha incumplido su primer límite

Proceso	Periodo(T)	Tiempo de ejecución (C)	Prioridad (P)	Utilización (U)
a	80	32	1	0,400
b	40	5	2	0,125
c	16	4	3	0,250

Tabla 3.5: El conjunto de procesos B.

Proceso	Periodo(T)	Tiempo de ejecución (C)	Prioridad (P)	Utilización (U)
a	80	40	1	0,50
b	40	10	2	0,25
c	20	5	3	0,25

Tabla 3.6: El conjunto de procesos C.

temporal.

El segundo ejemplo está contenido en la Tabla 3.5. Ahora la utilización combinada es de 0,775, que está por debajo del límite, y por lo tanto se garantiza que el conjunto de procesos cumplirá todos sus tiempos límite. Si se dibujará una línea de tiempo para este conjunto, todos los límites temporales son satisfechos.

Aunque resulten engorrosas, las líneas temporales pueden utilizarse para probar la planificabilidad. Pero, ¿hasta dónde se ha de continuar dibujando la temporal antes de que se pueda concluir que el futuro no guarda sorpresas?. Para conjuntos de procesos que comparten un tiempo de activación común (esto es, un instante crítico), se puede demostrar que una línea temporal igual al tamaño del periodo más largo es suficiente. De forma que, si todos los procesos cumplen su primer límite temporal, entonces cumplirán todos sus límites futuros.

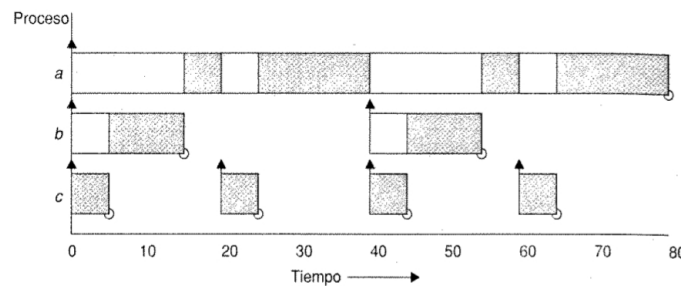


Figura 3.12: Línea de tiempo para el conjunto de procesos C.

La Tabla 3.6 muestra un último ejemplo. Éste es un sistema de tres procesos, pero la utilización combinada es del 100 %, luego no aprueban el test. Sin embargo, en tiempo de ejecución el comportamiento parece correcto: todos los límites temporales se cumplen hasta el instante 80 (Figura 3.12). Aunque el conjunto de procesos no pasa el test, en tiempo de ejecución no se incumple ningún tiempo límite. Por lo tanto, se dice que el test es suficiente pero no necesario. Si un conjunto de procesos pasa el test, entonces cumplirá todos los tiempos límite; si lo falla, puede o no fallar en tiempo de ejecución. Un último punto digno de atención es que este test basado en la utilización sólo aporta un respuesta del tipo sí/no. No da ninguna indicación sobre los tiempos de respuesta de los procesos.

4

Redes de Comunicaciones

Esta sección mostrará los principios más importantes de las redes de computadores. Se detallará el modelo OSI de ISO, y se mostrará el funcionamiento de las redes locales. También se expondrá los niveles de enlace y de red de protocolo TCT/IP.

4.1. Introducción.

La implantación en todos los sectores de la sociedad de la tecnologías de la información, y en nuestro caso dentro del ámbito industrial, ha provocado la necesidad de comunicación entre los diferentes equipos repartidos en lugares geográficos diferentes. De esta función se encargan los sistemas de comunicación de datos, que deben posibilitar el intercambio de datos y los medios para compartir recursos.

El intercambio de datos abarca la transferencia de ficheros entre los diferentes equipos, consulta en bases de datos, gestión del correo electrónico, y en general cualquier acción que requiera paso de información entre diferentes equipos utilizando un medio de comunicación (cable, aire, etc). También debe dotar de los medios necesarios para compartir recursos, en ocasiones únicos y caros, y que además ocupan espacio. Estos recursos serán periféricos o cualquier otro dispositivo. Dentro de nuestro ámbito se define comunicación de datos como el proceso de intercambio de información entre diferentes equipos interviniendo dos elementos físicos:

1. Emisor/Receptor: equipos que intervienen en la comunicación (PCs, periféricos, PLCs. .).
2. Canal: medio físico que transmite la señal, por ejemplo: cable eléctrico, aire, fibra óptica, buses de campo, etc.

El proceso de comunicación consiste en que los emisores/receptores intercambian información a través de un canal de comunicación. La información se transmite en forma de mensaje, y para el proceso de comunicación se utilizan las redes de comunicación de datos.

Las redes de comunicación de datos agrupan dispositivos de comunicación de datos empleando un canal común, que a su vez pueden formar parte de estructuras de mayor extensión, mediante la interconexión de éstas, que se denominarán subredes. Las diferentes formas de conexión y organización del canal se conocen como topologías. Las redes se clasifican en base a su extensión:

Redes de área local (Local Area Network - LAN). Los equipos están distribuidos en una distancia que oscila entre 10m y 1 km, y con velocidades de comunicación entre 10 Mbps y 1 Gbps. Un ejemplo son las redes Ethernet, empleadas usualmente en empresas, Universidades e industrias a nivel de gestión.

Redes de área extensa (Wide Area Network - WAN). Conectan equipos o redes de distintos lugares, y pueden cubrir áreas geográficas muy extensas. Normalmente se emplean estas redes en grandes corporaciones públicas o privadas.

Dependiendo de las técnicas de transferencia de datos que se empleen entre los componentes de una red podemos distinguir entre:

- Redes de conmutación de circuitos. Son redes en las que se establece un circuito de comunicación que se mantiene durante toda la comunicación, y al final se libera el circuito. Por ejemplo, las redes de telefonía: se establece un camino de conexión físico desde el teléfono hasta el receptor de la llamada utilizando unas centrales intermedias.
- Redes de conmutación de paquetes. En este caso se dispone de una red de transmisión de datos que interconectan diferentes nodos a los que se conectan los equipos o subredes que originan y reciben las transmisiones. Los mensajes enviados se descomponen en paquetes de menor tamaño, a los que se les añade información para identificar el equipo emisor y receptor. Los paquetes son enviados por separado pudiendo circular por diferentes caminos en la red de nodos intermedios. Existen dos tipos de conmutación de paquetes:
 1. Conmutación de paquetes por circuito virtual: Se realiza una reserva previa de recursos, estableciendo en cada nodo el nodo siguiente al que se enviarán los datos.
 2. Conmutación de paquetes por datagrama: Los paquetes, denominados datagramas, se envían sin establecer reserva previa, de modo que cada nodo debe determinar a qué nodo debe enviar cada datagrama cuando lo reciba.

4.2. Modelo de Referencia OSI de ISO.

Es necesario la utilización de un protocolo que permita entenderse a los dos interlocutores. Un protocolo es un conjunto de reglas que regulan el flujo de la información, quién y cómo comienza la transmisión, quién puede transmitir en cada momento y cómo termina la comunicación. También se deben establecer mecanismos de control de detección de errores y recuperación de datos en caso de error en la transmisión que están recogidos en el protocolo.

Los primeros sistemas de comunicación eran propietarios con protocolos diseñados por las propias compañías. Posteriormente se hizo necesaria una normalización de protocolos

de comunicación de datos a la que se deben ajustar los diferentes fabricantes y diseñar sistemas abiertos debido a la necesidad común del intercambio de datos entre ellos. Por ello, la International Standardization Organization (ISO) en 1977 constituyó un comité para la creación de una arquitectura de comunicaciones que pudiera ser adoptada por los diferentes fabricantes de ordenadores y que permitiera interconectar equipos de diferente naturaleza. De este modo surgió el modelo de referencia OSI (Open Systems Interconnection: interconexión de sistemas abiertos), recogido en la norma ISO7498.

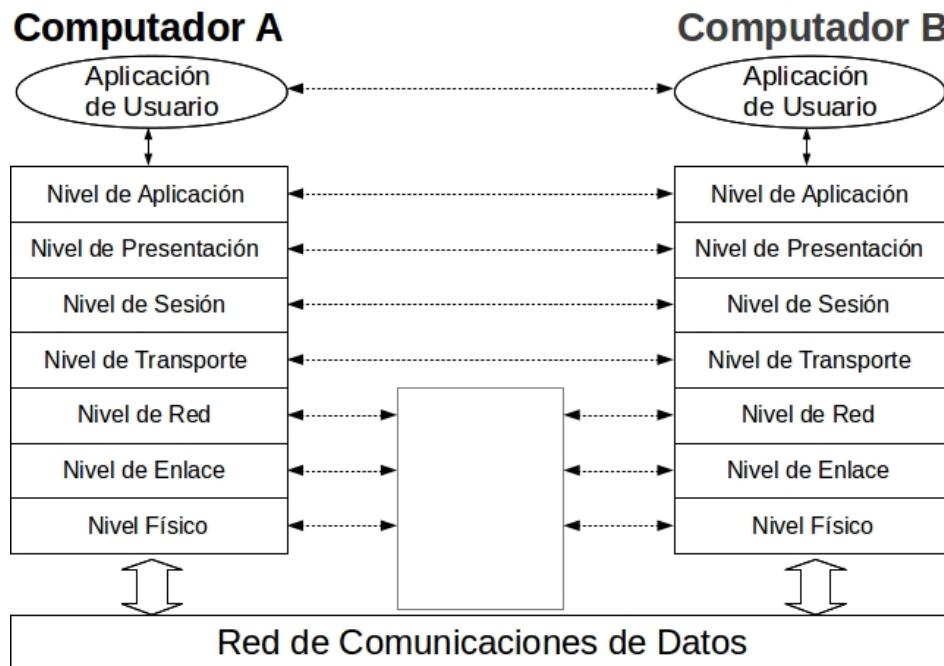


Figura 4.1: Modelo de Referencia OSI de ISO.

El objetivo del Modelo OSI es permitir la comunicación entre aplicaciones que se ejecutan en diferentes equipos, eliminando las diferencias entre estos componentes físicos que deben atravesar los mensajes. La comunicación vista del usuario consiste en una comunicación entre las aplicaciones. Para que esta pueda llevarse a cabo, la información sufre varias transformaciones que permiten su transmisión por el medio físico. Para realizar estas transformaciones el modelo de referencia OSI propone una arquitectura de capas (Figura 4.1) que define el comportamiento del subsistema de comunicación de los equipos. Este modelo regula el comportamiento externo de cada capa, dando libertad al fabricante para su implementación interna.

Cada capa del modelo desempeña una función bien definida según un protocolo definido e intercambia datos virtualmente con su capa equivalente del sistema remoto (comunicación entre pares):

- Cada capa ofrece servicios sólo a la del nivel inmediatamente superior.
- Cada capa solicita servicios a la capa inmediatamente inferior.
- La capa no tiene comunicación con nadie más.

Este mecanismo permite una implementación de cada capa encapsulada y que no es visible al exterior. Así cada fabricante puede realizar su implementación de las capas sin afectar a su funcionamiento externo, que será compatible con otras implementaciones. Los mensajes intercambiados entre las capas se denominan Unidad de Datos de Protocolo (UDP). Si la aplicación del equipo A (Figura 4.1) quiere enviar un UDP a la aplicación de equipo B ocurren los siguientes pasos:

1. Cada capa toma los datos de la superior y le añade una cabecera con información necesaria para el protocolo de la misma capa en el equipo destino.
2. La capa solicita un servicio a su capa inferior para mandar su UDP.
3. La capa inferior repite el proceso hasta llegar a la capa física, que transmite los datos por el medio físico.
4. Cuando llega el UDP al equipo B cada capa toma los datos de la capa inferior y procesa la cabecera de su nivel que le da información de qué debe hacer.
5. Cada capa manda a la capa superior su UDP para que lo procese.
6. Este proceso se repite hasta que los datos llegan a la aplicación de usuario.

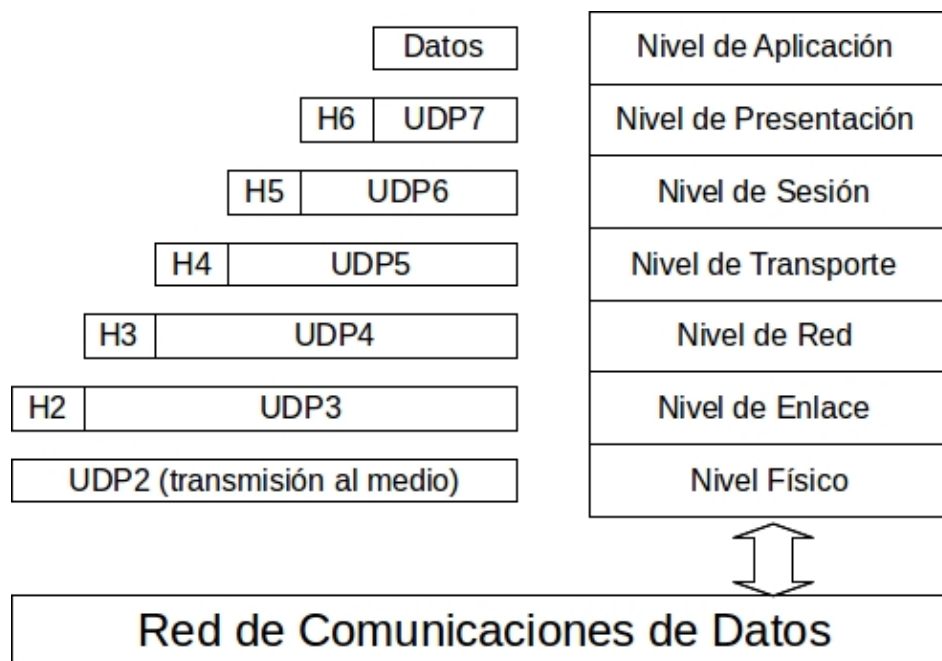


Figura 4.2: Cabeceras en Modelo OSI.

Cuando se envía el UDP cada capa añade su cabecera, y a la recepción cada la procesa y la elimina. Las cabeceras contienen la información necesaria para el funcionamiento del protocolo de ese nivel. La Figura 4.2 muestra el esquema general de UDPs implicadas en la transmisión de datos mediante su procesamiento por las diferentes capas del modelo OSI.

A continuación se detalla brevemente la función de las capas del Modelo OSI:

Capa Física: contiene las especificaciones eléctricas y mecánicas para garantizar la transmisión del flujo de bits serie entre equipos: voltajes, periodo del bit, forma de codificación, sincronización a nivel de bit, tipo de medio físico, tipos de conectores, número de hilos del cable, etc. Esta capa no garantiza una comunicación fiable, es decir, no se realiza ningún tipo de comprobación de errores para asegurar que los datos se reciben correctamente en el destinatario. La UDP de este nivel está constituida por los datos de nivel 2 (enlace), ya que el nivel físico se encarga únicamente de la transformación de los datos en señales transmisibles por el medio o viceversa. Ejemplos son las normas RS232, RS422 y RS485 empleadas en la industria.

Capa de Enlace: Garantiza la fiabilidad de la transmisión de datos entre dos equipos de un enlace de datos unidos por el mismo medio físico en la red. Realiza funciones de detección y corrección de errores (recuperación o reenvío de los datos erróneos) así como de regulación y control de flujo de datos (sincronización entre emisor y receptor). La UDP de este nivel incluye cabeceras (o en algunos casos colas) para la detección de errores, como códigos de redundancia cíclica (CRC), paridad o suma de comprobación. Destacar que este nivel garantiza una comunicación sin errores entre equipos del enlace de datos.

Capa de Red: Responsable del encaminamiento de paquetes a través de la red de nodos intermedios o encaminadores a través de los cuales se comunican diferentes equipos. Se realizan funciones como cálculo de las rutas óptimas. Esta capa no proporciona fiabilidad en la transmisión. Por una parte, no garantiza la llegada de todos los paquetes en los que se ha fraccionado un mensaje, y por otra, tampoco garantiza que éstos lleguen en el mismo orden en que fueron enviados (ya que al llegar por diferentes rutas en la red de conmutación de paquetes el tiempo de llegada de éstos puede variar). La UDP de red incluye información sobre los equipos origen y destino de los paquetes.

Capa de Transporte: Proporciona una comunicación fiable extremo a extremo (host to host), dotando a los extremos de una sesión transparente y libre de errores; garantiza la fiabilidad en el transporte del mensaje. Se encarga de la búsqueda de rutas alternativas en caso de error y de los mecanismos para la reconstrucción de mensajes a partir de paquetes. Un ejemplo de protocolo de nivel de transporte es el conocido TCP (Transport Control Protocol) perteneciente a la familia de protocolos TCP/IP.

Capa de Sesión: Establece los mecanismos necesarios para el control y sincronización de conversaciones entre aplicaciones de distintos sistemas: (1) Cómo se inicia la conversación. (2) Cómo se desarrolla la conexión: Maestro/esclavo, Peer to peer. (3) Cómo finaliza la conexión. Se pueden establecer varias sesiones o conexiones sobre una misma capa de transporte, tal como se muestra en la Figura 4.3, donde varias aplicaciones pueden establecer diferentes sesiones de comunicación.

Capa de Presentación: Se ocupa de la representación (sintaxis) de los datos durante la transferencia entre aplicaciones. Los datos se transforman a formatos de transporte común (sintaxis de transferencia): Conversiones entre diferentes juegos de caracteres, compresión de datos y encriptación. Utilizando protocolos de este nivel (incluyendo los anteriores) se pueden conectar equipos muy diferentes de manera fiable y garantizando el entendimiento entre ellos.

Capa de Aplicación: Proporciona servicios y funciones para que los procesos de aplicación tengan acceso al entorno OSI y se puedan comunicar con otras aplicaciones. Suelen presentarse en forma de API (Application Programming Interface), una serie de librerías que suelen formar parte del sistema operativo. Además de la transferencia de

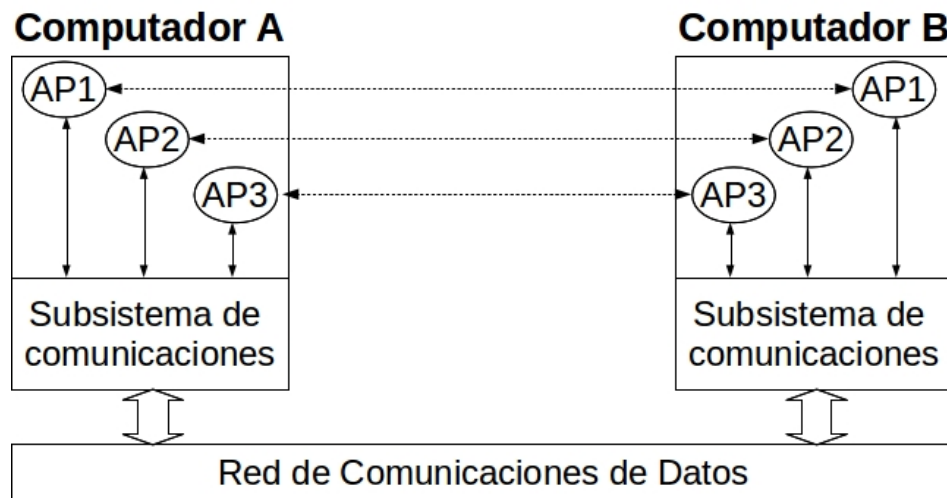


Figura 4.3: Varias sesiones de comunicación.

información proporcionan servicios como correo electrónico, telnet, transferencia de ficheros (ftp), etc. La UDP de la capa de aplicación está constituida por los datos que la aplicación requiera enviar o recibir en su caso.

No todos los protocolos de comunicación implementan todos los niveles. Son necesarias las capas física y de aplicación para permitir la comunicación. Dentro de los buses industriales de bajo nivel es típico encontrar tan sólo las capas física, enlace y aplicación. Las redes más complejas sí utilizan las siete capas del modelo.

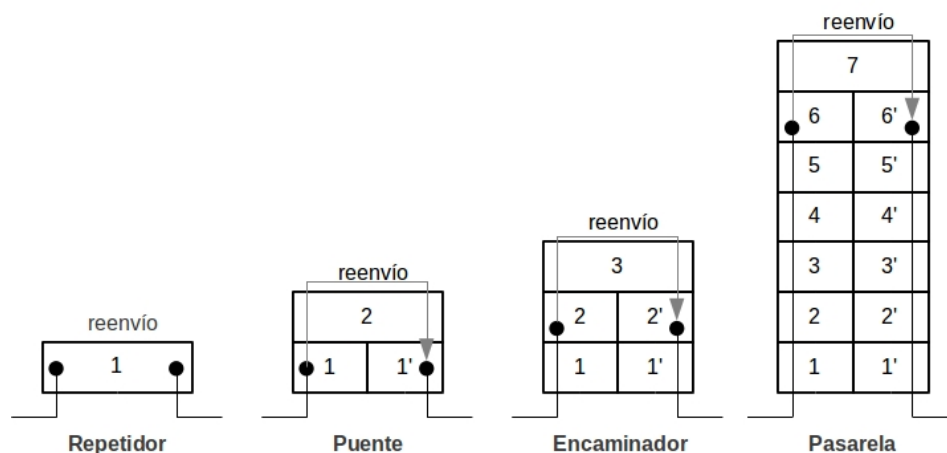


Figura 4.4: Interconexión de redes y capas OSI.

Para finalizar esta sección se va a mostrar la clasificación de los equipos de interconexión en base a la capa que tienen implementada:

- Repetidores: trabajan a nivel físico amplificando señales eléctricas. Se utilizan para prolongar la longitud en el cableado de las redes de comunicación.
- Puentes (bridges): operan a nivel de enlace permitiendo control de errores y filtrado de direcciones físicas, aislando el tráfico entre las subredes que comunican.
- Encaminadores (routers): operan a nivel de red y pueden conectar redes con diferentes niveles físico y de enlace. Realizan funciones como cálculo de rutas óptimas.
- Pasarelas (gateways): se emplean para interconectar redes con arquitecturas totalmente diferentes, como por ejemplo buses de campo con redes de área local de tipo Ethernet. Las pasarelas duplican los siete niveles OSI, ya que éstos son diferentes para cada red. Operan a nivel de aplicación.

4.3. Redes de Área Local.

4.3.1. Arquitectura de Red.

La Arquitectura de Red es el conjunto de técnicas empleadas para su diseño y que además controlan su funcionamiento. Los aspectos más importantes en la arquitectura son:

1. **Topología:** Es la distribución de los equipos conectados a la red. Se distinguen:
 - Topología Física (aparente). Es la distribución física de los equipos y el cableado entre ellos (la que vemos).
 - Topología Lógica (real). Define el camino seguido por los datos en la red, y que no siempre coincide con la topología física.

En la mayoría de los casos las topologías lógica y física coinciden. La topología también condicionará la idoneidad del método de acceso al medio utilizado. Las principales topologías son: estrella, anillo, bus y árbol.

2. **Sistema de modulación/codificación:** Es el tipo de transformación que se realiza sobre los datos para obtener una señal capaz de ser transmitida por un medio físico. Los sistemas más empleados son: banda ancha o banda base, y dentro de éstos, los diferentes sistemas de modulación (FSK, ASK, PSK) y de codificación (Manchester, NRZ, etc.).
3. **Medios de transmisión:** Son los diferentes soportes físicos para la transmisión de la señal, como cables eléctricos, fibra óptica u ondas de radio.
4. **Métodos de acceso al medio:** Dado de que los canales de transmisión de datos en redes de ordenadores son compartidos (un mensaje transmitido por un equipo es escuchado por todos los demás), debe disponerse de un protocolo de acceso que regule quién puede transmitir en cada momento.

Este problema no se presenta en sistemas de transmisión elementales, como los enlaces punto a punto, por ejemplo RS-232, donde al existir tan sólo dos interlocutores conectados mediante un canal full-duplex, cualquiera de ellos puede transmitir en todo momento. Los métodos de acceso al medio definen cómo se inicia, se envía y finaliza la propagación de un mensaje por la red sin interferir o afectar a los demás sistemas. Los principales métodos empleados para tal fin son:

- **Maestro esclavo.** Un dispositivo (maestro) indica cuando puede transmitir cada uno de los restantes (esclavos) mediante consulta secuencial de los mismos. Dada su sencillez y fiabilidad es muy utilizado en buses de campo industriales (Modbus, Profibus), y en la comunicación entre autómatas y con dispositivos de entrada-salida remotos.
 - **Métodos de contienda.** Los dispositivos de la red compiten por acceder al medio de transmisión común. Sólo uno conseguirá iniciar la comunicación. Es el utilizado en redes Ethernet 802.3.
 - **Paso de testigo (Token-Passing).** En este método se va pasando un paquete de datos especial denominado *testigo* entre los diferentes equipos. Sólo podrá transmitir el que esté en posesión de dicho testigo. Existen diferentes variantes, con y sin prioridad, y se utilizan con diferentes topología (Token-ring, Token-bus).
5. **Protocolos empleados y formato de las tramas.** Tipo de protocolos empleados para la comunicación, niveles OSI implementados y campos que componen las tramas de dato transmitidas por la red.

4.3.2. Tecnología Ethernet.

4.3.2.1. Introducción.

Se utiliza para denominar a una familia de tecnologías de red de área local muy utilizadas en la actualidad. Su origen se remonta a principios de los años 70, en el centro de investigación de Palo Alto (PARC) de Xerox Corporation, que inició estudios para lograr un medio de transmisión eficiente de paquetes en un canal compartido (cable coaxial) para conectar un número variable de estaciones que transmiten en modo half-duplex (por ser cable coaxial en banda base).

Una parte fundamental de la investigación fue el método para organizar el acceso a este medio compartido. El protocolo usado fue CSMA/CD (Carrier Sense Multiple Access / Collision Detection). Las empresas del sector se interesaron por éste, lo que dio lugar al nacimiento de Ethernet v1.0 a 10Mbps en 1980 por el consorcio Digital Equipment Corporation (DEC), Intel Corp. y Xerox Corp. El estándar publicado por el consorcio (DIX) evoluciona a Ethernet II, que a su vez, fue la base para la norma IEEE802.3 que se creó en el año 1985. Ambas normas presentan pequeñas diferencias en el formato de trama, pero existen mecanismos que posibilitan su intercomunicación, y de hecho se utilizan conjuntamente sobre las mismas redes. A partir de 1985 se producen mejoras en las redes Ethernet, como: Aumento de la velocidad (100/1.000/10.000 Mbps), Nuevos tipos de cableado (par trenzado y fibra óptica), Ethernet full-duplex, etc.

Aunque existen otras tecnologías LAN, Ethernet acapara el 90 % del mercado en este sector. Además, con la introducción de las redes Ethernet de alta velocidad y largas distancias con fibra óptica, se está introduciendo en redes MAN y WAN. Sus ventajas son:

- Fácil instalación y mantenimiento, unidos a su bajo coste.
- Flexibilidad para la interconexión de diferentes topologías.
- Estándar estable que permite interconectar dispositivos de diferentes fabricantes.

Tipo	Velocidad	Banda	Longitud	Codificación	Medio	Topología
1 Base 5	1 Mbps	base	250 m	Manchester	Par trenzado	Árbol
10 Base 2	10 Mbps	base	185 m	Manchester	Coax. fino (50Ω)	Bus
10 Base 5	10 Mbps	base	500 m	Manchester	Coax. grueso (50Ω)	Bus
10 Base T	10 Mbps	base	100 m	Manchester	Par trenzado	Árbol
10 Base F	10 Mbps	base	2100 m	Manchester	Fibra óptica	Árbol
10 Broad 36	10 Mbps	ancha	1800 m	FSK	Coax. grueso (75Ω)	Bus
100 Base TX	100 Mbps	base	100 m	4B5B (MLT3)	Par trenzado	Árbol
100 Base FX	100 Mbps	base	2100 m	4B5B (NRZI)	Fibra óptica	Árbol
1000 Base T	1 Gbps	base	100 m	PAM-5x5	Par trenzado	Árbol

Tabla 4.1: Tipos de Redes Ethernet.

Existen diferentes implementaciones de la red Ethernet, según las diferentes variantes en cuanto al medio físico se refiere (Tabla 4.1). La nomenclatura empleada consiste en XBY , donde:

- X es la capacidad del medio de transmisión en Mbps.
- B es la *Base* o *Broad* según se use banda base o banda ancha.
- Y es la longitud máxima del canal en cientos de metros o tipo de medio físico.

La Tabla 4.1 muestra los tipos de redes Ethernet, los más usados son el 10 Base 2, 10 Base 5 y 10/1000 Base T.

Para la conexión a la red se emplean tarjetas de interfaz de red (NIC: Network Interface Card). Estas tarjetas tienen una dirección única compuesta por 6 grupos de 8 bits (6 bytes) cada uno que se representan en hexadecimal por comodidad, denominada dirección MAC, que no se puede modificar por el usuario y que tiene la siguiente estructura:

XX:XX:XX:XX:XX:XX, por ejemplo: 00:23:8b:a8:93:59

Esta dirección es necesaria para direccionar los equipos a nivel físico conectados a la red. Se introducirá en la trama de datos para el envío de tramas entre estaciones.

4.3.2.2. Estándares de redes de área local.

Los protocolos más empleados con redes Ethernet de área local con IPX/SPX (de Novell), y TCP/IP (se trata de protocolos de nivel de red/transporte). Ethernet sólo implementa los niveles más bajos del Modelo de Referencia OSI (físico y de enlace):

- Físico: concierne a la transmisión de bits entre equipos conectados directamente a través de un medio físico. Especifica tipo de cable, conectores, interfaces, mecanismos de recepción-transmisión de la señal eléctrica, etc.
- Enlace: se subdivide en LLC (IEEE 802.2) y MAC.

En estos estándares, los protocolos del nivel de enlace se dividen en dos subcapas o subniveles (Figura 4.5):

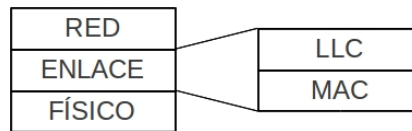


Figura 4.5: Las subcapas del nivel de enlace LLC y MAC.

- MAC (Media Access Control o Control de Acceso al Medio). Son los protocolos definidos en los drivers de la tarjeta de red (NIC: Network Interface Card).
- LLC (Logic Link Control o Control de Enlace Lógico). Define cómo funciona el nivel de enlace.

Los estándares de redes de área local están definidos por normas de la IEEE y también por normas de la ISO, equivalentes a las anteriores. El nivel LLC (802.2) es el mismo para cualquier tipo de adaptador de red, y es independiente del dispositivo y del medio físico. Este nivel se comunica con los drivers del mismo, definidos los niveles MAC y Físico:

- IEEE 802.2: LLC (Logic Link Control = control de enlace lógico).
- IEEE 802.3: CSMA/CD: método de acceso al medio por contienda.
- IEEE 802.4: Token-Bus: Paso de testigo en bus.
- IEEE 802.5: Token-Ring: Paso de testigo en anillo.
- IEEE 802.11: Transmisión por radio-frecuencia (redes WLAN: 802.11, 802.11b, 802.11g, 802.11a).
- ANSI FDI: Interfaz de datos distribuido para fibra óptica.

4.3.2.3. Métodos de Acceso al Medio.

Esta capa está muy vinculada al medio físico, y es donde se define el método de acceso al medio que se utiliza en la red. Existen varios métodos de acceso al medio dependiendo del tipo de red, los más destacables son:

- CSMA/CD (Collision Detection: detección de colisiones): acceso por contienda.
- Token-bus: acceso por paso de testigo.

CSMA/CD (IEEE 802.3)

Fue el primero en redes Ethernet y desarrollado por Rank Xerox y es el más utilizado. El funcionamiento de este método consiste en lo siguiente:

1. Los equipos escuchan la red antes de transmitir, y sólo iniciarán la transmisión si el medio está libre.
2. Cuando el equipo transmite, escucha a la vez el mensaje que circula por la red, y si éste no coincide con el que está transmitiendo indica que se ha producido una colisión (hay otro equipo transmitiendo), entonces:

- a) Deja de transmitir para volver a intentarlo tras un tiempo de espera aleatorio.
- b) Se introduce ruido en la red (señal de jamming) para asegurar que el otro equipo en contienda también detecta la colisión.

Se denomina *Dominio de colisión* al conjunto de dispositivos conectados físicamente entre sí, es decir, el área en la que se propaga una colisión.

CSMA/CD es un método válido como solución económica en redes con poco tráfico (pocas colisiones), siempre que los retardos ocasionados por las colisiones sean tolerables desde el punto de vista de los requerimientos del sistema de comunicación.

Es un método inviable en STR por la no determinación que introduce en los tiempos de transmisión. El rendimiento de la red depende del número de tramas circulando en cada momento, y no del número de equipos conectados.

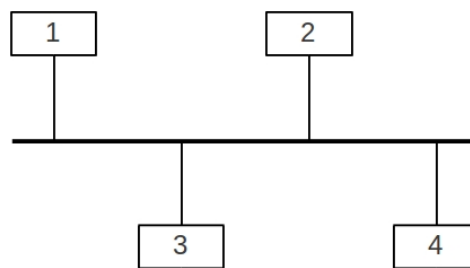


Figura 4.6: Ejemplo de numeración token-ring.

Token-bus (IEEE 802.4)

En paso de testigo (token) en bus se asigna un número de orden cada equipo (Figura 4.6). Existe un “testigo” que permite transmitir, de tal forma que sólo puede transmitir el equipo que tiene el testigo. El testigo circula ordenadamente por los nodos, y cuando llega al último vuelve a pasar al primero. El tiempo que el testigo tarda en llegar a una estación nunca excede de un valor límite, que es proporcional al número de nodos. Por lo tanto, este método de acceso al medio es apto para STR ya que es un protocolo determinista.

Es más complejo que CSMA/CD debido a que se deben prever situaciones como la pérdida del testigo. La topología aparente es en bus (cableado de la red), pero la topología lógica o real es en anillo.

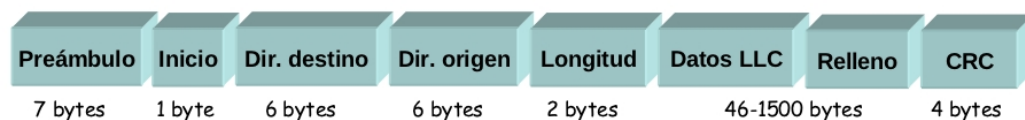


Figura 4.7: Formato de la trama MAC.

Formato de trama MAC

La trama en el nivel MAC consiste en 8 campos (Figura 4.7). En redes Ethernet existen dos tipos de tramas, que pueden coexistir en el nivel MAC:

1. Ethernet IEEE 802.3, utilizada en IPX/SPX.
2. Ethernet II, utilizada en TCP/IP.

Los campos ordenados son:

Preámbulo: Patrón fijo de 7 bytes (AAH) para sincronizar emisor y receptor (transmisión síncrona).

Inicio: Comienzo de la trama, 1010 1011.

Dirección de destino: Dirección física de la tarjeta NIC a la que va dirigida la trama. Existen direcciones de difusión que hacen referencia a todos los host de la misma red.

Dirección de origen: Dirección física de la tarjeta NIC que envía la trama.

Longitud: Indica el número de bytes en el campo de datos (1500 como máximo), o bien el tipo de protocolo que tratará la trama:

- En IEEE 802.3 indica el número de bytes en el campo de datos.
- Si el campo longitud contiene un valor mayor que 1500, esto indica que la trama es de tipo Ethernet II, y este campo indica qué protocolo de nivel superior será el encargado de tratar el mensaje (en este caso no hay campo de longitud, y ésta se determina por conteo en el receptor).

Por lo tanto, este campo permite distinguir unas tramas de otras, de modo que los protocolos Ethernet II e IEEE 802.3 pueden coexistir en la misma red.

Datos LLC: Contiene los datos suministrados por el nivel superior LLC.

Relleno: Si el campo de datos LLC no contiene suficientes bytes para completar la longitud mínima de trama, se aumenta su longitud con caracteres de relleno. (en CSMA/CD la longitud mínima para que funcione el algoritmo de detección de colisiones es de 64 bytes).

CRC: Código de redundancia cíclica de 32 bits, calculado con los campos: dirección de origen, dirección de destino, longitud, datos y relleno.

En Ethernet el nivel de enlace ofrece servicios sin conexión y sin confirmación. Por ello, todas las tramas deben especificar la dirección de origen y de destino.



Figura 4.8: Formato de la trama LLC.

4.3.2.4. Control de Enlace Lógico (LLC).

Ofrece a los protocolos del nivel de red servicios de la capa de enlace, es decir, servicios fiables de transmisión sin encaminamiento (entre dispositivos en la misma red física). La capa LLC es independiente del medio físico y de la subcapa MAC. Los servicios pueden ser orientados a conexión (CO) o sin conexión (CL). La trama en el nivel LLC tiene los siguientes campos ordenados secuencialmente (Figura 4.8):

Inicio: Consiste en un 0 seguido de seis unos, y finalmente un 0 (01111110). En el resto de la trama el protocolo añade un 0 siempre que haya cinco 1's seguidos. A la recepción se realiza la operación inversa, suprimiendo dichos ceros.

Dirección: Dirección del destinatario si se trata de una orden, y dirección del remitente si es una respuesta.

Control: Tipo de trama, de información, de supervisión o no numerada.

Datos: Los datos, encapsula la UDP recibida del nivel de red.

CRC: Código de redundancia cíclica para la comprobación de la trama.

4.4. Protocolos de nivel de Red/Transporte. El protocolo TCP/IP.**4.4.1. Introducción.**

La familia de protocolos TCP/IP (Transmission Control Protocol / Internet Protocol; Protocolo de Control de Transmisión / Protocolo de Internet) es la más utilizada debido a su empleo en Internet. Estos protocolos también se usan en muchos niveles (LAN, WAN) debido a la fiabilidad que ofrece, puesto que se trata de un protocolo establecido y sobradamente verificado por su uso.

Tiene su origen en la red de computadoras ARPANET patrocinada por el Departamento de Defensa de los Estados Unidos). El objetivo perseguido consistía en encontrar protocolos no propietarios para poder interconectar sistemas heterogéneos. Inicialmente tuvo tan sólo cuatro nodos, y con el tiempo terminó conectando cientos de Universidades e instalaciones del Gobierno.

Se conoce como el modelo de referencia TCP/IP que se definió por primera vez en 1974. A finales de los años 70 se hicieron mucho más versátiles, eficaces y fiables. En un principio, TCP/IP estuvo muy ligado al sistema operativo UNIX, ya que éste era el predominante en máquinas multipuesto y multiusuario con capacidad de conectividad a niveles de red e inter-red. En 1990 se disolvió la red Arpanet y se fundó la conocida Internet. La existencia de esta inter-red no habría sido posible sin un protocolo como TCP/IP. La principal ventaja de TCP/IP es que permite interconectar de modo fiable equipos totalmente heterogéneos y diferentes redes con diferentes topologías y protocolos en los niveles inferiores de enlace y físico.

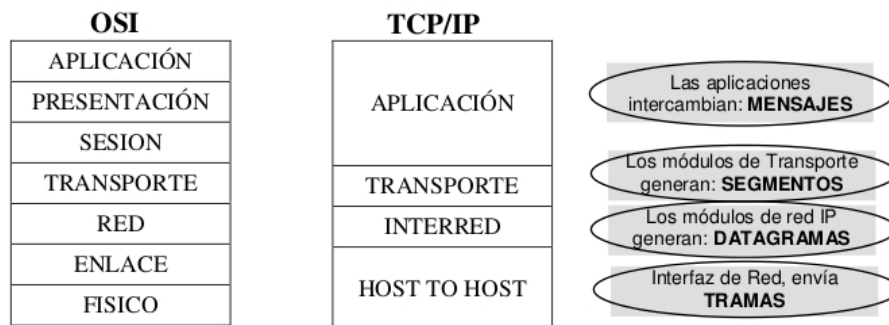


Figura 1. Relación del protocolo TCP/IP con niveles OSI.

Figura 4.9: Relación niveles OSI y TCP/IP.

4.4.2. Arquitectura de los Protocolos TCP/IP.

El protocolo TCP/IP es anterior al Modelo de Referencia OSI, por lo que no sigue exactamente la misma distribución en capas, se puede establecer una equivalencia (Figura 4.9). Como puede verse, TCP/IP utiliza una sola capa que es equivalente a las de enlace y física del Modelo OSI. Existen tanto las capas de red y transporte en ambas propuestas. En TCP/IP existe una capa aplicación que engloba los niveles de sesión, presentación y aplicación del modelo OSI.

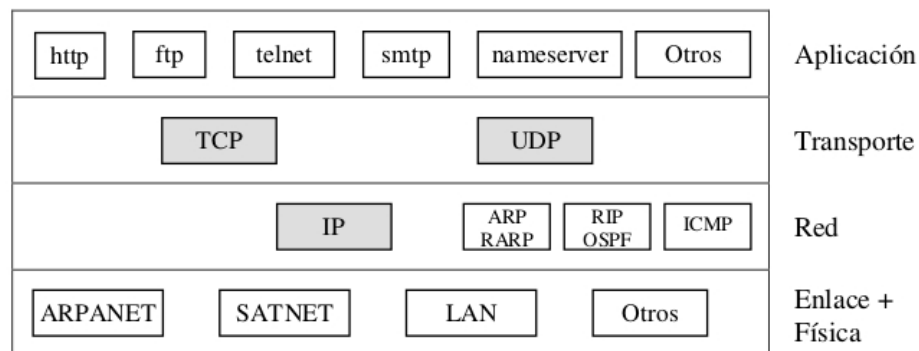


Figura 2. Protocolos en TCP/IP.

Figura 4.10: Protocolos en TCP/IP.

La Figura 4.10 muestra los protocolos que contiene TCP/IP. Más oscuros se detallan los protocolos que serán estudiados. Más concretamente, serán los protocolos IP de nivel de red, TCP y UDP de nivel de transporte. Se detallarán brevemente los diferentes protocolos TCP/IP según los niveles descritos:

- *Interfaz de red*: Equivalente al nivel físico y de enlace de datos del Modelo OSI. Describe el medio físico y de enlace. Destacable que TCP/IP puede funcionar sobre diferentes

implementaciones de estos niveles. Se suele usar en LANs de tecnología Ethernet, con protocolos Ethernet II o IEEE 802.2 (subcapa LLC de la capa de enlace) sobre IEEE 802.3 (subcapa MAC en redes Ethernet con (CSMA/CD). Se encarga de encapsular los datos proporcionados por la capa de red dentro del campo de datos de las tramas Ethernet.

- **Nivel de Red:** Existe varios protocolos de nivel de red en TCP/IP siendo el más destacado el protocolo IP. Es el encargado de hacer circular los paquetes generados en el nivel de transporte añadiendo las direcciones IP de origen y destino, permitiendo así el encaminamiento de los mismos desde el equipo origen al equipo destino. Se utiliza en los encaminadores o routers para la gestión del tráfico entre redes. Acepta los datos de la capa superior de transporte (TCP/UDP) y los transforma en datagramas. Su funcionamiento es en modo no fiable, no se garantiza la entrega de los datagramas ni su llegada en el orden de envío.
- **Nivel de Transporte (TCP y UDP):** Realiza el control de la transmisión. Se distinguen dos formas de realizar la conexión, dependiendo de la necesidad o no de un flujo de datos garantizado entre estaciones:
 - Comunicación orientada a la conexión: se ofrece comunicación fiable extremo a extremo. Se usa, por ejemplo, en aplicaciones de telnet (terminal remoto) o FTP (transferencia de archivos). Es el protocolo utilizado por TCP.
 - Comunicación no orientada a la conexión: Las aplicaciones intercambian mensajes independientes unos de otros, es decir, no hay una interacción continua entre las aplicaciones. Es la forma de conexión utilizada por UDP.

El protocolo UDP se sitúa en el nivel de transporte, aunque no ofrece servicios fiables (una de las principales características de este nivel). La razón de situarlo en esta capa es que se basa en IP, al igual que TCP. Las características de estos protocolos son las siguientes:

- Características de TCP:
 - Tiene Servicios Orientados a Conexión con entrega fiable de datagramas.
 - Parte los mensajes en paquetes denominados *datagramas*.
 - Los datagramas se envían por separado y se reensamblan en destino.
- Características de UDP:
 - Tiene Servicios no Orientados a Conexión con entrega no fiable de datagramas.
 - Útil en aplicaciones donde importa la velocidad sobre la fiabilidad: sonido o vídeo.
- **Nivel de aplicación:** Los protocolos TCP/IP sólo implementan los niveles de red y transporte, por ello las aplicaciones finales deben implementar funciones propias de los protocolos de niveles superiores. Estos servicios utilizan un modelo cliente-servidor. Entre estos servicios, por ejemplo, se encuentran:
 - **Servicios de correo electrónico:**
 - SMTP (Simple Mail Transfer Protocol): transferencia entre cliente y servidor para el envío de correo electrónico.
 - POP3 (Post Office Protocol): para la lectura de los mensajes de correo electrónico desde el buzón.

- IMAP (Internet Message Access Protocol). Es una alternativa a POP3 para acceder al buzón de correo por Internet.
- **Ficheros compartidos:** FTP (File Transfer Protocol).
- **Navegadores WWW:** “http://”, “ftp://”, “telnet://”, “mailto:”, etc.

4.4.3. Direccionamiento y subdireccionamiento.

En TCP/IP se emplean las direcciones IP. El estándar que se utiliza actualmente para direcciones IP es el IPv4, que consta de cuatro bytes. El problema es que se están agotando direcciones para asignar a nuevos equipos. Para solucionarlo se está desarrollando un nuevo estándar (IPv6) en el que las direcciones constan de 16 bytes (128 bits), que en el futuro coexistirá con el IP4. También se está empleando protocolos de tipo CIDR (Classless Interdomain Routing) para alargar la existencia de IPv4 (esto se aclara más adelante).

La notación de las direcciones IP en IPv4 consiste en XXX.XXX.XXX.XXX, donde cada uno de los campos separados por puntos es un número decimal entre 0 a 255 (2^8), y que representa el byte correspondiente. Por ejemplo, la dirección 11001101 11000110 01000000 00000010 se escribe como 205.198.64.2. Las direcciones IP tienen dos partes:

- **Parte de red:** que identifica unívocamente la red física donde se ubica el sistema (*netid*), r primeros bits de la dirección.
- **Parte de host:** que identifica al host dentro de la red (*hostid*), h últimos bits de la dirección, donde $r + h = 32$.

	0	8	16	24	31	
	Byte 1		Byte 2	Byte 3	Byte 4	Valor byte 1
Clase A	0	<i>netid</i>	<i>hostid</i>			0-127
Clase B	10	<i>netid</i>		<i>hostid</i>		128-191
Clase C	110	<i>netid</i>			<i>hostid</i>	192-223
Clase D	1110	<i>Dirección multicast</i>				224-239
Clase E	11110	<i>Reservado</i>				240-255

Figura 4.11: Clases de redes según su dirección IP.

Los dígitos empleados para *netid* y para *hostid* dependen de la clase de la red que se determina por los primeros bits de dirección, tal como se muestra en la Figura 4.11. Las clases A, B y C son estándares para diálogos entre origen y destino, mientras que la clase D se emplea para el envío simultáneo a muchos destinos, y la clase E está reservada. Por ejemplo:

La dirección IP 205.198.64.2 es 11001101.11000110.01000000.00000010 en binario, de clase C porque comienza por 110 (Figura 4.11). En esta red se pueden asignar $2^8 - 2 = 254$ direcciones de host. No se pueden usar las direcciones de red y de difusión, de ahí que se resten 2.

Como puede verse, las direcciones de clase A utilizan 1 byte para el *netid* y 2 bytes para el *hostid*, las direcciones de clase B utilizan 2 bytes para el *netid* y 2 bytes para el *hostid*, y

las direcciones de clase C utilizan 3 bytes para el *netid* y 1 byte para el *hostid*. En base al ejemplo anterior se observa que se puede identificar la clase de dirección IP conociendo la misma.

Dentro del abanico de direcciones hay dos reservadas, que son la dirección de red y la dirección de difusión. La dirección de red se especifica indicando el *netid* con los campos de *hostid* a 0, por ejemplo, *132.147.0.0*. La dirección de difusión (broadcast) es como la de red pero con los bits de *hostid* a 1, en el caso anterior sería *132.147.255.255*. Un mensaje enviado a la dirección de difusión es recogido por todos los nodos de la red, con lo que se evitan sobrecargas. La dirección de difusión total es *255.255.255.255*. Esta dirección es identificada por el router como broadcast en todas las redes físicas a las que está conectado.

Existe una dirección especial que se denomina dirección de bucle de retorno (loopback o localhost). Es la dirección *127.0.0.1* y se usa para el diagnóstico del protocolo TCP/IP. Un paquete enviado a esta dirección será devuelto al mismo equipo por el protocolo IP en el nivel de red, es decir, si algo falla en la red, es posible saber si falla el protocolo o alguna otra cosa en el resto de la red. Si un paquete enviado a esta dirección es devuelto, el protocolo funciona adecuadamente, y el fallo estará en otro sitio. Para ello es necesaria la instrucción “*ping <dir IP>*” que envía un paquete a la dirección especificada y muestra los tiempos de envío del mismo.

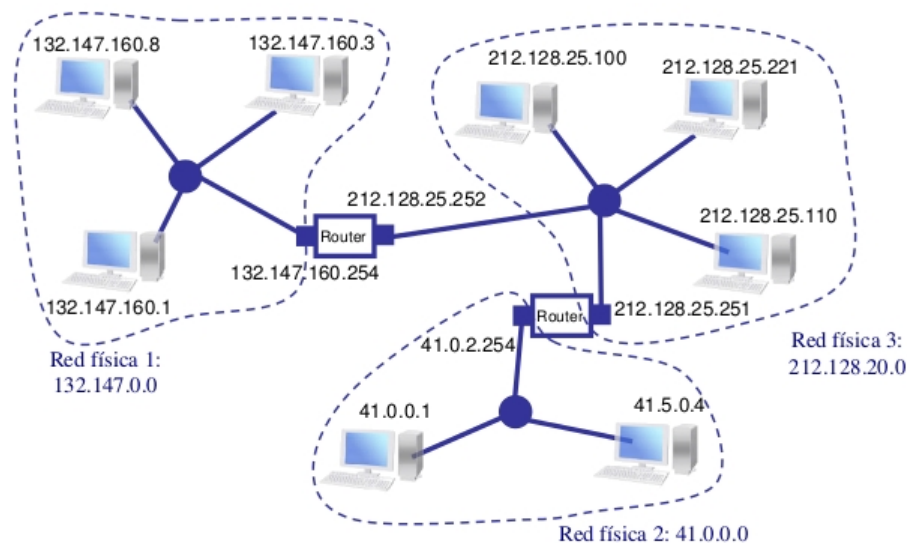


Figura 4.12: Ejemplo de redes de diferentes clases.

La Figura 4.12 muestra redes de diferentes clases. La red física 1 es de clase B pues tiene la dirección de red *132.147.0.0*, la red física 2 es de clase C pues tiene la dirección de red *212.128.20.0*, y finalmente, la red física 3 es de clase A pues tiene la dirección de red *41.0.0.0*. En este ejemplo, tanto la clase de red como la dirección de red se puede calcular a partir de la dirección IP de alguno de los host. Para calcular la dirección de red, primero se calcula el tipo de red a partir de la dirección IP, como se ha mostrado anteriormente, y después se copian de la dirección del host tantos bytes de los primeros como indique la clase y se ponen a cero el resto. En resumen, la dirección de la red se especifica indicando el *netid* con los campos de *hostid* a 0.

11010100 . 10000000 . 00010100 . 11111001	=	212.128.20.249
11111111 . 11111111 . 11111111 . 00000000	=	255.255.255.0
----- AND		
11010100 . 10000000 . 00010100 . 00000000	=	212.128.20.0 (Dir Red)

Tabla 4.2: Ejemplo de cálculo de dirección de red.

Este esquema de direccionamiento simplifica enormemente el trabajo de los encaminadores, ya que tan sólo tienen que tener en cuenta la porción de host de la dirección de destino de los datagramas.

Las direcciones IP deben seguir algunas reglas:

- No puede haber dos direcciones IP públicas iguales.
- Los hosts de una red deben pertenecer a la misma red (y por lo tanto a la misma clase).
- Cualquier nodo puede dialogar con cualquier otro especificando su dirección IP, aunque estén en redes diferentes.
- Los valores 0 y 255 no deben utilizarse para *hostid*, pues su uso está reservado para funciones especiales.

4.4.3.1. Asignación de direcciones.

La IANA (Internet Assigned Numbers Authority), dirección “<http://www.iana.org>”, es una autoridad central que se encarga de la asignación de las direcciones de red. Esta autoridad asigna sólo la porción de red, es la propia organización que la solicita quien se encarga de asignar las direcciones de host. La clase de red asignada (A, B o C) se hace en base a las necesidades de la organización en lo referente al número de hosts.

La máscara de red es en un patrón de 32 bits que permite obtener la parte de la dirección IP que corresponde a *netid* (dirección de red). Para ello se colocan a 1 todos los bits de la parte de la dirección IP que corresponden a *netid*. Por ejemplo, la dirección 212.128.20.249 con la máscara 255.255.255.0 indica que los primeros 3 bytes (clase C) identifican la dirección de red. A continuación se aplica una máscara mediante la orden *dir_IP AND mascara_red* (Tabla 4.2). La primera fila representa la dirección de un host, y la segunda la máscara que se debe usar, teniendo en cuenta que la dirección es de clase C. Finalmente, la última fila representa el resultado, donde los 3 primeros bytes son iguales a la dirección del host original y el último byte se asigna a 0. En este esquema de direccionamiento por clases, la máscara de red está determinada por la clase a la que pertenece la dirección del host:

1. Las direcciones de clase A usan la máscara 255.0.0.0.
2. Las direcciones de clase B usan la máscara 255.255.0.0.
3. Las direcciones de clase C usan la máscara 255.255.255.0.

En LANs grandes es usual que una red esté compuesta por varias LANs más pequeñas. En estos casos se utiliza un subdireccionamiento local dentro de la red que consiste en dividir el campo *hostid* en los campos *subnetid* y *hostid* de la siguiente forma:

10000100 . 10010011 . 00001010 . 00000010	=	132.147.10.2	dirección IP
11111111 . 11111111 . 11111111 . 00000000	=	255.255.255.0	máscara subred
AND			
10000100 . 10010011 . 00001010 . 00000000	=	132.147.10.0	dirección subred

Tabla 4.3: Ejemplo de cálculo de dirección de subred.

1. *netid*, que identifica a la red.
2. *subnetid*, que identifica a la subred.
3. *hostid*, que identifica a cada uno de los nodos en la subred.

El número de bytes empleado para el *netid* viene determinado por el tipo de red. El resto de la dirección IP, que previamente se utilizaba para el *hostid* se emplea para *subnetid* y *hostid*. Para poder obtener estos dos últimos se emplea la máscara de subred. La máscara de subred consiste en un patrón de 32 bits que rellena a 1 los bits de la parte de las direcciones IP que va a corresponder a los campos *netid* y *subnetid*, y a 0 la parte que corresponde al *hostid*. La Tabla 4.3 muestra un ejemplo de este cálculo. La primera fila representa la dirección IP del host, la segunda la máscara de subred a aplicarle (formada por 3 bytes, 2 de *netid* más 1 de *subnetid*), y finalmente, la última fila que tiene la dirección de subred obtenida. Como puede verse, está formada por los bytes de *netid* y *subnetid* copiados de la dirección original y el último byte con todos los bits a 0. La red es de clase B luego el *netid* estará compuesto por los dos primeros bytes. La máscara de subred indica que los tres primeros campos están dedicados a *netid* más *subnetid*. Puesto que los dos primeros correspondían a *netid*, el tercero habrá de ser el *subnetid*, y cuarto el *hostid*.

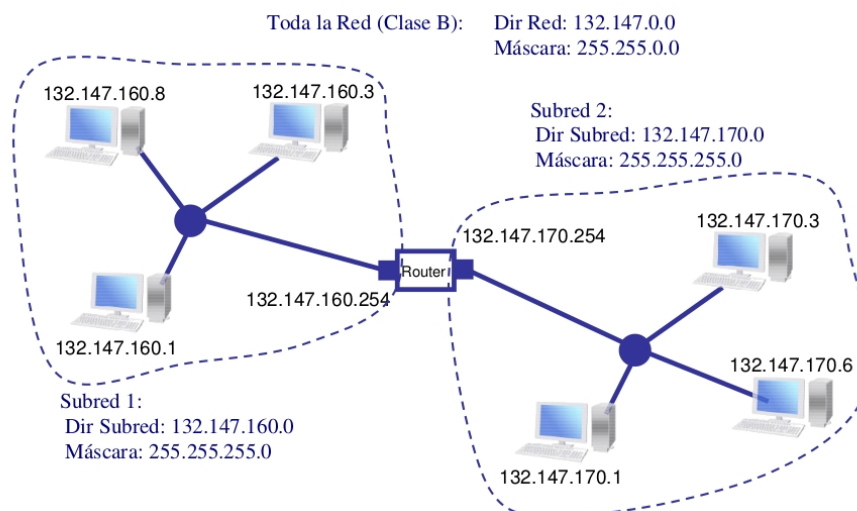


Figura 4.13: Ejemplo de máscara de subred.

La Figura 4.13 se muestra un ejemplo de dos subredes utilizando esta técnica de subdiseccionamiento:

- Se utilizan los dos primeros bytes para *netid*.

192.60.128.0	11000000.00111100.10000000.00000000	subred con supernetting
255.255.252.0	11111111.11111111.11111000.00000000	Máscara de subred
AND	-----	
192.60.128.255	11000000.00111100.10000000.00000000	Dir. De Red

Tabla 4.4: Ejemplo de Supernetting.

- El tercer byte corresponde a *subnetid*, dos subredes diferentes en nuestro caso.
- El cuarto byte es el *hostid*.

4.4.3.2. Direccionamiento CIDR.

CIDR (Classless Interdomain Routing) surgió para evitar que se agotaran las direcciones de Internet. El sistema de asignación de direcciones IP mediante el sistema de clases es poco eficiente, por ejemplo, si alguna organización necesita 300 direcciones se le asignan 65534 direcciones. Esto da lugar a que sólo una pequeña parte de las direcciones de clase A y B se han utilizado realmente en máquinas conectadas a Internet.

Eliminando el sistema de clases se puede asignar sólo la cantidad de direcciones que realmente le hacen falta a la organización, retrasando así el agotamiento de todas las direcciones IPv4. Esta solución se propuso 1992 y se denomina Supernetting (superredes). Por ejemplo, si se necesitan 1000 direcciones, se crea una superred a partir de cuatro redes de clase C (254 direcciones cada una). (Tabla 4.4). En el ejemplo, la subred 192.60.128.0 incluye todas las direcciones desde la 192.60.128.0 a la 192.60.131.255. En la representación binaria de la máscara de subred se observa que la parte de red tiene una longitud de 22 bits y la de host de 10 bits. Con CIDR la notación de máscara de subred se reduce considerablemente; en lugar de especificar la parte de subred se detalla el número de unos en que comienza la máscara. En el ejemplo anterior, en vez de escribir la dirección y la máscara de subred como

192.60.128.0, Subnet Mask 255.255.252.0

la dirección de red se escribe como

192.60.128.0/22

que indica la dirección inicial de la red y el número de unos en la máscara (22) que corresponden a la dirección de la red. El resto son para indicar el número de host), el valor binario de la máscara de subred es

11111111.11111111.11111100.00000000.

Esta notación permite expresar las direcciones con clases IP de la siguiente forma: Clase A = /8, Clase B = /16, y Clase C = /24.

Cuando se agoten las direcciones IPv4 se pasará al sistema IPv6 que permite un dominio de direcciones mucho más amplio.

4.4.3.3. Direccionamiento por dominios.

Existe un sistema de nombres de dominio que facilita la identificación de un host con el que se desee establecer comunicación debido a que las direcciones IP son incómodas de manejar. Estos dominios se organizan de forma jerárquica, luego la totalidad de las redes se ha dividido en dominios, que a su vez se subdividen en subdominios. Algunos de los dominios más utilizados son:

.com: previsto para empresas comerciales. Es el dominio más difundido en Internet.

.org: previsto para organizaciones sin ánimo de lucro, instituciones y fundaciones.

.net: previsto para empresas relacionadas con Internet.

.info: esta terminación de dominio es utilizada por sitios web cuyo principal cometido es la difusión o publicación de contenidos informativos.

.edu: los dominios con esta terminación son utilizados para fines educativos.

Dominios de países: *.es*, *.uk*, *.ar*, etc.

A cada uno de estos nombres de host direccionados por dominio le corresponde una dirección IP. Para la obtención de las direcciones IP equivalentes a estos nombres se utiliza una base de datos distribuida en diferentes ordenadores que se comunican de forma jerárquica. Así, si nuestro equipo necesita conocer una dirección IP para una dirección de dominio, la consultará a un servidor cuya dirección IP conoce, este servidor es un servidor DNS (Domain Name Server = Servidor de Nombres de Dominio). Si el nombre es desconocido para este servidor, lo consultará a otro servidor de nombres de dominio de orden superior, y así sucesivamente hasta obtenerla. Por ejemplo, el dominio *www.google.es* tiene asociada la dirección IP 173.194.34.216. Para direccionar dicho equipo se pueden utilizar indistintamente su dirección IP o su nombre de dominio, en cuyo caso se obtendrá la IP equivalente a través de un servidor DNS.

4.4.3.4. Puertos y Sockets en TCP/IP.

En sistema multitarea hay que establecer un mecanismo para determinar a qué proceso se entregan los mensajes recibidos por el protocolo TCP/IP. Para ello, en los protocolos TCP y UDP, del nivel de transporte, se establece una palabra de 16 bits (2 bytes) para identificar a la aplicación que se denomina *número de puerto* o simplemente *puerto*. Los primeros 1024 números de puerto (0 a 1023) están reservados por el sistema para servicios habituales del protocolo, y los valores por encima de éste pueden ser utilizados por aplicaciones del usuario.

Se denomina *socket* al par <dirección IP, número de puerto>. El socket identifica una conversación concreta realizada por una aplicación en una máquina, donde la dirección IP identifica a la máquina y el número de puerto a la aplicación. Los *sockets* se utilizan para comunicarse a la vez en más de una conexión. Por ejemplo:

155.54.16.20:80 ↔ 132.47.160.1:1048

155.54.16.20:80 ↔ 132.47.160.8:1201

155.54.16.20:80 ↔ 142.47.160.8:1300

UDP		
Nº Puerto	Servicio	Descripción
7	echo	Devuelve los datagramas al equipo
13	daytime	Hora del host remoto
53	nameserver	Nombre del host remoto
161	snmp	Monitoriza estado red
69	tftp	Protocolo Trivial de transferencia de ficheros
TCP		
7	echo	Devuelve los datagramas al equipo
21	ftp	Protocolo de transporte transferencia de ficheros
23	telnet	Emulación de terminal remoto
25	smtp	Correo electrónico
80	http	Protocolo de hipertexto

Tabla 4.5: Puertos más usados en TCP y UDP.

El puerto 80 es el utilizado para servicio http (web). El equipo 155.54.16.20 tiene un socket en el puerto 80, que corresponde a la aplicación del servidor web, y hay establecidas tres conexiones: una con el equipo 132.47.160.1 (con la aplicación navegador el puerto 1048), y otras dos con el equipo 142.47.160.8 (con aplicaciones para navegar en los puertos 1201 y 1300). Cada conexión se define por una pareja de sockets, uno en el origen y otro en el destino, así cada aplicación sabe a donde mandar los datos y desde donde los recibe. Los protocolos UDP y TCP funcionan con el sistema de puertos y pueden coexistir simultáneamente. La Tabla 4.5 muestra algunos de los puertos más utilizados, y que no se deben utilizar en el resto de aplicaciones.

versión	Long. Cabecera	Tipo de servicio	Longitud total	
Identificador			Flags	Offset Fragmento
Tiempo de vida	Protocolo		Código de redundancia de cabecera	
DIRECCIÓN FUENTE				
DIRECCIÓN DESTINO				
OPCIONES				
DATOS				

Figura 4.14: Estructura del datagrama IP.

4.4.3.5. Datagramas IP y tramas TCP y UDP

La Figura 4.14 muestra la estructura del **datagrama IP**, donde:

- **Versión:** Dado que los protocolos evolucionan con el tiempo existe este campo para indicar qué versión ha generado el datagrama.

- *Longitud*: Es la longitud de la cabecera medida en palabras de 32 bits (4 bytes), dado que este campo tiene 4 bits de longitud, la longitud máxima de la cabecera es de 64 octetos ($2^4 * 4$).
- *Servicio*: Lo rellena quien envía el datagrama. Su utilidad actual es muy escasa, pero irá aumentando en la medida en que se empleen diferentes tipos de tráfico. Tiene un formato interno que no se detallará.
- *Longitud total*: Es la longitud total del mensaje medida en octetos incluida la cabecera. Por ser un campo de 16 bits permite una longitud de hasta 65535 octetos (2^{16}).
- *Identificador o Numero de secuencia*. Es el mismo para todos los datagramas generados en la segmentación e igual al del datagrama original.
- *Offset*: Posición de los datos del datagrama segmentado en el original, se cuenta por octetos.
- *Flags*: son O, DT, MF. El más interesante es MF, que se pone a 0 si el datagrama es el último fragmento de una segmentación, en caso contrario estará a 1.
- *TTL (Time To Life)*: Limita el tiempo que un datagrama puede pasar en la red. TTL se decrementa en una unidad cada vez que pasa por un router si todo va bien, o en una unidad por segundo en el router si hay congestión. Al llegar a cero el datagrama es descartado.
- *Protocolo*: Especifica qué protocolo está por encima de IP, normalmente será TCP o UDP.
- *Checksum*: Es el resultado de aplicar un código de protección de errores a la cabecera con los bits del campo checksum puestos a cero. Normalmente, se suman todos los bits de la cabecera, se complementa la suma a uno y se pone el resultado en checksum. Este campo se modifica en cada router por decrementarse el campo TTL.
- *Opciones*: En este campo se especifican algunas opciones de las que se puede hacer uso. Por ejemplo, una de ellas es la denominada registro de ruta. Si se emplea esta opción todos los Routers por los que pase el datagrama copiarían en su campo de opciones su dirección. Como máximo este campo puede tener 40 octetos, es decir, 10 direcciones.

PUERTO FUENTE	PUERTO DESTINO
LONGITUD MENSAJE UDP	CÓDIGO REDUNDANCIA
DATOS	

Figura 4.15: Formato mensaje UDP.

La **trama UDP** consta de tan sólo 5 campos:

- *PF (2 bytes)*: Puerto de Origen.
- *PD (2 bytes)*: Puerto de Destino.

- *LONG (2 bytes)*: Longitud de datos.
- *CS (2 bytes)*: Check-Sum, para detección de errores.
- *DATOS*: campo de datos.

La trama contiene los puertos de origen y de destino. Toda la trama irá encapsulada después en el campo de datos de las tramas IP, en las que también se incluirán las direcciones IP origen y destino.

PUERTO FUENTE			PUERTO DESTINO		
NÚMERO DE SECUENCIA					
NÚMERO ACK					
LONGITUD	RESERVADO	CODE BITS	VENTANA		
CÓDIGO REDUNDANCIA			PUNTERO DATOS URGENTE		
OPCIONES				RELLENO	
DATOS					

Figura 4.16: Formato mensaje TCP.

La **trama TCP** es más compleja (Figura 4.16) y consta de 13 campos, entre los que hay algunos destinados a garantizar la entrega de paquetes en el orden correcto y suministrar las confirmaciones de entrega de tramas. sus campos son:

- *Puerto origen*: identifica el número de puerto que envió el paquete.
- *Puerto destino*: indica el número de puerto que recibe el paquete.
- *Número de secuencia*: identifica la posición del primer byte de datos del segmento en la secuencia de bytes de la máquina fuente.
- *Número de acuse de recibo*: el número que el emisor del acuse de recibo espera recibir en el siguiente paquete. Significa que se han recibido correctamente los bytes anteriores pero no se incluye ese byte.
- *Longitud cabecera*: especifica la longitud de la cabecera en palabras de 32 bits. El mínimo valor para la cabecera es de 5 palabras y el máximo de 15.
- *Reservado*: campo reservado para un futuro uso.
- *Código*: conjunto de 8 flags que representan diferentes aspectos relacionados con la congestión, sincronización y otros aspectos de la comunicación.
- *Ventana*: número de bytes que se pueden enviar sin haber recibido confirmación del primero de ellos.
- *Suma de verificación (Checksum)*: campo de 16 bits utilizado para comprobación de errores.

- *Puntero urgente*: si uno de los flags del campo Código así lo indica, este campo de 16 bits indica los bytes con mayor urgencia.
- *Opciones*: campo reservado para introducir diferentes opciones.

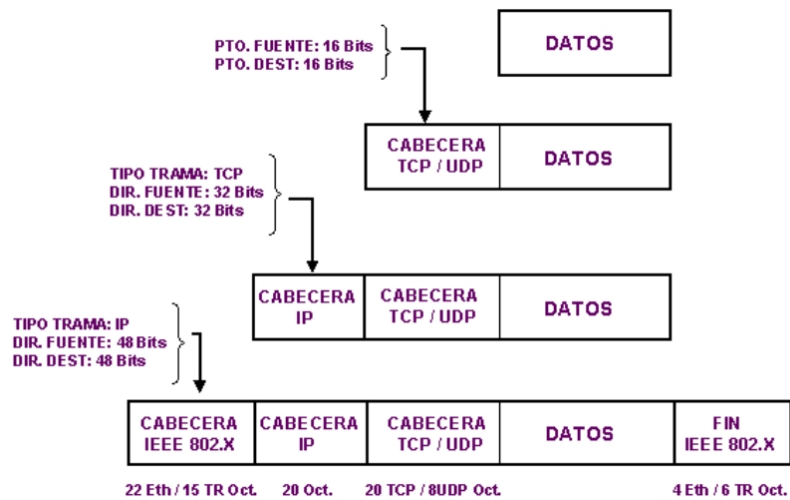


Figura 4.17: Encapsulado de mensaje de nivel de transporte.

Las tramas TCP ó UDP se encapsulan en el campo de datos de las tramas IP, y, a su vez, las tramas completas IP se encapsularán en el campo de datos de las tramas Ethernet II (Figura 4.17).

Bibliografía

- [1] Jose Maria Angulo, Javier García, and Ignacio Angulo. *Fundamentos y Estructuras de Computadores*. Thomson, 2003.
- [2] Alan Burns and Andy Wellings. *Sistemas de tiempo real y lenguajes de programación*. Pearson Addison-Wesley, 2003.
- [3] Yiseth Giraldo. *Los periféricos*. <http://www.monografias.com/trabajos5/perif/perif.shtml>. (Último acceso 26-09-12), 2008.
- [4] Pedro de Miguel Anasagasti. *Fundamentos de computadores*. Ediciones Paraninfo S.A., 2004.
- [5] Aquilino Rodríguez Penin. *Sistemas SCADA : guía práctica*. Marcombo, 2007.